

Оглавление

Abstract.....	6
Реферат	7
Введение	8
1 Постановка задачи и аналитический обзор аналогичных решений	9
1.1 Постановка задачи.....	9
1.2 Аналитический обзор аналогов	9
1.2.1 Сервис Ultimate Guitar	9
1.2.2 Веб-приложение «Songsterr».....	10
1.3 Выводы по разделу	11
2 Проектирование веб-приложения	12
2.1 Диаграмма вариантов использования	12
2.2 Логическая схема базы данных PostgreSQL	13
2.3 Архитектура веб-приложения.....	18
2.4 Блок-схема алгоритма создания аккорда	18
2.5 Проектирование диаграммы последовательности.....	19
2.6 Выводы по разделу	20
3 Разработка веб-приложения.....	22
3.1 Обзор технологий реализации веб-приложения	22
3.2 Описание используемых паттернов проектирования.....	23
3.3 ORM Entity Framework	24
3.4 Реализация функций для гостя	33
3.4.1 Реализация функции регистрации	33
3.4.2 Реализация функции просмотра публичных песен	35
3.4.3 Реализация функции просмотра публичных аккордов.....	36
3.4.4 Реализация функции просмотра публичных паттернов	37
3.4.5 Реализация функции фильтрация и сортировка контента.....	39
3.5 Реализация функций для гитариста.....	40
3.5.1 Реализация функции просмотр своего и публичного контента	40
3.5.2 Реализация функции просмотра своих и чужих публичных паттернов	40
3.5.3 Реализация функции просмотра своих и публичных аккордов	41
3.5.4 Реализация функции просмотра своих и публичных песен	42
3.5.5 Реализация функции просмотра своих и публичных альбомов....	42
3.5.6 Реализация функции добавления или удаления песни в или из избранного	44
3.5.7 Реализация функции написания отзыва и выставление оценки ...	45

					ДП 00.00.ПЗ			
		ФИО	Подпись	Дата	Оглавление			
Разраб.		Селицкий Н.Е.						
Пров.		Харланович А.В.						
Н. контр.		Авдеева В.Д.						
Утв.		Смелов В.В.			БГТУ 1-40 01 01, 2025			
					Лит. Лист Листов			
					У		1	3

3.5.8 Реализация функции создание, обновление и удаление контента с лимитами	46
3.5.9 Реализация функций создание или удаление подписки	47
3.5.10 Реализация функций создания, обновления и удаления аккордов	49
3.5.11 Реализация функций создания, обновления и удаления паттернов	50
3.5.12 Реализация функций создания, обновления и удаления песен ...	50
3.5.13 Реализация функций создания, обновления и удаления аудио для песен	51
3.5.14 Реализация функций покупки премиума.....	52
3.5.15 Реализация функций аутентификация без привилегий	53
3.6 Реализация функций для премиума.....	54
3.6.1 Реализация функций создания, обновления и удаления контента без лимита	54
3.6.2 Реализация функций создания, обновления и удаления альбома .	55
3.6.3 Реализация функций аутентификация с привилегиями	56
3.7 Реализация функций для администратора	56
3.7.1 Реализация функции выдачи или изъятия прав администратора	56
3.7.2 Реализация функций блокировки и разблокировки пользователей	58
3.7.3 Реализация функции просмотра любого контента	60
3.7.4 Реализация функции удаления любого контента.....	60
3.8 Маршруты контроллеров	61
3.9 Реализация клиентской части при помощи библиотеки React	63
3.9.1 Реализация структуры проекта	63
3.9.2 Реализация компонент.....	64
3.9.3 Реализация хранилища при помощи библиотеки Redux ToolKit..	65
3.10 Используемые форматы передачи данных.....	66
3.11 Выводы по разделу	66
4 Тестирование веб-приложения	67
4.1 Функциональное тестирование.....	68
4.2 Модульное тестирование	72
4.3 Выводы по разделу	75
5 Руководство пользователя	76
5.1 Руководство клиента	76
5.1.1 Регистрация.....	76
5.2 Руководство Гитариста	77
5.2.1 Просмотреть песен	77
5.2.2 Создание песни	78
5.2.3 Запись аудио	78
5.2.4 Просмотр информации о песне.....	79

5.2.5 Оставить комментарий	79
5.2.6 Просмотр альбомов	80
5.2.7 Создание альбома	80
5.3 Руководство администратора	81
5.3.1 Просмотр всей информации.....	81
5.3.2 Управление пользователями	82
5.4 Выводы по разделу	84
6 Технико-экономическое обоснование проекта.....	85
6.1 Общая характеристика разрабатываемого программного средства	85
6.2 Исходные данные для проведения расчетов для продажи.....	85
6.3 Обоснование цены программного средства.....	88
6.3.1 Расчет затрат рабочего времени на разработку программного средства.....	88
6.3.2 Расчет основной заработной платы	89
6.3.3 Расчет дополнительной заработной платы	90
6.3.4 Расчет отчислений в Фонд социальной защиты населения и по обязательному страхованию	90
6.3.5 Расчет суммы прочих прямых затрат	91
6.3.6 Расчет суммы накладных расходов	91
6.3.7 Сумма расходов на разработку программного средства	92
6.3.8 Расходы на сопровождение и адаптацию	92
6.3.9 Расчет полной себестоимости.....	92
6.3.10 Определение цены, оценка эффективности	93
6.4 Выводы по разделу	94
Заключение	95
Список используемых источников.....	96
Диаграмма вариантов использования ДП 01.00 ГЧ	98
Логическая схема базы данных ДП 02.00 ГЧ	99
Диаграмма развертывания ДП 03.00 ГЧ	100
Диаграмма классов ДП 04.00 ГЧ.....	101
Блок схема функции отправки сообщения ДП 05.00 ГЧ	102
Скриншот-работа ДП 06.00 ГЧ	103
ПРИЛОЖЕНИЕ А	104
ПРИЛОЖЕНИЕ Б.....	111
ПРИЛОЖЕНИЕ В	112
ПРИЛОЖЕНИЕ Г.....	113
ПРИЛОЖЕНИЕ Д.....	114
ПРИЛОЖЕНИЕ Е	115

Abstract

The explanatory note contains 113 pages, 21 figures, 26 tables, and 28 references.

ASP.NET Core, React, Next.js, TypeScript, PostgreSQL, Entity Framework Core, Tailwind CSS, JWT, MediatR.

The goal of this thesis is to develop a web application for creating, storing, and sharing guitar parts, chords, strumming patterns, and albums. The thesis summary consists of an abstract, table of contents, six chapters, a conclusion, and a list of references.

The first section describes the tasks addressed in developing the software, examines web service alternatives for guitarists, highlights the advantages and disadvantages of each, and provides an overview of theoretical material on web application design.

The second section describes the software tools and technologies selected for implementing the web application. The selection of widely used and supported solutions for the server and client components is substantiated.

Section three describes the development process of the Guitar Notepad web application. Source code fragments with explanations are provided, along with flowcharts of the main algorithms and interactions between the system components.

Section four presents the results of web application testing: functional checks, unit tests, and API integration tests.

Section five provides a user guide for the system for guest, user, Premium owner, and administrator roles.

Section six calculates the economic parameters and cost of the software, taking into account the subscription monetization model and the paid Premium plan.

The conclusion summarizes the results of the work performed and formulates recommendations for the practical use of the developed software.

The graphic section consists of six A3-sized diagrams.

					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата	Abstract		
Разраб.		Селицкий Н.Е.					
Пров.		Харланович А.В.					
Н. контр.		Авдеева В.Д.					
Утв.		Смелов В.В.			БИТУ 1-40 01 01, 2025		
					Лит.	Лист	Листов
					У	1	1

Реферат

Пояснительная записка содержит 113 страниц, 21 рисунок, 26 таблиц, 28 литературных источников.

ASP.NET Core, React, Next.js, TypeScript, PostgreSQL, Entity Framework Core, Tailwind CSS, JWT, MediatR.

Целью работы является разработка веб-приложения для создания, хранения и совместного использования гитарных партий, аккордов, паттернов боя и альбомов. Пояснительная записка дипломного проекта состоит из реферата, оглавления, шести глав, заключения и списка использованных источников.

В первом разделе описаны задачи, решаемые при разработке программного средства, рассмотрены аналоги веб-сервисов для гитаристов, выделены достоинства и недостатки каждого из них, дан обзор теоретического материала по проектированию веб-приложений.

Во втором разделе описаны программные средства и технологии, выбранные для реализации веб-приложения. Обоснован выбор распространённых и сопровождаемых решений для серверной и клиентской частей.

В третьем разделе приведено описание процесса разработки программного обеспечения веб-приложения «*Guitar Notepad*». Представлены фрагменты исходного кода с пояснениями, а также блок-схемы основных алгоритмов и взаимодействий компонентов системы.

В четвёртом разделе приведены результаты тестирования веб-приложения: функциональные проверки, модульные и интеграционные *API*-тесты.

В пятом разделе приведено руководство пользователя по работе с системой для ролей гостя, пользователя, владельца *Premium* и администратора.

В шестом разделе выполнен расчёт экономических параметров и себестоимости программного средства с учётом модели монетизации через подписки и платный тариф *Premium*.

В заключении обобщены результаты выполненной работы и сформулированы рекомендации по практическому использованию разработанного программного средства.

Объем графической части составляет шесть схем формата листов А3.

					ДП 00.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.		Селицкий Н.Е.			Реферат	Лит.	Лист	Листов
Пров.		Харланович А.В.				У	1	1
						БГТУ 1-40 01 01, 2025		
Н. контр.		Авдеева В.Д.						
Утв.		Смелов В.В.						

Введение

Веб-приложение «Гитарный блокнот» – это программное обеспечение, предназначенное для создания, систематизации и совместного использования музыкальных материалов для гитаристов. Оно предоставляет пользователям единую цифровую среду для работы с аккордами, паттернами (боем, перебором), а также объединять песни в альбомы.

Данное решение направлено на устранение разрозненности музыкальных материалов, с которой часто сталкиваются гитаристы при использовании традиционных инструментов. «Гитарный блокнот» консолидирует все ключевые элементы творческого процесса в одном месте, позволяя хранить не только последовательности аккордов, но и привязывать к ним конкретные ритмические рисунки, а также сопровождающий текст. Такой подход упрощает как начальное обучение, так и профессиональную работу над композициями.

Благодаря такой структуре «Гитарный блокнот» становится не просто хранилищем, а полноценным творческим инструментом для различных людей и разных целях. Пользователь может быстро подобрать подходящий паттерн к аккордовой прогрессии, поэкспериментировать с разными вариантами боя или перебора, не переписывая каждый раз всю песню заново.

Цели проекта:

- Приложение должно предоставлять интуитивно понятный интерфейс для создания и управления библиотеками аккордов и паттернов игры, являющихся фундаментом для написания песен.

- Система должна позволять пользователям комбинировать аккорды, паттерны и текст для создания структурированных песен, с возможностью их организации в публичные или приватные альбомы.

- Приложение должно обеспечивать функции социального взаимодействия: поиск и просмотры, и сохранение чужих публичных песен.

- Для достижения поставленной цели были определены следующие основные задачи, которые нужно реализовать:

- Целевая аудитория проекта включает гитаристов всех уровней – от начинающих, ищущих готовые материалы для разбора и обучения, до опытных музыкантов и композиторов, нуждающихся в удобном инструменте для записи песен.

- В качестве программной платформы для серверной части выбрана платформа ASP.NET Core [1]. Она обеспечивает высокую производительность, модульность и кроссплатформенность, что является оптимальным для разработки надежного и масштабируемого *web-API*. Для клиентской части используется фреймворк *Next.js* [2].

					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата	Введение		
Разраб.		Селицкий Н.Е.					
Пров.		Харланович А.В.					
Н. контр.		Авдеева В.Д.					
Утв.		Смелов В.В.			БГТУ 1-40 01 01, 2025		
					Лит.	Лист	Листов
					У	1	1

1 Постановка задачи и аналитический обзор аналогичных решений

1.1 Постановка задачи

Веб-приложение «Гитарный блокнот» строится на системе ролевого доступа, включающей три ключевые категории пользователей: гость, гитарист и администратор.

Гость может находиться только на страницах регистрации и аутентификации. После регистрации и аутентификации пользователь переходит в статус гитарист. Ему становятся доступны инструменты для просмотра и создания музыкальных материалов: аккорды, паттерны игры и песни. Пользователь может комбинировать созданные песни в тематические альбомы, настраивая их видимость – приватные или публичные. Социальный функционал позволяет находить и изучать чужие песни, а также сохранять понравившиеся материалы в персональное избранное.

Администраторская роль обеспечивает доступ к просмотру всех данных. Администратор может аутентифицироваться в системе, просматривать и удалять данные – от отдельных аккордов до готовых альбомов, управлять учетными записями участников, включая приостановку доступа, а также выдачу прав администратора.

1.2 Аналитический обзор аналогов

В рамках данной работы будет проведен анализ существующих аналогов, рассмотрены их сильные и слабые стороны, а на основе этого анализа будут выделены и сформулированы требования к разрабатываемому программному средству.

1.2.1 Сервис Ultimate Guitar

Ultimate Guitar – это крупнейший в мире онлайн-сервис и мобильное приложение для гитаристов, содержащий миллионы аккордов, табулатур и нот к популярным песням всех жанров. Он создан для того, чтобы музыканты любого уровня могли быстро найти точную схему аккордов любимой композиции, разобрать сложное соло или ритмический рисунок, не тратя часы на подбор «на слух». Главная задача сервиса – заменить бумажные сборники песен и помочь сыграть любую музыку прямо сейчас, предоставляя удобные инструменты для смены тональности, замедления темпа и наглядного отработки партий шаг за шагом.

					ДП 01.00.ПЗ		
		ФИО	Подпись	Дата	1 Постановка задачи и аналитический обзор аналогичных решений		
Разраб.		Селицкий Н.Е.					
Пров.		Харланович А.В.					
Н. контр.		Авдеева В.Д.					
Утв.		Смелов В.В.					
					Лит.	Лист	Листов
					У	1	3
					БГТУ 1-40 01 01, 2025		

На рисунке 1.1 представлен скриншот крупнейшего в мире сообщественного архива гитарных разборов «Ultimate Guitar» [3].

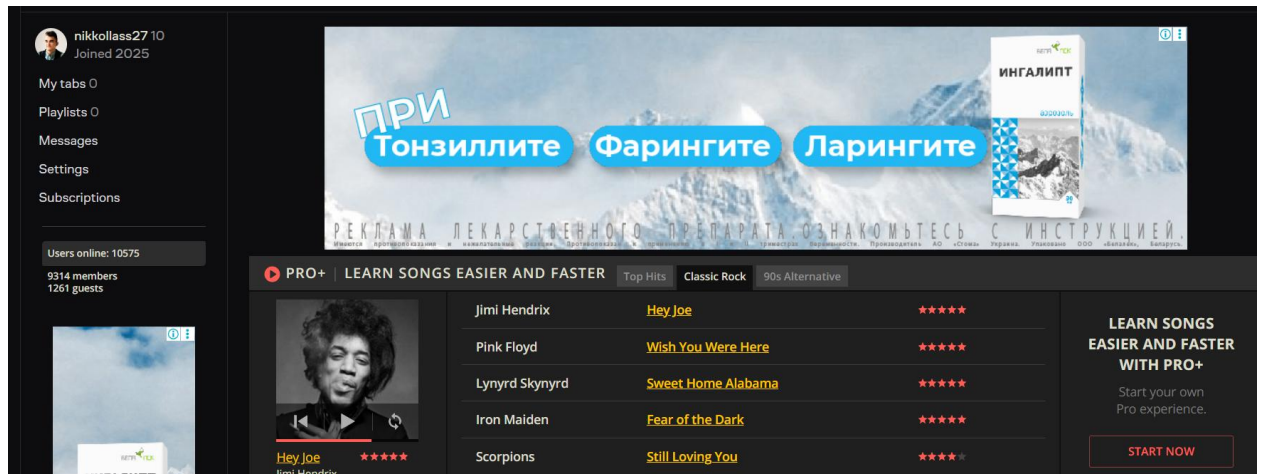


Рисунок 1.1 – Интерфейс «Ultimate Guitar»

Преимущества этого сервиса:

- простота использования: интуитивно понятный интерфейс, позволяющий быстро найти необходимые песни по оценкам и названиям;
- удобная возможность создавать наборы аккордов для использования их сразу в нескольких песнях;
- удобная привязка аккорда к тексту и графическое отображение.

Недостатки:

- неудобное отображение паттерна игры, который больше предполагает, что пользователь сам для себя решит, что лучше;
- нет возможности создавать свою песню.

Это стандартный сервис, предполагающий базовые, но ограниченные возможности для создания песен.

1.2.2 Веб-приложение «Songsterr»

Songsterr[4] – это онлайн-сервис и мобильное приложение, специализирующееся на высококачественных табулатурах для гитары, баса, укулеле и барабанов, каждая из которых сопровождается интерактивным *MIDI*-плеером. Главное отличие *Songsterr* от обычных сборников аккордов в том, что табы здесь не просто статичный текст, а «живая» дорожка с проигрыванием: нота за нотой, с правильным ритмом и расставленными паузами. Сервис идеально подходит для гитаристов, которым важно услышать, как именно звучит партия, а не просто увидеть аппликатуры – можно замедлить проигрывание, выключить гитарную дорожку (сыграть вместо плеера) и учить песню под караоке-формат для игры в реальном времени.

На рисунке 1.2 представлен скриншот приложения «Songsterr».

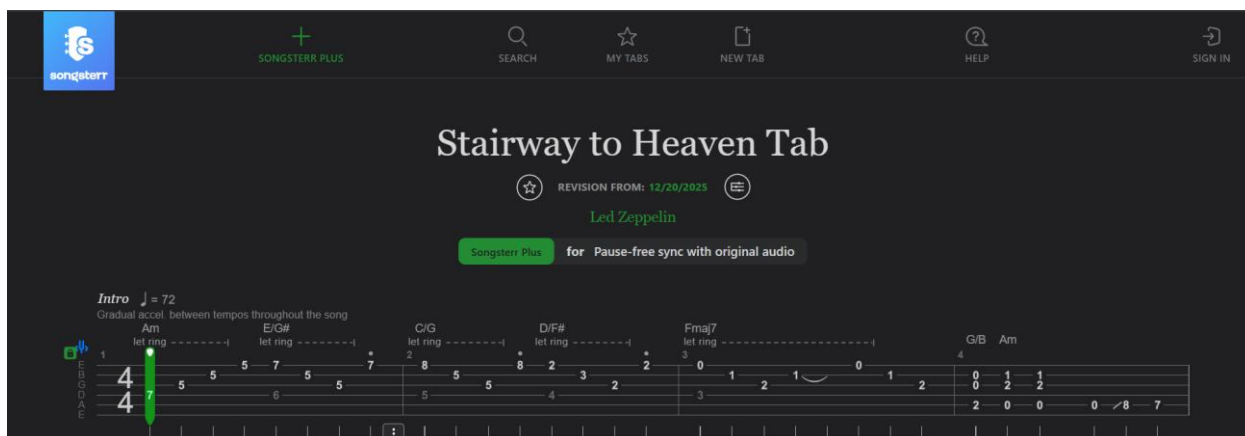


Рисунок 1.2 – Интерфейс «Songsterr»

Преимущества данного веб-приложения:

- очень удобный интерфейс отображения паттерном игры;
- есть возможность выбирать разные паттерны игры и видоизменять их;
- есть возможность прослушивать песни через аудио.

Недостатки приложения:

- плохая оптимизация, каждая песня, аккорд и паттерн загружаться и рисуется очень долго;
- нет возможности сохранять песни в альбомы или в избранное из-за чего каждый раз нужно искать песню заново.

Имеет гораздо лучшую логику взаимодействия с пользователем, но сложно использовать для постоянной игры.

1.3 Выводы по разделу

Анализ существующих на рынке решений выявил чёткое разделение на два типа сервисов: многофункциональные, но перегруженные общественные архивы и специализированные, но ограниченные проигрыватели. Первые, как «*Ultimate Guitar*», страдают от информационного шума, избыточного контента и отсутствия структурированного подхода – пользователь получает разрозненные табулатуры и аккорды без привязки к целостной композиции или альбому. Вторые, как «*Songsterr*», решают узкий круг задач (например, проигрывание отдельной партии с возможностью замедления и визуального отображения нот), но совершенно не охватывают полный цикл работы музыканта – от подбора первых аккордов и паттернов до финального оформления и организации готовых песен. В итоге оба типа решений не предоставляют пользователю целостной, структурированной и персонализированной цифровой среды, которая бы начиналась с создания базового музыкального элемента и логически завершалась формированием готового альбома, объединяющего несколько композиций в единую структуру.

2 Проектирование веб-приложения

2.1 Диаграмма вариантов использования

Для описания основных сценариев взаимодействия пользователей с веб-приложением была разработана диаграмма вариантов использования. Диаграмма демонстрирует ключевые роли в системе и действия, доступные для каждой из них.

На диаграмме отражены все действующие в приложении роли и функции, которые доступны этим ролям. Описание ролей представлено в таблице 2.1.

Таблица 2.1 – Описание ролей

Роль	Описание
Гость	Может только просматривать чужой публичный контент или зарегистрироваться и получить роль гитариста.
Гитарист	Обладает правами лимитированного создания контента, просмотр и сохранение своего или чужого публичного контента в избранное, подписываться на альбомы, комментирование и оценивание чужого контента, покупки премиума для получения роли премиум.
Премиум	Обладает всеми правами гитариста, но в безлимитном варианте, так же имеет права на создание альбомов.
Администратор	Обладает правами просматривать любой, даже приватный, контент, удалять любой контент, а также блокировать и разблокировать пользователей, выдавать или изымать права администратора у пользователя.

Таким образом, на основе каждой роли и её функциональных прав можно выделить соответствующие варианты использования. Для роли «Гость» – это просмотр контента и регистрация; для «Гитариста» – создание контента, избранное, подписки, комментарии, оценки и покупка премиума; для «Премиум» – те же действия без лимитов плюс создание альбомов; для «Администратора» – удаление любого контента, блокировка пользователей и управление правами.

Каждое действие из описания роли становится отдельным прецедентом. Следовательно, имея перед глазами таблицу с ролями и их возможностями, мы можем перейти к построению диаграммы вариантов использования в нотации UML. Эта диаграмма визуально покажет всех акторов (Гость, Гитарист, Премиум, Администратор), границы системы и связи между прецедентами, наглядно отразив, например, что права Премиума наследуют права Гитариста. Таблица 2.1, таким образом, служит содержательной основой для последующей графической модели.

					ДП 02.00.ПЗ		
		ФИО	Подпись	Дата	2 Проектирование веб-приложения		
Разраб.		Селицкий Н.Е.					
Пров.		Харланович А.В.					
Н. контр.		Авдеева В.Д.					
Утв.		Смелов В.В.					
					Лит.	Лист	Листов
					У	1	10
					БГТУ 1-40 01 01, 2025		

Функциональность веб-приложения представлена на диаграмме вариантов использования. Часть диаграммы вариантов использования представлена на рисунке 2.1.



Рисунок 2.1 – Часть диаграммы вариантов использования

В ДП 01.00 ГЧ представлена полная диаграмма вариантов использования.

2.2 Логическая схема базы данных PostgreSQL

Диаграмма базы данных таблиц (*Database Table Diagram*) – это визуальное представление структуры базы данных и отношений между ее таблицами.

Для веб-приложения была спроектирована база данных, включающая 14 таблиц, обеспечивающих хранение информации о пользователях, аккордах, паттернах, песнях и прочих объектах. Логическая схема показана в ДП 02.00 ГЧ.

Таблица *Users* содержит данные о пользователях, включая их уникальные идентификаторы, пароль, роль и прочие данные.

Таблица *Chords* содержит данные об аккордах.

Таблица *StrummingPatterns* содержит данные об паттернах игры.

Таблица *Songs* содержит общую информацию об песнях, такую как название, автора, жанр и т.д.

Таблица *Notifications* содержит информацию об уведомлениях.

Таблица *Subscriptions* связующая между пользователем и альбомом, на который он подписан.

Таблица *Albums* содержит информацию об альбомах.

Таблица *SongReviews* содержит отзыв определенного пользователя об определенной песне.

Таблица *SongComments* содержит комментарии к песне от ее создателя.
 Таблица *SongPatterns* связующая между паттернами и песнями.
 Таблица *SongSegments* содержит информацию об сегментах песни.
 Таблица *SongChords* связующая между аккордами и песнями.
 Таблица *SongAlbums* связующая между альбомами и песнями.
 Таблица *SongSegmentPositions* содержит информацию об сегментах.
 Далее будут приведены более подробные описания таблиц и их полей.
 В таблице 2.2 показано описание полей таблицы «*Users*».

Таблица 2.2 – Описание структуры таблицы «*Users*»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор пользователя.
<i>Email</i>	<i>VARCHAR(255)</i>	Уникальный адрес почты.
<i>NikName</i>	<i>VARCHAR(255)</i>	Уникальное имя пользователя.
<i>PasswordHash</i>	<i>VARCHAR(255)</i>	Хешь пароля.
<i>Role</i>	<i>VARCHAR(255)</i>	Роль пользователя.
<i>AvatarUrl</i>	<i>VARCHAR(255)</i>	Адрес аватарки пользователя.
<i>Bio</i>	<i>VARCHAR(255)</i>	Описание пользователя.
<i>HasPremium</i>	<i>BOOLEAN</i>	После есть ли у пользователя подписка.
<i>BlockReason</i>	<i>VARCHAR(255)</i>	Причина, по которой заблокирован пользователь.
<i>BlockUntil</i>	<i>TIMESTAMP</i>	Дата до которой заблокирован пользователь.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания пользователя.

В таблице 2.3 показано описание полей таблицы «*Chords*».

Таблица 2.3 – Описание структуры таблицы «*Chords*»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор аккорда.
<i>OwnerId</i>	<i>UUID</i>	Уникальный идентификатор создателя.
<i>Name</i>	<i>VARCHAR(255)</i>	Название аккорда.
<i>Fingering</i>	<i>VARCHAR(100)</i>	Расположения пальцев.
<i>Description</i>	<i>VARCHAR(255)</i>	Описание аккорда.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания аккорда.

В таблице 2.4 показано описание полей таблиц «*StrummingPatterns*».

Таблица 2.4 – Описание структуры таблицы «*StrummingPatterns*»

Поле	Тип данных	Описание
1	2	3
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор паттерна.
<i>OwnerId</i>	<i>UUID</i>	Уникальный идентификатор создателя.
<i>Name</i>	<i>VARCHAR(255)</i>	Название паттерна.
<i>Pattern</i>	<i>VARCHAR(255)</i>	Как играть паттерн.

Продолжение таблицы 2.4

1	2	3
<i>Description</i>	<i>VARCHAR(255)</i>	Описание паттерна.
<i>IsFingerStyle</i>	<i>BOOLEAN</i>	Значение – это бой или перебор.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания паттерна.

В таблице 2.5 показано описание полей таблицы «Songs».

Таблица 2.5 – Описание структуры таблицы «Songs»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор песни.
<i>OwnerId</i>	<i>UUID</i>	Уникальный идентификатор создателя.
<i>ParantSongId</i>	<i>UUID</i>	Уникальный идентификатор родительской песни.
<i>Title</i>	<i>VARCHAR(255)</i>	Название песни.
<i>Artist</i>	<i>VARCHAR(255)</i>	Автор песни.
<i>Description</i>	<i>VARCHAR(255)</i>	Описание песни.
<i>IsPublic</i>	<i>BOOLEAN</i>	Значение, публичная ли песня.
<i>Genre</i>	<i>VARCHAR(255)</i>	Жанр песни.
<i>Theme</i>	<i>VARCHAR(255)</i>	Тема песни.
<i>CustomAudioType</i>	<i>VARCHAR(255)</i>	Адрес аудио для песни.
<i>FullText</i>	<i>VARCHAR(500)</i>	Полный текст песни.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания песни.

В таблице 2.6 представлена структура таблицы «Notifications».

Таблица 2.6 – Описание структуры таблицы «Notifications»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор оповещения.
<i>UserId</i>	<i>UUID</i>	Уникальный идентификатор создателя.
<i>SongId</i>	<i>UUID</i>	Уникальный идентификатор песни с изменением.
<i>AlbumId</i>	<i>UUID</i>	Уникальный идентификатор альбома с изменением.
<i>Type</i>	<i>VARCHAR(255)</i>	Тип уведомления.
<i>Message</i>	<i>VARCHAR(255)</i>	Сообщение уведомления.
<i>IsRead</i>	<i>BOOLEAN</i>	Значение, прочитано ли уведомление.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания уведомления.

В таблице 2.7 представлена структура таблицы «Subscriptions».

Таблица 2.7 – Описание структуры таблицы «Subscriptions»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор подписки.
<i>UserId</i>	<i>UUID</i>	Уникальный идентификатор создателя.
<i>TargetId</i>	<i>UUID</i>	Уникальный идентификатор альбома.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания подписки.

В таблице 2.8 представлена структура таблицы «Albums».

Таблица 2.8 – Описание структуры таблицы «Albums»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор альбома.
<i>OwnerId</i>	<i>UUID</i>	Уникальный идентификатор создателя.
<i>Title</i>	<i>VARCHAR(255)</i>	Название альбома.
<i>CoverUrl</i>	<i>VARCHAR(255)</i>	Адрес обложки альбома.
<i>Description</i>	<i>VARCHAR(255)</i>	Описание альбома.
<i>IsPublic</i>	<i>BOOLEAN</i>	Значение, публичный ли альбом.
<i>Genre</i>	<i>VARCHAR(255)</i>	Жанр альбома.
<i>Theme</i>	<i>VARCHAR(255)</i>	Тема альбома.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания альбома.

В таблице 2.9 представлено описание структура таблицы «SongReviews».

Таблица 2.9 – Описание структуры таблицы «SongReviews»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор отзыва.
<i>UserId</i>	<i>UUID</i>	Уникальный идентификатор создателя.
<i>SongId</i>	<i>UUID</i>	Уникальный идентификатор песни.
<i>ReviewText</i>	<i>VARCHAR(255)</i>	Текст отзыва.
<i>BeautifulLevel</i>	<i>INTEGER</i>	Оценка красоты.
<i>DifficultyLevel</i>	<i>INTEGER</i>	Оценка сложности.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания отзыва.

В таблице 2.10 описание структуры таблицы «SongComments».

Таблица 2.10 – Описание структуры таблицы «SongComments»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор комментария.
<i>UserId</i>	<i>UUID</i>	Уникальный идентификатор создателя.
<i>SongId</i>	<i>UUID</i>	Уникальный идентификатор песни.
<i>SegmentId</i>	<i>UUID</i>	Уникальный идентификатор сегмента.
<i>Text</i>	<i>VARCHAR(255)</i>	Текст комментария.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания комментария.

В таблице 2.11 представлено описание структуры таблицы «SongPatterns».

Таблица 2.11 – Описание структуры таблицы «SongPatterns»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор связи.
<i>SongId</i>	<i>UUID</i>	Уникальный идентификатор песни.
<i>StrummingPatternId</i>	<i>UUID</i>	Уникальный идентификатор паттерна.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания связи.

В таблице 2.12 описание структуры таблицы «SongSegments».

Таблица 2.12 – Описание структуры таблицы «SongSegments»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор сегмента.
<i>ChorId</i>	<i>UUID</i>	Уникальный идентификатор аккорда.
<i>PatternId</i>	<i>UUID</i>	Уникальный идентификатор паттерна.
<i>Type</i>	<i>VARCHAR(255)</i>	Тип сегмента.
<i>Lyric</i>	<i>VARCHAR(255)</i>	Текст сегмента.
<i>Duration</i>	<i>INTAGER</i>	Продолжительность сегмента.
<i>Description</i>	<i>VARCHAR(255)</i>	Описание сегмента.
<i>Color</i>	<i>VARCHAR(255)</i>	Цвет текст.
<i>BackgroundColor</i>	<i>VARCHAR(255)</i>	Цвет фона.
<i>ContentHash</i>	<i>VARCHAR(255)</i>	Хешь сегмента.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания сегмента.

В таблице 2.13 представлено описание структуры таблицы «SongChords».

Таблица 2.13 – Описание структуры таблицы «SongChords»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор связи.
<i>SongId</i>	<i>UUID</i>	Уникальный идентификатор песни.
<i>ChordId</i>	<i>UUID</i>	Уникальный идентификатор аккорда.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания связи.

В таблице 2.14 представлено описание структуры таблицы «SongAlbums».

Таблица 2.14 – Описание структуры таблицы «SongAlbums»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор связи.
<i>SongId</i>	<i>UUID</i>	Уникальный идентификатор песни.
<i>AlbumId</i>	<i>UUID</i>	Уникальный идентификатор альбома.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания связи.

В таблице 2.15 представлено описание таблицы «SongSegmentPositions».

Таблица 2.15 – Описание структуры таблицы «SongSegmentPositions»

Поле	Тип данных	Описание
<i>Id</i>	<i>UUID</i>	Уникальный идентификатор позиции.
<i>SongId</i>	<i>UUID</i>	Уникальный идентификатор песни.
<i>SegmentId</i>	<i>UUID</i>	Уникальный идентификатор сегмента.
<i>PositionIndex</i>	<i>INTAGER</i>	Номер сегмента в песне.
<i>RepeatGroup</i>	<i>INTAGER</i>	Номер группы повторения сегмента.
<i>CreatedAt</i>	<i>TIMESTAMP</i>	Дата создания позиции.

В результате спроектированная реляционная схема базы данных из 14 таблиц полностью покрывает все функциональные требования веб-приложения «Гитарный блокнот». Она обеспечивает ролевую модель доступа хранения

пользовательских музыкальных материалов, формирование альбомов с настройкой видимости, а также механизмы уведомлений. Связующие таблицы позволяют гибко компоновать сегменты песен, привязывать к ним аккорды и паттерны, объединять песни в альбомы и управлять порядком их следования, что реализует ключевую идею проекта – поддержку полного творческого цикла от отдельного аккорда до готового публичного или приватного альбома.

2.3 Архитектура веб-приложения

Архитектура приложения представляет собой распределённую систему, развернутую с использованием современных технологий, обеспечивающих производительность, масштабируемость и удобство управления. Диаграмма развертывания представлена в ДП 03.00 ГЧ. Пояснение назначения каждого элемента веб-приложения представлено в таблице 2.16.

Таблица 2.16 – Назначение элементов веб-приложения

Элемент	Назначение
<i>PostgreSQL</i>	Используется, для хранения данных в таблицах, которые необходимы для работы веб-приложения.
<i>Application Server</i>	Обрабатывать запросы пользователя, отвечает за создание и обработку объектов, запрашивать и сохраняет данные из базы данных.
<i>Chrome Browser</i>	Отображение объектов, интерфейс для работы с клиентом.
<i>Frontend Server</i>	Отвечает за доставку клиентского кода в браузер пользователя, управление маршрутизацией на стороне клиента, кэширование статических файлов и проксирование <i>API</i> -запросов к бэкенду. Обеспечивает рендеринг пользовательского интерфейса (в том числе <i>SSR</i> – серверный рендеринг) и взаимодействие с пользователем без прямого доступа к базам данных.

Таким образом были рассмотрены все ключевые элементы архитектуры веб-приложения.

2.4 Блок-схема алгоритма создания аккорда

Блок-схема алгоритма создания аккорда иллюстрирует пошаговый процесс обработки пользовательского запроса на добавление нового музыкального аккорда в рамках учебного или прикладного проекта. Блок-схема представлена в ДП 04.00.ГЧ. Сценарий начинается с ввода пользователем основных данных об аккорде.

Пользователь последовательно вводит название аккорда, расположение пальцев на грифе или клавиатуре, а также текстовое описание аккорда. После завершения ввода система приступает к проверке корректности введенных данных. Производится валидация каждого поля на соответствие ожидаемым типам и форматам: название не должно быть пустым и не может содержать

недопустимые символы, расположение пальцев должно быть описано в установленном формате, быть не слишком коротким или слишком длинным. В случае некорректного содержимого любого из полей клиенту отправляется сообщение об ошибке валидации с указанием причин.

Если все введенные данные успешно прошли валидацию, система выполняет проверку наличия такого аккорда в базе данных. Для этого осуществляется поиск по полю названия, а также, при необходимости, по уникальному сочетанию названия и расположения пальцев. Если в базе данных уже существует аккорд с таким же названием и совпадающим расположением пальцев, выполнение алгоритма прерывается, и клиенту возвращается сообщение о том, что такой аккорд уже существует. Пользователю предлагается изменить название или расположение пальцев.

В случае, когда аккорд с указанными параметрами отсутствует в базе данных, система инициирует создание нового объекта аккорда. В этот объект записываются введенные пользователем название, расположение пальцев и описание. После формирования объекта аккорд сохраняется в базе данных. Система автоматически присваивает новому аккорду уникальный идентификатор и фиксирует дату создания. В завершение клиенту возвращается сформированный объект аккорда, содержащий все данные, включая идентификатор и дату создания.

Блок-схема демонстрирует продуманный и модульный подход к созданию музыкальных аккордов, охватывающий все ключевые этапы: ввод данных, валидацию, проверку на дублирование, сохранение и возврат результата. Такой алгоритм позволяет обеспечить целостность базы данных, исключить дублирование записей и гарантировать корректность вносимой информации.

2.5 Проектирование диаграммы последовательности

Диаграмма последовательности создания паттерна игры, представленная в рамках проектирования веб-приложения или обучающего модуля, отражает порядок взаимодействия между клиентом, серверной логикой и базой данных при формировании нового игрового паттерна – набора повторяющихся действий, жестов или временных меток, используемых в процессе игры. Основным сценарием является создание пользователем уникального паттерна с последующей проверкой на дублирование и с сохранением в системе.

Взаимодействие начинается с того, что пользователь через клиентский интерфейс вводит основные характеристики паттерна. К ним относятся название паттерна, последовательность событий или действий, а также текстовое описание. После завершения ввода клиент формирует запрос на создание паттерна и отправляет его на сервер, который выступает в роли координатора всей последующей логики.

При получении запроса сервер в первую очередь инициирует процедуру валидации переданных данных. Проверяется корректность заполнения каж-

дого поля: название не должно быть пустым и не может содержать недопустимые символы, последовательность действий должна соответствовать ожидаемому формату, а описание – удовлетворять установленным ограничениям по длине и содержанию. Если какой-либо из этапов валидации не проходит, сервер немедленно возвращает клиенту сообщение об ошибке с указанием конкретной причины, и выполнение сценария завершается.

В случае успешной валидации данных сервер выполняет проверку наличия аналогичного паттерна в базе данных. Поиск осуществляется по названию паттерна, а также, при необходимости, по уникальному сочетанию названия и ключевых элементов последовательности. Если в базе данных уже существует паттерн с совпадающими параметрами, сервер прерывает дальнейшее выполнение и возвращает клиенту уведомление о том, что такой паттерн уже существует. Пользователю предлагается изменить название или уточнить последовательность действий.

Если же паттерн с указанными характеристиками отсутствует в базе данных, сервер приступает к созданию нового объекта паттерна. В этот объект последовательно записываются название, заданная пользователем последовательность действий и текстовое описание. После формирования объекта сервер сохраняет его в базе данных, которая автоматически присваивает паттерну уникальный идентификатор и фиксирует дату и время создания. По завершении сохранения сервер возвращает клиенту сформированный объект паттерна, содержащий все внесённые данные. Перед возвратом объекта сервер также выполняет повторную валидацию выходных данных на предмет их соответствия внутренней схеме хранения, чтобы исключить возможные рассинхронизации между этапом формирования и фактической записью в базу. Если по каким-либо причинам сохранённые данные не проходят эту контрольную проверку, сервер откатывает транзакцию и возвращает клиенту ошибку с требованием повторить операцию.

Таким образом, диаграмма последовательности демонстрирует чётко структурированный и надёжный процесс создания паттерна игры, построенный на принципах последовательной обработки запроса, обязательной валидации, контроля уникальности и атомарного сохранения данных. Каждый этап взаимодействия между клиентом, сервером и базой данных выполняет строго определённую задачу, что обеспечивает целостность данных, предотвращает дублирование и гарантирует предсказуемое поведение системы. Полная диаграмма последовательности представлена в ДП 05.00 ГЧ.

2.6 Выводы по разделу

В процессе проектирования веб-приложения для работы с музыкальным контентом были определены основные роли пользователей, их функции, архитектура системы, а также спроектирована логическая схема базы данных и связи между сущностями. Приложение разработано с учётом современных

технологий и стандартов, что обеспечивает его функциональность, масштабируемость и удобство использования.

В результате анализа предметной области было выделено четыре основные роли: Гость, Гитарист, Премиум и Администратор. Каждая роль имеет чётко определённые права и доступ к функционалу приложения. Гость может только просматривать публичный контент и регистрироваться. Гитарист обладает правами на создание контента с ограничениями, а также может сохранять понравившийся контент в избранное, подписываться на альбомы, оставлять комментарии и оценки, а также приобретать премиум-подписку. Премиум, в свою очередь, наследует все права Гитариста, но в безлимитном варианте, и дополнительно получает возможность создавать альбомы. Администратор обладает наивысшими привилегиями, включая просмотр любого контента, его удаление, а также блокировку и разблокировку пользователей, и управление их правами доступа и создания контента.

Спроектированная база данных включает четырнадцать таблиц, которые охватывают все основные сущности приложения, такие как пользователи, аккорды, паттерны игры, песни, альбомы, уведомления, подписки, отзывы, комментарии и связующие элементы. Связующие таблицы, такие как *SongPatterns*, *SongChords* и *SongAlbums*, реализуют связи многие ко многим, обеспечивая необходимую гибкость при формировании песен.

Архитектура веб-приложения представляет собой распределённую систему, включающую три ключевых элемента: клиентскую часть в виде браузера *Chrome*, сервер приложений, обрабатывающий бизнес-логику и запросы пользователей, а также базу данных *PostgreSQL*, отвечающую за хранение всех данных системы. Такое построение обеспечивает производительность, масштабируемость и удобство дальнейшего сопровождения проекта.

В рамках проектирования были разработаны две ключевые диаграммы, описывающие динамику работы системы. Диаграмма последовательности создания паттерна игры отражает порядок взаимодействия между клиентом, сервером и базой данных при формировании нового паттерна и включает этапы ввода данных, валидации, проверки на дублирование, сохранения и возврата результата создания объектов.

Таким образом, реализованное веб-приложение спроектировано с учётом всех ключевых аспектов, включая ролевую модель доступа, поддержку полного творческого цикла от отдельного аккорда до готового альбома, а также современные технологии и практики проектирования, что делает его эффективным инструментом для решения задач пользователей. Полные и непротиворечивые диаграммы вариантов использования, логической схемы базы данных, диаграммы развёртывания, диаграммы последовательности и блок-схемы алгоритма представлены в ДП 01.00 ГЧ, ДП 02.00 ГЧ, ДП 03.00 ГЧ, ДП 04.00 ГЧ и ДП 05.00 ГЧ соответственно.

3 Разработка веб-приложения

3.1 Обзор технологий реализации веб-приложения

ASP.NET Core [4] – это современная кросс-платформенная технология для разработки высокопроизводительных веб-приложений. Она поддерживает асинхронные запросы, что позволяет значительно повысить производительность системы, и включает встроенную поддержку протоколов *HTTP/1.1*. Преимущества *ASP.NET Core* включают модульность, гибкость настройки и интеграцию с различными библиотеками. Однако её освоение может быть сложным для разработчиков, не знакомых с экосистемой *.NET*, а использование требует наличия опытной команды для эффективной реализации сложных решений.

Node.js [5] представляет собой серверную платформу, основанную на движке *V8*, что обеспечивает высокую скорость обработки данных. Она отлично подходит для управления асинхронными операциями, таких как *WebSocket*-соединения, что делает её идеальной для работы в реальном времени. Среди преимуществ *Node.js* можно выделить огромное количество библиотек, доступных через *npm*, и активное сообщество разработчиков. Недостатки включают ограниченную производительность при работе с вычислительно сложными задачами и необходимость опытного подхода к проектированию событийной модели.

Next.js + TypeScript [6] – это фреймворк для создания серверных и клиентских веб-приложений на основе *React*, который обеспечивает производительность, *SEO*-оптимизацию и строгую типизацию. Ключевыми преимуществами являются встроенный серверный рендеринг (*SSR*), статическая генерация (*SSG*), маршрутизация на основе файловой системы и автоматическое разделение кода, что сокращает время загрузки страниц. *TypeScript* добавляет статическую типизацию, снижая количество ошибок на этапе разработки и улучшая поддержку кода в команде. *Next.js* предоставляет целостное и менее абстрактное решение «из коробки», но может требовать понимания серверных концепций (например, различие между клиентскими и серверными компонентами).

PostgreSQL [7] – это мощная реляционная база данных, известная своей надёжностью, масштабируемостью и поддержкой сложных запросов. Её сильные стороны включают высокую производительность, расширяемость и возможность работы с большими объёмами данных. Однако сложность настройки и управления может стать препятствием для новичков. *PostgreSQL* также требует тщательного планирования архитектуры базы данных для обеспечения максимальной производительности.

					ДП 03.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.		Селицкий Н.Е.			3 Разработка веб-приложения	Лит.	Лист	Листов
Пров.		Харланович А.В.				У	1	45
						БГТУ 1-40 01 01, 2025		
Н. контр.		Авдеева В.Д.						
Утв.		Смелов В.В.						

Docker [8] позволяет разворачивать приложения в изолированных контейнерах, обеспечивая удобство управления, масштабируемость и стабильность среды исполнения. Он облегчает развертывание и тестирование приложений благодаря согласованности окружения. Однако использование *Docker* требует знаний о контейнеризации, а в некоторых случаях могут возникнуть проблемы с производительностью из-за накладных расходов на виртуализацию.

Docker Compose [9] служит для упрощённого управления несколькими контейнерами, составляющими одно приложение. Инструмент предоставляет возможность быстро настраивать зависимости и запускать приложение с минимальными усилиями. Главным преимуществом является удобство автоматизации инфраструктуры. Однако сложные конфигурации в больших проектах могут усложнить управление и потребовать глубоких знаний *YAML*.

Swagger [10] – это набор инструментов и спецификаций, предназначенных для упрощения проектирования, документирования и тестирования *REST API* [11]. Основой *Swagger* является спецификация *OpenAPI*, которая задаёт формат описания *API*, включая маршруты, параметры, типы данных и возможные ответы сервера. *Swagger* предоставляет инструменты, такие как *Swagger Editor* и *Swagger UI*, для визуализации *API* и автоматической генерации документации.

Entity Framework [12] – это *ORM* [13] (объектно-реляционное отображение) для платформы *.NET*, которая позволяет разработчикам работать с базами данных, используя объектно-ориентированную парадигму. Вместо написания сложных *SQL*-запросов *EF* позволяет взаимодействовать с данными через объекты языка программирования *C#*. Это упрощает доступ к данным и ускоряет разработку приложений, исключая необходимость напрямую работать с реляционной моделью базы данных.

Axios [14] – удобная библиотека для выполнения *HTTP*-запросов в *Node.js* и клиентской части (*ReactJS* [15]). *Axios* используется для взаимодействия с *REST API* и позволяет легко обрабатывать запросы и ответы.

Express [16] – легкий фреймворк для *Node.js*, который обеспечивает создание серверов и маршрутизацию запросов. *Express* используется для настройки микросервиса, который взаимодействует с другими компонентами системы.

3.2 Описание используемых паттернов проектирования

Веб-приложение построено на основе *Onion* [17] архитектуры, где есть четыре слоя, каждый из которых отвечает за свою область действия: основы и интерфейсов, работы с данными, бизнес-логики и связи с фронтендом. Основной компонент системы – серверная часть, реализованная на *ASP.NET Core*, которая управляет бизнес-логикой, обработкой запросов и взаимодействием с базой данных через *Entity Framework*. Обращение к базе данных *PostgreSQL* также осуществляется через *EF*, что обеспечивает удобство работы с данными и прозрачность транзакций.

В *ASP.NET Core* реализованы стандартные паттерны проектирования, обеспечивающие модульность и поддержку современного программирования:

– *Dependency Injection* [18] (*DI*): позволяет управлять зависимостями компонентов и упрощает тестирование. *DI* встроен в *ASP.NET Core* и используется для предоставления сервисов, таких как доступ к базе данных или кэш.

– *Repository* [19] и *Unit of Work* [20]: эти паттерны часто используются с *Entity Framework* для абстрагирования доступа к данным и управления транзакциями.

– *Middleware* [21]: компоненты *ASP.NET Core* обрабатывают запросы в виде конвейера, который можно расширять через пользовательские промежуточные слои.

– *Singleton* [22]: обеспечивает создание единственного экземпляра объекта в рамках всего приложения. В *ASP.NET Core* реализуется через встроенный механизм *Dependency Injection* с использованием метода *AddSingleton*. Используется для логгеров, конфигураций и других сервисов с глобальным состоянием.

– *Builder* [23]: позволяет пошагово создавать сложные объекты, улучшая читаемость и управляемость кода. В *ASP.NET Core* применяется при настройке компонентов, таких как *WebHost*, *HttpClient* или *Middleware*. Реализуется через *Fluent API*, что делает конфигурацию более интуитивной.

– *Mediator* [24]: поведенческий паттерн проектирования, который заменяет прямые связи между компонентами системы на косвенные через центральный объект-посредник (медиатор). Вместо того чтобы объекты обращались друг к другу напрямую, они отправляют запросы медиатору, который самостоятельно определяет, какой компонент должен обработать этот запрос. Это снижает связанность системы, упрощает тестирование отдельных компонентов и централизует управление сложными взаимодействиями, но при этом медиатор может со временем превратиться в «божественный объект», знающий слишком много о всей логике приложения.

– *CQRS* [25] (*Command Query Responsibility Segregation*): это паттерн проектирования, который разделяет операции с данными на две отдельные модели: команды для изменения состояния системы (создание, обновление, удаление) и запросы для чтения данных. Команды не возвращают данные и могут изменять состояние системы, а запросы возвращают данные, но никогда их не изменяют. Такое разделение позволяет оптимизировать каждую сторону независимо – например, использовать разные базы данных для чтения и записи или масштабировать их по отдельности.

3.3 ORM *Entity Framework*

Entity Framework (EF) Core – это современная объектно-реляционная среда (*ORM*) для платформы *.NET*, которая позволяет разработчикам взаимодействовать с базами данных, используя объекты *.NET* и устраняя необходимость в написании большей части кода доступа к данным. *EF Core* обеспечивает отслеживание изменений, ленивую и жадную загрузку связей, миграции схемы базы данных и компилируемые запросы. Архитектура

системы построена по классической монолитной схеме: клиентская часть на *Next.js* и монолитный бэкенд на *ASP.NET Core*. В данной архитектуре бэкенд полностью берёт на себя всю бизнес-логику, обработку *API*-запросов и управление состоянием данных. Для обеспечения целостности данных и упрощения транзакций используется единая реляционная база данных (*PostgreSQL*), доступ к которой осуществляется через централизованный контекст данных – *DbContext*. Это позволяет работать с такими сущностями, как песни, аккорды, альбомы, подписки и уведомления, как с обычными объектами *C#*, инкапсулируя сложные *SQL*-запросы внутри методов *ORM*. Использование единого контекста в монолите упрощает управление транзакциями, так как все операции над различными агрегатами могут быть обёрнуты в один вызов *SaveChangesAsync* с поддержкой *ACID*.

Перед рассмотрением основного контекста данных стоит определить базовые интерфейсы и абстрактные классы, которые использует каждая модель. Все сущности наследуются от *BaseEntityWithId*, содержащего первичный ключ *Id* типа *Guid* и *CreatedAt* для хранения даты создания. Это обеспечивает единообразие идентификации объектов по всей системе и позволяет генерировать идентификаторы на клиенте до сохранения в базу данных, что упрощает работу с отношениями при массовых вставках. Базовый класс всех сущностей представлен в листинге 3.1.

```
public abstract class BaseEntityWithId
{
    public Guid Id { get; protected set; }
    public DateTime CreatedAt { get; private set; }
}
```

Листинг 3.1 – Базовые интерфейс всех сущностей

Базовый класс *BaseEntityWithId* также содержит виртуальные методы для реализации мягкого удаления или аудита, однако в текущей версии системы эти механизмы реализованы непосредственно в контексте через переопределение *SaveChangesAsync*. Такой подход инкапсулирует логику в одном месте, не загромождая бизнес-сущности техническими деталями.

Основным элементом уровня доступа к данным является контекст базы данных *AppDbContext*, который наследуется от базового класса *DbContext*. Данный контекст выступает в роли связующего звена между программными моделями и таблицами в базе данных. В классе определены наборы *DbSet* для всех ключевых сущностей, включая пользователей, песни, аккорды, схемы боя, обзоры, сегменты песен, их позиции, структуры, связки песен с паттернами и аккордами, комментарии, альбомы, связи песен с альбомами, подписки и уведомления. Конструктор контекста принимает параметры конфигурации, позволяя динамически настраивать подключение к базе данных, включая строку подключения, пул соединений, интерцепторы, логирование и поведение при обнаружении конфликтов параллелизма. При инициализации системы контекст обеспечивает

проверку структуры таблиц и эффективное управление их жизненным циклом в рамках сессий взаимодействия с клиентом. Полный класс *AppDbContext* представлен в приложении А.

В приведённом в приложении контексте обращает на себя внимание наличие двух конструкторов: параметрического, принимающего *DbContextOptions*, и конструктора по умолчанию. Параметрический конструктор используется при внедрении зависимостей в сервисный слой, тогда как конструктор по умолчанию может быть полезен для инструментов миграции или тестирования. Все *DbSet* объявлены как автоматические свойства, что позволяет *Entity Framework Core* генерировать соответствующие таблицы с именами, совпадающими с именами свойств.

Модель «*User*» является центральной в структуре управления доступом и содержит данные профиля, такие как электронная почта, никнейм, хеш пароля, роль, *URL* аватара, биография, причина блокировки, флаг наличия премиум-подписки, дата блокировки до и дата создания. Поле *Role* используется для разграничения прав доступа на уровне приложения, причём на уровне базы данных оно хранится как строка с ограничением максимальной длины, что позволяет легко добавлять новые роли без изменения схемы. В контексте для этой сущности настроены уникальные индексы по полям *Email* и *NikName*, что обеспечивает гарантию отсутствия дублирующихся записей на уровне СУБД и защиту от конкурентных вставок. Ограничения на длину строковых полей позволяют оптимизировать хранение и создавать индексы фиксированной ширины, что положительно влияет на производительность поиска контента. Класс «*User*» представлен в листинге 3.2.

```
public class User : BaseEntityWithId
{
    public string Email { get; private set; }
    public string NikName { get; private set; }
    public string PasswordHash { get; private set; }
    public string Role { get; private set; }
    public string? AvatarUrl { get; private set; }
    public string Bio { get; private set; }
    public string? BlockReason { get; private set; }
    public bool HasPremium { get; private set; }
    public DateTime? BlockedUntil { get; private set; }
    public virtual ICollection<Subscription> Subscriptions { get;
        private set; } = new List<Subscription>();
    public virtual ICollection<Notification> Notifications { get;
        private set; } = new List<Notification>();
}
```

Листинг 3.2 – Класс «*User*»

Как видно из листинга, модель «*User*» использует приватные сеттеры для большинства свойств, что означает изменение состояния объекта

только через публичные методы. Это реализует принцип инкапсуляции и гарантирует, что бизнес-правила (например, валидация электронной почты или проверка сложности пароля перед установкой хеша) будут выполнены. Навигационные свойства *Subscriptions* и *Notifications* инициализируются пустыми коллекциями, что предотвращает *NullReferenceException* при обращении к ним до явной загрузки из базы данных. Связи с песнями, аккордами, паттернами боя, обзорами и комментариями также присутствуют, но в целях краткости они опущены в листинге – полная версия модели содержит более десяти навигационных коллекций.

Система уведомлений реализована через модель «*Notification*», которая обеспечивает информирование пользователей о новых событиях, таких как добавление обзоров на отслеживаемые песни, обновления в избранных альбомах. Уведомление содержит тип события, преобразуемый в строку при сохранении, сообщение, флаг прочтения, дату создания, а также опциональные внешние ключи на пользователя-инициатора, связанную песню или альбом. Такая денормализованная структура позволяет эффективно загружать уведомления с их контекстом в один запрос через метод *Include*. Класс «*Notification*» представлен в листинге 3.3.

```
public class Notification : BaseEntityWithId
{
    public Guid UserId { get; private set; }
    public NotificationType Type { get; private set; }
    public string Message { get; private set; } = string.Empty;
    public Guid? ActorUserId { get; private set; }
    public Guid? SongId { get; private set; }
    public Guid? AlbumId { get; private set; }
    public bool IsRead { get; private set; }
    public virtual Song? Song { get; private set; }
    public virtual Album? Album { get; private set; }
}
```

Листинг 3.3 – Класс «*Notification*»

Важной особенностью модели «*Notification*» является наличие ссылки на *ActorUser* – пользователя, который совершил действие, вызвавшее уведомление. Например, если пользователь А оставил комментарий к песне пользователя Б, то в уведомлении для Б поле *ActorUserId* будет содержать идентификатор А. Это позволяет отображать в интерфейсе не только текст уведомления, но и аватар, и имя инициатора. Связь с сущностями *Song* и *Album* сделана опциональной, так как некоторые типы уведомлений (например, системное объявление) могут не относиться к конкретному контенту. Каскадное удаление настроено как *Restrict*, чтобы удаление песни или альбома не приводило к автоматическому удалению уведомлений – вместо этого уведомления остаются, но с *NULL* в соответствующем внешнем ключе, сохраняя историю для пользователя.

Музыкальный контент в рамках данной архитектуры описывается при помощи специально спроектированного набора взаимосвязанных моделей, где центральным элементом выступает сущность «*Song*». Она хранит метаданные произведения: название, исполнителя, описание, жанр, тематику, признак публичности, идентификатор владельца, ссылку на родительскую песню (для версионирования, кавер-версий или форков), *URL* кастомного аудио, его тип, полный текст песни, даты создания и обновления, а также агрегированные денормализованные поля: количество обзоров, среднюю оценку красоты и среднюю оценку сложности песни. Класс «*Song*» представлен в листинге 3.4.

```
public class Song : BaseEntityWithId
{
    public string Title { get; private set; }
    public string? Artist { get; private set; }
    public string? Description { get; private set; }
    public bool IsPublic { get; private set; }
    public Guid OwnerId { get; private set; }
    public Guid? ParentSongId { get; private set; }
    public string Genre { get; private set; }
    public string Theme { get; private set; }
    public string? CustomAudioUrl { get; private set; }
    public string? CustomAudioType { get; set; }
    public DateTime? UpdatedAt { get; set; }
}
```

Листинг 3.4 – Класс «*Song*»

Денормализация агрегатов позволяет отображать рейтинг песни в списках без выполнения тяжёлых подзапросов на каждую запись. Поддержка этих полей осуществляется через события в коде при добавлении или обновлении обзоров: каждый раз, когда пользователь оставляет новую рецензию, у соответствующей песни пересчитываются средние значения. Альтернативой могло бы быть использование вычисляемых столбцов или триггеров в базе данных, но реализация на уровне приложения даёт больше контроля над логикой округления и обработкой краевых случаев (например, отсутствие обзоров). Поле *FullText* хранит полный текст песни с аккордами в специальной разметке, что позволяет выполнять полнотекстовый поиск по содержанию.

Аккорды содержат название, аппликатуру, описание, даты создания и обновления, а также внешний ключ на создавшего пользователя. Для аккордов создан индекс по *Name* для быстрого поиска и автодополнения. Схемы боя включают название, строковое представление паттерна, описание, флаг принадлежности к *fingerstyle*, даты создания и обновления, а также внешний ключ на создателя. Классы «*Chord*» и «*StrummingPattern*» представлены в листинге 3.5.

```

public class Chord : BaseEntityWithId
{
    public string Name { get; private set; }
    public string Fingering { get; private set; }
    public string? Description { get; private set; }
    public Guid CreatedByUserId { get; private set; }
    public DateTime? UpdatedAt { get; private set; }
}
public class StrummingPattern : BaseEntityWithId
{
    public string Name { get; private set; }
    public string Pattern { get; private set; }
    public string? Description { get; private set; }
    public bool IsFingerStyle { get; private set; }
    public Guid CreatedByUserId { get; private set; }
    public DateTime? UpdatedAt { get; private set; }
}

```

Листинг 3.5 – Классы «*Chord*» и «*StrummingPattern*»

Особенностью моделей «*Chord*» и «*StrummingPattern*» является то, что они являются справочными данными, то есть создаются модераторами или опытными пользователями, а затем многократно используются в разных песнях, может даже использовать в нескольких сразу.

Для реализации связи «многие ко многим» между песнями и аккордами, а также между песнями и паттернами боя используются промежуточные сущности «*SongChord*» и «*SongPattern*». Каждая из этих сущностей имеет уникальный составной индекс на пару внешних ключей. Классы «*SongChord*» и «*SongPattern*» представлены в листинге 3.6.

```

public class SongChord : BaseEntityWithId
{
    public Guid SongId { get; private set; }
    public Guid ChordId { get; private set; }
}
public class SongPattern : BaseEntityWithId
{
    public Guid SongId { get; private set; }
    public Guid StrummingPatternId { get; private set; }
}

```

Листинг 3.6 – Классы «*SongChord*» и «*SongPattern*»

В листинге видно, что сущности «*SongChord*» и «*SongPattern*» предельно просты: они содержат только два внешних ключа и идентификатор первичного ключа. Наличие собственного первичного ключа *Id*, а не составного первичного ключа из двух внешних ключей, связано с тем, что *Entity Framework Core* легче работает с сущностями, имеющими отдельный первичный ключ, особенно при операциях удаления и обновления в связях. Уникальный индекс на паре внешних ключей обеспечивает ту же гарантию

уникальности, что и составной первичный ключ, но оставляет больше гибкости для будущих расширений. Каскадное удаление настроено как *Cascade* с обеих сторон, так как связь не может существовать без любого из родителей.

SongSegment представляет собой логический фрагмент песни (куплет, припев, бридж, соло, интро, аутро и т.д.) и содержит текст, описание, цветовую разметку для визуального выделения в редакторе, хеш содержимого, тип сегмента, длительность, а также опциональные внешние ключи на аккорд и паттерн боя. Класс «*SongSegment*» представлен в листинге 3.7.

```
public class SongSegment : BaseEntityWithId
{
    public SegmentType Type { get; private set; }
    public string? Lyric { get; private set; }
    public int? Duration { get; private set; }
    public string? Description { get; private set; }
    public string? Color { get; private set; }
    public string? BackgroundColor { get; private set; }
    public string ContentHash { get; private set; }
    public Guid? ChordId { get; private set; }
    public Guid? PatternId { get; private set; }}

```

Листинг 3.7 – Класс «*SongSegment*»

Хеш содержимого вычисляется на основе конкатенации текста, аккорда и паттерна. Это поле используется для оптимизации хранения: если два сегмента имеют одинаковый хеш, они могут ссылаться на одни и те же данные, что особенно актуально для повторяющихся секций. Индекс по *ContentHash* позволяет быстро находить дубликаты. Для внешних ключей *ChordId* и *PatternId* настроено *DeleteBehavior.SetNull*.

Позиционирование сегментов внутри песни реализовано через модель «*SongSegmentPosition*», которая хранит *SongId*, *SegmentId*, порядковый индекс и опциональную группу повторения. Класс «*SongSegmentPosition*» представлен в листинге 3.8.

```
public class SongSegmentPosition : BaseEntityWithId
{
    public Guid SongId { get; private set; }
    public Guid SegmentId { get; private set; }
    public int PositionIndex { get; private set; }
    public string? RepeatGroup { get; private set; }}

```

Листинг 3.8 – Класс «*SongSegmentPosition*»

Уникальный индекс на паре гарантирует отсутствие двух сегментов на одной позиции в рамках одной песни. Это важное ограничение целостности, поскольку позиции в песне должны быть строго упорядочены. Поле *RepeatGroup* не входит в индекс, так как несколько позиций могут иметь одинаковое значение *RepeatGroup*. Индексы по *SegmentId* и *PositionIndex*

ускоряют запросы получения всех позиций для конкретного сегмента и получения позиции по её номеру соответственно.

Комментарии пользователей могут привязываться не только к песне в целом, но и к конкретному сегменту, что позволяет обсуждать детали аранжировки, текста или исполнения конкретной части песни. Класс «*SongComment*» представлен в листинге 3.9.

```
public class SongComment : BaseEntityWithId
{
    public Guid UserId { get; private set; }
    public Guid SongId { get; private set; }
    public Guid? SegmentId { get; private set; }
    public string Text { get; private set; }
}
```

Листинг 3.9 – Класс «*SongComment*»

Каскадное удаление для комментариев настроено так: при удалении песни удаляются все её комментарии; при удалении сегмента – комментарии к нему также удаляются; при удалении пользователя – комментарии не удаляются, а внешний ключ *UserId* ограничен поведением *Restrict*. Это означает, что комментарии не исчезают при удалении аккаунта пользователя. Поле *SegmentId* опционально: если оно равно *NULL*, комментарий относится ко всей песне целиком. Это позволяет иметь единую таблицу комментариев без дублирования полей для общих и специфичных для сегментов комментариев.

Пользователи могут оставлять обзоры на песни через сущность «*SongReview*», которая содержит текст, числовые оценки красоты и сложности, а также дату создания. Оценки должны находиться в диапазоне от 1 до 5, валидация выполняется на уровне приложения. Класс «*SongReview*» представлен в листинге 3.10.

```
public class SongReview : BaseEntityWithId
{
    public Guid SongId { get; private set; }
    public Guid UserId { get; private set; }
    public string ReviewText { get; private set; }
    public DateTime? UpdatedAt { get; private set; }
    public int? BeautifulLevel { get; private set; }
    public int? DifficultyLevel { get; private set; }
}
```

Листинг 3.10 – Класс «*SongReview*»

Уникальный составной индекс предотвращает множественные обзоры от одного пользователя на одну песню, обеспечивая, что каждый

пользователь может оставить только одну рецензию на конкретное произведение. При обновлении обзора поле *UpdatedAt* обновляется, а первичный ключ остаётся неизменным, что позволяет хранить историю изменений.

Альбомы представляют собой коллекции песен, имеют название, URL обложки, описание, жанр, тематику, флаг публичности, внешний ключ на владельца, даты создания и обновления. Класс «*Album*» представлен в листинге 3.11.

```
public class Album : BaseEntityWithId
{
    public string Title { get; private set; }
    public string? CoverUrl { get; private set; }
    public string? Description { get; private set; }
    public bool IsPublic { get; private set; }
    public string? Genre { get; private set; }
    public string? Theme { get; private set; }
    public Guid OwnerId { get; private set; }
    public DateTime? UpdatedAt { get; private set; }
}
```

Листинг 3.11 – Класс «*Album*»

Флаг *IsPublic* со стартовым значением *false* означает, что по умолчанию альбом является приватным и доступен только владельцу. Владелец может в любой момент сделать альбом публичным, после чего другие пользователи смогут подписываться на обновления и просматривать его содержимое. Связь «многие ко многим» между песнями и альбомами реализована через промежуточную сущность. Каскадное удаление настроено как *Cascade* с обеих сторон – удаление альбома удаляет связи, удаление песни также удаляет связи. Это поведение логично, так как если песня удалена, она не должна больше появляться ни в одном альбоме, а если удалён альбом – его внутренняя структура больше не нужна.

Подписки позволяют пользователям отслеживать изменения в альбомах других пользователей. Модель содержит *UserId*, опциональный *TargetAlbumId* (на какой альбом подписан) и дату создания. Класс «*Subscription*» представлен в листинге 3.12.

```
public class Subscription : BaseEntityWithId
{
    public Guid UserId { get; private set; }
    public Guid? TargetAlbumId { get; private set; }
    public DateTime CreatedAt { get; private set; }
}
```

Листинг 3.12 – Класс «*Subscription*»

Для обеспечения отзывчивости API при масштабировании библиотеки песен в контексте созданы индексы, покрывающие основные шаблоны запросов. Индексы по полям *IsPublic*, *CreatedAt*, *Genre*, *Theme* на таблице *Songs* позволяют эффективно выполнять фильтрацию и сортировку в каталоге. Составные уникальные индексы предотвращают дублирование связей на уровне базы данных. Фильтрованный уникальный индекс на «*Subscription*» – пример продвинутой возможности *PostgreSQL*, которая поддерживается через *HasFilter* в *EF Core*. Индекс на *ContentHash* в *SongSegments* оптимизирует поиск дублирующихся сегментов.

Все индексы были проанализированы на предмет перекрытия: например, индекс на *UserId* и *TargetAlbumId* заменяет отдельные индексы по *UserId* и *TargetAlbumId*, так как является левосторонним и может использоваться для поиска по *UserId* отдельно, но для таблицы «*Subscription*» оставлены оба индекса, так как есть запросы поиска по *TargetAlbumId* без *UserId*. Для текстового поиска по полям *Title*, *Artist*, *FullText* в перспективе может быть добавлен полнотекстовый индекс *GIN*, но на начальном этапе используются обычные индексы *B-tree* с оператором *LIKE* с учётом регистра, что достаточно для библиотеки среднего размера.

Таким образом, единый контекст *AppDbContext* в монолитной архитектуре объединяет 14 сущностей предметной области – от пользователей и музыкального контента до систем подписок, уведомлений и детальной разметки структуры песен. Централизованное управление схемой данных, индексами и правилами каскадного удаления позволяет обеспечить согласованность, производительность запросов и упростить сопровождение кода доступа к данным во всём приложении. Выбранные стратегии конфигурации создают основу для масштабирования как по объёму данных, так и по сложности бизнес-логики. Отдельно стоит отметить, что все сущности используют закрытые или защищённые сеттеры для полей, что инкапсулирует изменения, а контекст работает через интерфейс репозитория или напрямую через *DbSet* в сервисах, что соответствует паттерну *Unit of Work*.

3.4 Реализация функций для гостя

3.4.1 Реализация функции регистрации

Функция *Register* предназначена для регистрации новых пользователей в системе. Это асинхронный метод, который обрабатывает *HTTP POST*-запросы, поступающие по маршруту «*register*». Метод принимает тело запроса в формате *JSON*, выполняет валидацию всех обязательных полей и проверку уникальности пользователя в базе данных. Его задача заключается в том, чтобы принять данные, отправленные клиентом, провести их обработку и вернуть пользователю набор токенов, необходимых для работы с системой. При успешной регистрации в базе данных создаётся новая запись, а пароль сохраняется исключительно в хешированном виде. Функция представлена в листинге 3.13.

```

[HttpPost("register")]
[AllowAnonymous]
public async Task<ActionResult<AuthResponseDto>>
Register([FromBody] RegisterUserDto dto)
{
    var command = _mapper.Map<RegisterUserCommand>(dto);
    var result = await _mediator.Send(command);
    return Ok(result);
}

```

Листинг 3.13 – Функция регистрации

Метод *Register* принимает от клиента объект *RegisterUserDto*, который содержит данные для регистрации нового пользователя: логин, адрес электронной почты, пароль и, при необходимости, дополнительную информацию, такую как имя или номер телефона. Поскольку метод асинхронный, он не блокирует поток выполнения во время обработки запроса, что особенно важно при работе с базой данных или внешними сервисами.

После получения входных данных происходит их преобразование с помощью *AutoMapper*: объект *RegisterUserDto* передается в команду *RegisterUserCommand*. Этот шаг позволяет отделить транспортную модель от внутренней команды, которую будет обрабатывать *MediatR*. Далее команда отправляется через *MediatR* в соответствующий обработчик. Использование паттерна *Mediator* позволяет разгрузить контроллер от бизнес-логики: контроллер отвечает только за приём и возврат данных, а вся логика регистрации инкапсулирована в обработчике команды. Все конечные точки используют этот паттерн, поэтому далее это будет опускаться.

Обработчик выполняет необходимые проверки: убеждается, что пользователь с таким *email* ещё не зарегистрирован, валидирует сложность пароля, при необходимости применяет дополнительные правила. Если данные корректны, создаётся новая запись в базе данных, пароль хешируется с использованием соли, после чего генерируется пара токенов – *Access Token* и *Refresh Token*. *Access Token* служит для авторизации пользователя при последующих запросах к *API*, а *Refresh Token* позволяет получить новую пару токенов без повторного ввода логина и пароля.

В случае успешного завершения всех этапов обработчик возвращает результат, который контроллер упаковывает в HTTP-ответ с кодом 200 (OK). Тело ответа содержит объект *AuthResponseDto*, включающий сгенерированные токены. Если же в процессе регистрации возникает ошибка, обработчик выбрасывает соответствующее исключение. Глобальный обработчик исключений преобразует его в ответ с кодом 400 (*Bad Request*) и понятным сообщением для клиента.

3.4.2 Реализация функции просмотра публичных песен

Функция *SearchSongs* предназначена для поиска и просмотра песен в системе. Это асинхронный метод, который обрабатывает *HTTP GET*-запросы, поступающие по маршруту «*Songs*». Его задача заключается в том, чтобы принять параметры фильтрации, переданные клиентом, выполнить поиск с учётом прав доступа и вернуть пользователю список песен, соответствующих критериям. Функция представлена в листинге 3.14.

```
[HttpGet] [AllowAnonymous]
public async Task<ActionResult<SongSearchResultDto>>
SearchSongs([FromQuery] string? searchTerm = null,
    [FromQuery] Guid? ownerId = null,
    [FromQuery] bool? isPublic = null,
    [FromQuery] Guid? chordId = null,
    [FromQuery] Guid? patternId = null,
    [FromQuery] decimal? minRating = null,
    [FromQuery] decimal? maxRating = null,
    [FromQuery] DateTime? createdFrom = null,
    [FromQuery] DateTime? createdTo = null,
    [FromQuery] string sortBy = "createdAt",
    [FromQuery] string sortOrder = "desc",
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 20){
    var currentUserId = TryGetCurrentUserId();
    if (!currentUserId.HasValue) isPublic = true;
    else if (!isPublic.HasValue && ownerId.HasValue &&
ownerId.Value != currentUserId.Value)
        isPublic = true;
    var filters = new SongSearchFilters{UserId = currentUserId,
        SearchTerm = searchTerm, OwnerId = ownerId,
        IsPublic = isPublic, ChordId = chordId,
        PatternId = patternId, MinRating = minRating,
        MaxRating = maxRating, CreatedFrom = createdFrom,
        CreatedTo = createdTo, SortBy = sortBy,
        SortOrder = sortOrder, Page = page,
        PageSize = pageSize    };
    var query = new SearchSongsQuery(filters);
    var result = await _mediator.Send(query);
    return Ok(result);}
```

Листинг 3.14 – Просмотр публичных песен

Метод *SearchSongs* принимает от клиента набор необязательных параметров фильтрации: поисковый запрос, идентификатор владельца, флаг публичности, фильтры по аккордам и паттернам, диапазон рейтинга, временной промежуток, параметры сортировки и пагинации. Все эти параметры передаются через строку запроса, что позволяет гибко настраивать условия поиска. Первый этап обработки – определение прав доступа: если пользователь не авторизован или запрашивает песни другого пользователя,

флаг *isPublic* принудительно устанавливается в *true*, что исключает доступ к чужим приватным песням.

После определения прав формируется объект *SongSearchFilters*, который вместе с идентификатором текущего пользователя передаётся в команду *SearchSongsQuery* через *MediatR*. Обработчик строит динамический запрос к базе данных: ищет по названию, фильтры по идентификаторам преобразуются в условия равенства, диапазоны рейтинга и дат накладывают ограничения. К результату применяются сортировка и пагинация.

В случае успеха обработчик возвращает объект *SongSearchResultDto* с найденными песнями и метаданной, а контроллер отправляет ответ с кодом 200. При ошибке глобальный обработчик исключений возвращает ответ с кодом 400. Такая архитектура обеспечивает гибкий и безопасный поиск песен в системе.

3.4.3 Реализация функции просмотра публичных аккордов

Функция *GetAllChords* предназначена для получения списка аккордов в системе. Это асинхронный метод, который обрабатывает HTTP GET-запросы, поступающие по маршруту «*Chords*». Его задача заключается в том, чтобы принять параметры фильтрации от клиента и вернуть отфильтрованный список аккордов с поддержкой пагинации. Функция представлена в листинге 3.15.

Метод *GetAllChords* принимает от клиента необязательные параметры: имя аккорда, аппликатуру, флаг *myChordsOnly*, а также параметры пагинации и сортировки. Первым шагом метод проверяет аутентификацию пользователя с помощью функции *IsUserAuthenticated*. Если пользователь не авторизован, но при этом запрашивает только свои аккорды, метод возвращает ошибку с кодом 400, так как невозможно определить, какие аккорды принадлежат неавторизованному пользователю.

Далее формируется объект *ChordFiltersDto*, который содержит все параметры фильтрации. Здесь происходит важная логика: если пользователь не авторизован, флаг *MyChordsOnly* принудительно устанавливается в *false*, что гарантирует показ только публичных аккордов. Если же пользователь авторизован и запрашивает свои аккорды, в фильтры добавляется его идентификатор *UserId*. Такой подход обеспечивает безопасный доступ к данным: приватные аккорды пользователя видны только ему самому.

Таким образом, метод *GetAllChords* реализует гибкую и безопасную стратегию доступа к данным: для авторизованных пользователей открывается возможность работы с личными аккордами, тогда как неавторизованным предоставляется только публичная часть базы. Разделение логики фильтрации и непосредственного запроса к данным позволяет легко модифицировать параметры поиска без изменения основного алгоритма, а поддержка пагинации гарантирует стабильную производительность даже при значительном объёме хранимых аккордов.

```

[HttpGet]
public async Task<ActionResult<PaginatedDto<ChordDto>>>
GetAllChords(
    [FromQuery] string? name = null,
    [FromQuery] string? fingering = null,
    [FromQuery] bool? myChordsOnly = false,
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 20,
    [FromQuery] string sortBy = "name",
    [FromQuery] string sortOrder = "asc"){
    try{
        var isAuth = IsUserAuthenticated();
        if (!isAuth && myChordsOnly == true){
            return BadRequest(new { error = "Only
authenticated users can view their own chords" });}
        var filters = new ChordFiltersDto{
            Name = name,
            Fingering = fingering,
            MyChordsOnly = isAuth ? myChordsOnly : false,
            UserId = isAuth && myChordsOnly == true ?
GetCurrentUserId() : null,
            Page = page,
            PageSize = pageSize,
            SortBy = sortBy,
            SortOrder = sortOrder};
        var query = new GetAllChordsQuery(filters);
        var result = await _mediator.Send(query);
        return Ok(result);}
    catch (Exception ex){
        return BadRequest(new { error = ex.Message });}}

```

Листинг 3.15 – Метод для просмотра публичных аккордов

После формирования фильтров создаётся команда *GetAllChordsQuery* и отправляется через *MediatR* в обработчик. Обработчик строит запрос к базе данных, применяя фильтры по имени и аппликатуре, ограничивая выборку публичными аккордами или аккордами конкретного пользователя в зависимости от флага *MyChordsOnly*. К результату применяются сортировка и пагинация. В случае успеха возвращается объект *PaginatedDto<ChordDto>* с аккордами и метаданными. При возникновении ошибки метод возвращает сообщение об исключении с кодом 400.

3.4.4 Реализация функции просмотра публичных паттернов

Функция *GetAllStrummingPatterns* предназначена для получения списка ритмических паттернов в системе. Это асинхронный метод, который обрабатывает *HTTP GET*-запросы, поступающие по маршруту «*Patterns*». Его задача заключается в том, чтобы принять параметры фильтрации от клиента и вернуть отфильтрованный список паттернов игры на гитаре с поддержкой пагинации. Функция представлена в листинге 3.16.

```

[HttpGet]
public async
Task<ActionResult<PaginatedDto<StrummingPatternsDto>>>
GetAllStrummingPatterns(
    [FromQuery] string? name = null,
    [FromQuery] bool? myPatternsOnly = false,
    [FromQuery] bool? isFingerStyle = null,
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 20,
    [FromQuery] string sortBy = "name",
    [FromQuery] string sortOrder = "asc"){
    try{
        var isAuthenticated = IsUserAuthenticated();
        if (!isAuthenticated && myPatternsOnly == true) {
            return BadRequest(new { error = "Only
authenticated users can view their own patterns" });}
        var filters = new StrummingPatternsFiltersDto{
            Name = name,
            MyPatternsOnly = isAuthenticated ? myPatternsOnly : false,
            IsFingerStyle = isFingerStyle,
            UserId = isAuthenticated && myPatternsOnly == true ?
GetCurrentUserId() : null,
            Page = page,
            PageSize = pageSize,
            SortBy = sortBy,
            SortOrder = sortOrder};
        var query = new GetAllPatternsQuery(filters);
        var result = await _mediator.Send(query);
        return Ok(result);}
    catch (Exception ex){
        return BadRequest(new { error = ex.Message });}}

```

Листинг 3.16 – Функция для просмотра публичных паттернов

Метод *GetAllStrummingPatterns* принимает от клиента необязательные параметры: название паттерна, флаг *myPatternsOnly*, флаг *isFingerStyle*, а также параметры пагинации и сортировки. Первым шагом метод проверяет аутентификацию пользователя. Если пользователь не авторизован, но при этом запрашивает только свои паттерны, метод возвращает ошибку с кодом 400, так как невозможно определить владельца паттернов.

Далее формируется объект *StrummingPatternsFiltersDto*, содержащий все параметры фильтрации. Здесь применяется та же логика безопасности, что и для аккордов: если пользователь не авторизован, флаг *MyPatternsOnly* принудительно устанавливается в *false*, что гарантирует показ только публичных паттернов. Если пользователь авторизован и запрашивает свои паттерны, в фильтры добавляется его идентификатор *UserId*. Параметр *isFingerStyle* позволяет дополнительно отфильтровать паттерны по типу исполнения: бой или перебор (*fingerstyle*).

После формирования фильтров создаётся команда *GetAllPatternsQuery* и отправляется через *MediatR* в обработчик. Обработчик строит запрос к базе

данных, применяя фильтры по названию и типу игры, ограничивая выборку публичными паттернами или паттернами конкретного пользователя. К результату применяются сортировка и пагинация. В случае успеха возвращается объект *PaginatedDto<StrummingPatternsDto>* с паттернами и метаданной. Благодаря такой архитектуре клиенты могут гибко искать ритмические схемы, а система гарантирует, что приватные паттерны пользователей остаются недоступными для посторонних.

3.4.5 Реализация функции фильтрация и сортировка контента

Функции фильтрации и сортировки контента являются сквозной функциональностью, пронизывающей все методы получения списков в системе. Эта возможность позволяет клиенту гибко настраивать выдаваемые результаты, получая только те данные, которые ему необходимы в заданном порядке. Пример реализации такой фильтрации представлен в листинге 3.16 на примере метода *GetAllStrummingPatterns*.

Все методы получения коллекций в системе поддерживают единый подход к фильтрации и сортировке. Параметры фильтрации передаются клиентом через строку запроса, что позволяет легко формировать запросы на стороне клиента без использования сложных тел запросов. Основными параметрами фильтрации являются: текстовый поиск по названию, фильтрация по принадлежности пользователю, фильтры-флаги (например, *isPublic*, *isFingerStyle*), фильтрация по диапазонам значений, точные фильтры по идентификаторам.

Параметры сортировки представлены двумя ключевыми полями: *sortBy* и *sortOrder* (направление сортировки: *asc* – по возрастанию, *desc* – по убыванию). При этом сервер проверяет допустимость переданного значения *sortBy*: если указано поле, не входящее в белый список разрешённых для сортировки, запрос не выполняется, а клиент получает ошибку валидации. По умолчанию для каждого метода заданы разумные значения сортировки: для аккордов и паттернов – по имени в порядке возрастания, для песен – по дате создания в порядке убывания. Такие настройки обеспечивают наиболее удобное представление данных для большинства сценариев использования без необходимости явно указывать параметры сортировки. Пагинация реализована через параметры *page* и *pageSize*, что позволяет эффективно загружать большие коллекции частями, снижая нагрузку на сервер и сеть. Кроме того, вместе с результатами сервер возвращает метаданную: общее количество записей, текущую страницу, количество страниц и фактический размер страницы, что упрощает построение интерфейсов с бесконечной прокруткой или постраничной навигацией.

Внутри обработчиков команд все переданные параметры преобразуются в динамический запрос к базе данных. Текстовые фильтры реализуются через оператор *LIKE*, фильтры по диапазонам – через сравнения *>=* и *<=*, а точные фильтры – через равенство. Сортировка выполняется динамически с исполь-

зованием рефлексии или библиотек вроде *System.Linq.Dynamic.Core*, что позволяет избежать громоздких конструкций *switch* для каждого возможного поля. Пагинация реализуется через методы *Skip()* и *Take()*, а общее количество записей подсчитывается отдельным запросом для возврата клиенту метайнформации. Такой унифицированный подход обеспечивает предсказуемость *API* и удобство его использования.

3.5 Реализация функций для гитариста

3.5.1 Реализация функции просмотр своего и публичного контента

Система предоставляет гитаристам возможность просматривать как публичный, так и собственный приватный контент. Эта функциональность реализована во всех методах получения списков и детального просмотра сущностей – песен, аккордов и ритмических паттернов. Основным принципом заключается в том, что любой неавторизованный пользователь видит только публичные записи, в то время как авторизованный пользователь дополнительно получает доступ к своим приватным материалам.

Конкретная реализация этой логики будет подробно рассмотрена в следующих пунктах при описании каждого типа контента. В общем виде алгоритм работы выглядит следующим образом: при поступлении запроса метод сначала определяет статус аутентификации пользователя через вспомогательную функцию. В зависимости от результата и переданных клиентом параметров формируются фильтры, ограничивающие выборку. Если флаг *myOnly* установлен в *true*, система возвращает только те записи, владельцем которых является текущий аутентифицированный пользователь. Если же флаг не установлен или пользователь не авторизован, возвращаются только публичные записи, принадлежащие любым пользователям системы.

Важным аспектом безопасности является тот факт, что система никогда не позволяет одному пользователю получить доступ к приватному контенту другого пользователя. Даже если злоумышленник попытается явно указать чужой идентификатор владельца в параметрах запроса, система либо проигнорирует этот параметр, либо ограничит выборку только публичными записями запрашиваемого пользователя. При попытке прямого обращения к приватной записи будет возвращена ошибка с кодом *404 Not Found* или *403 Forbidden*, что исключает утечку конфиденциальных данных. Такая архитектура сочетает удобство использования для легитимных пользователей с надёжной защитой их приватного контента.

3.5.2 Реализация функции просмотра своих и чужих публичных паттернов

Логика просмотра ритмических паттернов полностью аналогична рассмотренной ранее реализации для просмотра публичных паттернов. Метод *GetAllStrummingPatterns* позволяет пользователям получать список паттернов

с учётом прав доступа: неавторизованные пользователи видят только публичные паттерны, а авторизованные дополнительно могут запросить только свои собственные (включая приватные) с помощью флага *myPatternsOnly*. Код данной реализации представлен в листинге 3.16.

Всё работает по тому же принципу, что и для гостя. Сначала метод проверяет аутентификацию пользователя с помощью функции *IsUserAuthenticated()*. Если пользователь не авторизован, но при этом запрашивает только свои паттерны (*myPatternsOnly = true*), возвращается ошибка, так как система не может определить владельца. Далее формируются фильтры: для неавторизованных пользователей флаг *MyPatternsOnly* принудительно устанавливается в *false*, а *UserId* остаётся *null* – это гарантирует показ только публичных паттернов. Для авторизованных пользователей, запрашивающих свои паттерны, в фильтры добавляется их идентификатор, что позволяет получить полный список как публичных, так и приватных записей, принадлежащих только им. Дополнительно метод поддерживает фильтрацию по типу исполнения (*isFingerStyle* – бой или перебор), сортировку и пагинацию. Таким образом, авторизация расширяет доступные данные, но не нарушает приватность чужих записей.

3.5.3 Реализация функции просмотра своих и публичных аккордов

Логика просмотра аккордов реализована аналогичным образом и полностью соответствует описанному ранее подходу для гостя. Метод *GetAllChords* позволяет получать список аккордов с учётом статуса аутентификации пользователя: неавторизованные пользователи видят только публичные аккорды, а авторизованные могут запрашивать как публичные аккорды других пользователей, так и свои собственные (включая приватные) с помощью флага *myChordsOnly*. Код данной реализации представлен в листинге 3.15.

Принцип работы полностью совпадает с рассмотренным для гостя. Метод сначала проверяет аутентификацию пользователя. Если неавторизованный пользователь пытается запросить только свои аккорды (*myChordsOnly = true*), возвращается ошибка с кодом 400. При формировании фильтров для неавторизованных пользователей флаг *MyChordsOnly* принудительно устанавливается в *false*, а *UserId* остаётся *null*, что обеспечивает показ исключительно публичных аккордов. Для авторизованных пользователей, которые запрашивают свои аккорды, в фильтры подставляется их идентификатор, что даёт доступ ко всем их аккордам – как публичным, так и приватным. Такой же подход применяется и для других сущностей системы, например, для паттернов боя и переборов, что обеспечивает единообразие логики разграничения доступа. Метод также поддерживает фильтрацию по названию и аппликатуре, а также сортировку и пагинацию. Таким образом, авторизация расширяет объём доступных данных, но при этом приватные аккорды других пользователей остаются недоступными.

3.5.4 Реализация функции просмотра своих и публичных песен

Логика просмотра песен реализована аналогично рассмотренному подходу для гостя. Метод *SearchSongs* позволяет пользователям осуществлять поиск и просмотр песен с фильтрацией по множеству критериев. Неавторизованные пользователи видят только публичные песни, а авторизованные – дополнительно могут запрашивать свои собственные песни (включая приватные) или публичные песни других пользователей. Код данной реализации представлен в листинге 3.14.

Принцип работы во многом схож с рассмотренными ранее методами для аккордов и паттернов. Сначала метод пытается получить идентификатор текущего авторизованного пользователя через функцию *TryGetCurrentUserId()*. Если пользователь не авторизован, флаг *isPublic* принудительно устанавливается в *true*, что ограничивает выборку только публичными песнями. Если же авторизованный пользователь запрашивает песни другого пользователя, метод также принудительно ограничивает выборку только публичными записями. Такая логика обеспечивает безопасность: никто не может получить доступ к чужим приватным песням.

Для авторизованного пользователя, который хочет увидеть свои песни, достаточно не устанавливать флаг *isPublic* в *true* или явно передать *ownerId*, равный своему идентификатору. В этом случае система вернёт все его песни – как публичные, так и приватные. Метод поддерживает богатый набор фильтров: поиск по названию, фильтрацию по владельцу, аккордам, паттернам, диапазону рейтинга и дате создания, а также сортировку и пагинацию. Таким образом, авторизация расширяет доступные данные, предоставляя пользователю полный контроль над своими песнями, но при этом чужие приватные записи остаются надёжно защищёнными.

3.5.5 Реализация функции просмотра своих и публичных альбомов

Функция *SearchAlbums* предназначена для поиска и просмотра альбомов в системе. В отличие от ранее рассмотренных методов (песни, аккорды, паттерны), доступ к альбомам имеют только авторизованные пользователи, за исключением гостевой роли. Это асинхронный метод, который обрабатывает *HTTP GET*-запросы, поступающие по маршруту «Albums». Его задача – принять параметры фильтрации и вернуть пользователю список альбомов с учётом прав доступа. Код реализации представлен в листинге 3.17.

Метод *SearchAlbums* принимает от клиента необязательные параметры фильтрации: поисковый запрос, идентификатор владельца, флаг публичности, жанр, тематику альбома, а также параметры сортировки и пагинации. Первым шагом метод получает идентификатор текущего авторизованного пользователя через *GetCurrentUserId()* и загружает его полную информацию из сервиса пользователей. Это принципиальное отличие от предыдущих методов – здесь предполагается, что пользователь уже авторизован.

Далее выполняется проверка роли пользователя. Если текущий пользователь имеет роль «Гость», метод немедленно возвращает пустой результат с кодом 200. Гостевой доступ к альбомам не предоставляется, так как альбомы являются более сложной структурой. Такое решение принято для упрощения системы и снижения нагрузки на базу данных.

```
[HttpGet]
public async Task<ActionResult<AlbumSearchResultDto>>
SearchAlbums(
    [FromQuery] string? searchTerm = null,
    [FromQuery] Guid? ownerId = null,
    [FromQuery] bool? isPublic = null,
    [FromQuery] string? genre = null,
    [FromQuery] string? theme = null,
    [FromQuery] string sortBy = "createdAt",
    [FromQuery] string sortOrder = "desc",
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 20){
    var currentUserId = GetCurrentUserId();
    var currentUser = await
_userService.GetByIdAsync(currentUserId);
    if (currentUser.Role == Constants.Roles.Guest){
        return Ok(new AlbumSearchResultDto { Albums = new
List<AlbumDto>() });}
    var filters = new AlbumSearchFilters{UserId =
currentUserId,
        SearchTerm = searchTerm,
        OwnerId = ownerId,
        IsPublic = isPublic,
        Genre = genre,
        Theme = theme,
        SortBy = sortBy,
        SortOrder = sortOrder,
        Page = page,
        PageSize = pageSize};
    var query = new SearchAlbumsQuery(filters);
    var result = await _mediator.Send(query);
    return Ok(result); }
```

Листинг 3.17 – Функция поиска и просмотра альбомов

Если пользователь не является гостем, формируется объект *AlbumSearchFilters*, содержащий все параметры фильтрации вместе с идентификатором текущего пользователя. В отличие от песен, здесь нет автоматической подстановки флага публичности при запросе чужих альбомов – логика разграничения доступа полностью реализована на уровне обработчика команды *SearchAlbumsQuery*. Обработчик самостоятельно определяет, какие альбомы может видеть пользователь: свои собственные (и публичные, и приватные) и публичные альбомы других пользователей. После выполнения за-

проса метод возвращает объект *AlbumSearchResultDto* с найденными альбомами, общим количеством и метаданной пагинации. Такой подход обеспечивает гибкий поиск альбомов для авторизованных пользователей.

3.5.6 Реализация функции добавления или удаления песни в или из избранного

Функции *AddSongToFavorite* и *RemoveSongFromFavorite* предназначены для управления списком избранных песен пользователя. Эти асинхронные методы обрабатывают *HTTP POST* и *DELETE* запросы, поступающие по маршруту «*favorite/{songId}*». Их задача – добавить или удалить указанную песню в избранное текущего авторизованного пользователя. Данные функции доступны только для аутентифицированных пользователей, так как требуют идентификации пользователя. Код реализации представлен в листинге 3.18.

```
[HttpPost("favorite/{songId}")]
public async Task<ActionResult> AddSongToFavorite(Guid songId)
{
    try{
        var userId = GetCurrentUser();
        var command = new AddSongToFavoriteCommand(userId, songId);
        await _mediator.Send(command);
        return Ok(new { message = "Song added to favorites successfully" });
    }
    catch (KeyNotFoundException ex){
        return NotFound(new { error = ex.Message });
    }
    catch (InvalidOperationException ex){
        return Conflict(new { error = ex.Message });
    }
}

[HttpDelete("favorite/{songId}")]
public async Task<ActionResult> RemoveSongFromFavorite(Guid songId) {
    try{
        var userId = GetCurrentUser();
        var command = new
        RemoveSongFromFavoriteCommand(userId, songId);
        await _mediator.Send(command);
        return NoContent();
    }
    catch (KeyNotFoundException ex){
        return NotFound(new { error = ex.Message });
    }
}
```

Листинг 3.18 – Функции для добавления и удаления песен в/из избранного

Метод *AddSongToFavorite* принимает идентификатор песни, переданный через маршрут. Первым шагом он получает идентификатор текущего авторизованного пользователя с помощью *GetCurrentUser()*, после чего формирует команду *AddSongToFavoriteCommand*, содержащую оба идентификатора. Команда отправляется через *MediatR* в соответствующий обработчик. При успешном выполнении метод возвращает ответ с кодом 200 и сообщением об успешном добавлении песни в избранное.

В случае возникновения ошибки метод обрабатывает два типа исключений. Если песня с указанным идентификатором не существует в системе, обработчик выбрасывает *KeyNotFoundException* – метод преобразует его в ответ с кодом 404 и соответствующим сообщением. Если пользователь уже добавлял эту песню в избранное ранее, выбрасывается *InvalidOperationException*, который преобразуется в ответ с кодом 409, информирующий клиента о том, что песня уже находится в избранном. Такая обработка позволяет клиенту правильно реагировать на различные сценарии ошибок.

Метод *RemoveSongFromFavorite* работает аналогично, но с двумя отличиями. Во-первых, он использует команду *RemoveSongFromFavoriteCommand* для удаления песни из избранного. Во-вторых, при успешном выполнении он возвращает ответ с кодом 204, что соответствует стандартному подходу *REST API* для операций удаления – тело ответа не требуется, так как клиенту не передается никакой дополнительной информации. В случае отсутствия песни в избранном пользователя или отсутствия самой песни в системе выбрасывается *KeyNotFoundException*, и метод возвращает код 404. Благодаря такой архитектуре пользователи могут эффективно управлять списком избранных песен.

3.5.7 Реализация функции написания отзыва и выставление оценки

Функция *CreateReview* предназначена для написания отзыва и выставления оценки песне. Это асинхронный метод, который обрабатывает *HTTP POST*-запросы, поступающие по маршруту «*songs/{songId}*». Его задача – принять текст отзыва и оценки по критериям красоты и сложности исполнения, после чего сохранить их в системе. Данная функция доступна только для аутентифицированных пользователей, так как отзыв должен быть привязан к конкретному пользователю. Код реализации представлен в листинге 3.19.

```
[HttpPost("songs/{songId}")]
public async Task<ActionResult<SongReviewDto>> CreateReview(
    Guid songId, [FromBody] CreateSongReviewDto dto){
    try{
        var userId = GetCurrentUser();
        var command = new CreateSongReviewCommand(
            userId, songId,
            dto.ReviewText,
            dto.BeautifulLevel,
            dto.DifficultyLevel);
        var result = await _mediator.Send(command);
        return CreatedAtAction(nameof(GetReview), new { id = result.Id
    }, result);
    }
    catch (ArgumentException ex) {
        return BadRequest(new { error = ex.Message });
    }
    catch (InvalidOperationException ex) {
        return BadRequest(new { error = ex.Message });
    }
}
```

Листинг 3.19 – Функции для создания отзыва с оценками

Метод *CreateReview* принимает идентификатор песни через маршрут, а данные отзыва – через тело запроса в виде объекта *CreateSongReviewDto*. Этот *DTO* содержит текст отзыва, оценку красоты исполнения и оценку сложности. Первым шагом метод получает идентификатор текущего авторизованного пользователя с помощью *GetCurrentUserId()*, после чего формирует команду *CreateSongReviewCommand*, объединяющую идентификаторы пользователя и песни, а также все переданные данные.

Команда отправляется через *MediatR* в соответствующий обработчик, который выполняет основную бизнес-логику. Обработчик проверяет, существует ли песня с указанным идентификатором, и не оставлял ли уже этот пользователь отзыв на данную песню ранее. Если проверки пройдены успешно, создаётся новая запись отзыва в базе данных, вычисляется средняя оценка песни на основе всех полученных отзывов, после чего обновляется рейтинг песни.

В случае успешного выполнения метод возвращает ответ с кодом 201 – стандартный код *HTTP* для успешного создания ресурса. В заголовке *Location* клиенту передаётся *URL* для получения созданного отзыва, а в теле ответа возвращается объект *SongReviewDto* с информацией о созданном отзыве.

При возникновении ошибок метод обрабатывает два типа исключений. *ArgumentException* выбрасывается, если переданные оценки выходят за допустимый диапазон (например, меньше 1 или больше 5) или текст отзыва не соответствует требованиям по длине – метод возвращает ответ с кодом 400 (*Bad Request*). *InvalidOperationException* возникает, если пользователь уже оставлял отзыв на эту песню (система не позволяет повторное голосование) или песня не найдена – также преобразуется в ответ с кодом 400. Такая архитектура предотвращает накрутку оценок и обеспечивает честность рейтинга песен.

3.5.8 Реализация функции создание, обновление и удаление контента с лимитами

Функции создания, обновления и удаления контента (песен, аккордов, паттернов и альбомов) являются ключевыми возможностями системы, но они ограничены для обычных пользователей. В системе действует система лимитов: обычные пользователи имеют ограничение на количество создаваемого контента, например, не более 5 песен, 10 аккордов и т.д. Для снятия этих ограничений пользователь может приобрести премиум-подписку, либо получить роль администратора, который не имеет лимитов.

Принцип работы лимитов реализован на уровне обработчиков команд в *MediatR*. Перед выполнением операции создания нового контента обработчик сначала загружает информацию о текущем пользователе из базы данных, проверяет его роль и статус подписки. Если пользователь имеет роль администратора или активную премиум-подписку, проверка лимитов пропускается, и пользователь может создавать неограниченное количество контента.

Если же пользователь является обычным, обработчик выполняет подсчёт уже существующих сущностей данного типа, созданных этим пользователем. Например, перед созданием новой песни система считает, сколько песен уже создал пользователь, и сравнивает это значение с установленным лимитом. Если лимит превышен, операция прерывается, и пользователю возвращается ошибка с кодом 403 или 400, содержащая сообщение о необходимости приобретения премиум-подписки для создания дополнительного контента. Аналогичные проверки выполняются для аккордов, паттернов и альбомов, при этом для каждого типа контента может быть установлен свой лимит.

Для операций обновления и удаления контента система также выполняет проверку прав доступа. Обработчик проверяет, является ли текущий пользователь владельцем контента. Если пользователь не является владельцем и не имеет роли администратора, операция отклоняется с кодом 403. Это гарантирует, что пользователь может изменять или удалять только свой собственный контент. Администратор же имеет право управлять любым контентом в системе, независимо от владельца. Такая система лимитов стимулирует пользователей приобретать премиум-подписку, одновременно защищая систему от чрезмерной нагрузки со стороны бесплатных пользователей и обеспечивая контроль над объёмом хранимых данных.

3.5.9 Реализация функций создание или удаление подписки

Функции *Subscribe* и *Unsubscribe* предназначены для управления подписками пользователей на альбомы. Это асинхронные методы, которые обрабатывают *HTTP POST* и *DELETE* запросы, поступающие по маршруту «*{albumId}*». Их задача – подписать пользователя на обновления альбома или отписать от него. Данные функции доступны только для авторизованных пользователей и имеют ограничения для бесплатных аккаунтов. Код реализации представлен в листинге 3.20.

Метод *Subscribe* принимает идентификатор альбома через маршрут. Первым шагом он получает идентификатор текущего пользователя с помощью *GetCurrentUserId()* и загружает полную информацию о пользователе через сервис *_userService*. Далее выполняется проверка статуса пользователя: если пользователь является бесплатным (*IsFreeUser*), система подсчитывает количество альбомов, на которые он уже подписан, отправляя запрос *GetUserSubscriptionsCountQuery* через *MediatR*.

Если текущее количество подписок достигло максимального лимита для бесплатных пользователей (*Constants.Limits.FreeUserMaxSubscriptions*), метод возвращает ошибку с кодом 400. В сообщении клиенту указывается причина отказа и предлагается приобрести премиум-подписку для снятия ограничений. Для премиум-пользователей и администраторов проверка лимитов пропускается – они могут подписываться на неограниченное количество альбомов.

После успешного прохождения проверки лимитов формируется команда *SubscribeCommand*, содержащая идентификаторы пользователя и альбома. Команда отправляется через *MediatR* в обработчик, который проверяет существование альбома и отсутствие активной подписки (пользователь не может подписаться на один альбом дважды). При успешном выполнении метод возвращает ответ с кодом 200 и объектом *SubscriptionResponseDto*, содержащим информацию о созданной подписке.

```
[HttpPost("{albumId}")]
public async Task<ActionResult<SubscriptionResponseDto>>
Subscribe(Guid albumId) {
    try{
        var currentUserId = GetCurrentUserId();
        var user = await _userService.GetByIdAsync(currentUserId);
        if (user.IsFreeUser) {
            var subscriptionsCount = await _mediator.Send(new
            GetUserSubscriptionsCountQuery(currentUserId));
            if (subscriptionsCount >=
            Constants.Limits.FreeUserMaxSubscriptions) {
                return BadRequest(new {
                    error = $"Free users can only subscribe
                    to {Constants.Limits.FreeUserMaxSubscriptions} albums. " +
                    "Upgrade to Premium for unlimited subscriptions."});
            }
            var command = new SubscribeCommand(currentUserId,
            albumId);
            var result = await _mediator.Send(command);
            return Ok(result);
        }
        catch (KeyNotFoundException ex) {
            return NotFound(new { error = ex.Message });
        }
        catch (InvalidOperationException ex) {
            return BadRequest(new { error = ex.Message });
        }
        catch (Exception ex) {
            return BadRequest(new { error = ex.Message });
        }
    }
    [HttpDelete("{albumId}")]
    public async Task<ActionResult> Unsubscribe(Guid albumId) {
        try{
            var currentUserId = GetCurrentUserId();
            var command = new UnsubscribeCommand(currentUserId, albumId);
            await _mediator.Send(command);
            return NoContent();
        }
        catch (KeyNotFoundException ex) {
            return NotFound(new { error = ex.Message });
        }
        catch (Exception ex) {
            return BadRequest(new { error = ex.Message });
        }
    }
}
```

Листинг 3.20 – Функции создания и удаления подписки на альбом

Метод *Unsubscribe* работает проще: он получает идентификатор текущего пользователя, формирует команду *UnsubscribeCommand* и отправляет её через *MediatR*. Обработчик удаляет существующую подписку пользователя на указанный альбом. В случае успеха метод возвращает ответ с кодом 204

(*NoContent*), что соответствует стандарту *REST API* для операций удаления. При возникновении ошибок оба метода обрабатывают *KeyNotFoundException* (если альбом или подписка не найдены) и общее исключение, возвращая соответствующие коды 404 или 400. Такая архитектура позволяет гибко управлять подписками с учётом статуса пользователя.

3.5.10 Реализация функций создания, обновления и удаления аккордов

Функции *CreateChord*, *UpdateChord* и *DeleteChord* предназначены для управления аккордами в системе. Это асинхронные методы, которые обрабатывают *HTTP POST*, *PUT* и *DELETE* запросы, поступающие по соответствующим маршрутам. Их задача – создавать новые аккорды, обновлять существующие или удалять их. Данные функции доступны только для авторизованных пользователей, что обеспечивается атрибутом *[Authorize]*. Код реализации представлен в приложении Б.

Метод *CreateChord* принимает от клиента объект *CreateChordDto*, содержащий название аккорда, аппликатуру (схему расположения пальцев) и описание. Первым шагом метод получает идентификатор текущего пользователя через *GetCurrentUserId()*, после чего формирует команду *CreateChordCommand* с переданными данными. Команда отправляется через *MediatR* в обработчик, который выполняет проверку: существует ли уже аккорд с таким же названием у этого пользователя. Если аккорд-дубликат обнаружен, выбрасывается *InvalidOperationException*, и метод возвращает ответ с кодом 409 (*Conflict*). При успешном создании возвращается ответ с кодом 201 (*Created*) и объектом созданного аккорда, а в заголовке *Location* передаётся *URL* для его получения.

Метод *UpdateChord* принимает идентификатор аккорда через маршрут и объект *UpdateChordDto* с обновлёнными данными. После получения идентификатора пользователя формируется команда *UpdateChordCommand*. Обработчик выполняет несколько проверок: существует ли аккорд с указанным идентификатором, является ли текущий пользователь его владельцем. Если аккорд не найден, выбрасывается *KeyNotFoundException*, и метод возвращает код 404. Если пользователь не является владельцем, выбрасывается *UnauthorizedAccessException*, и метод возвращает *Forbid()*. При успешном обновлении возвращается ответ с кодом 200 и обновлённым объектом аккорда.

Метод *DeleteChord* принимает идентификатор аккорда, а также получает идентификатор и роль текущего пользователя. Команда *DeleteChordCommand* передаёт эти данные в обработчик. Логика удаления учитывает роль пользователя: обычный пользователь может удалить только свой собственный аккорд, а администратор – любой аккорд в системе. При попытке удалить чужой аккорд без прав администратора выбрасывается *UnauthorizedAccessException*, и метод возвращает *Forbid()*. В случае успешного удаления возвращается ответ с кодом 204 (*NoContent*). Такая архитектура обеспечивает полный контроль

пользователей над своими аккордами при сохранении административных возможностей для модерации контента.

3.5.11 Реализация функций создания, обновления и удаления паттернов

Функции создания, обновления и удаления ритмических паттернов реализованы по тому же принципу, что и для аккордов. Это асинхронные методы, обрабатывающие *HTTP POST*, *PUT* и *DELETE* запросы, доступные только для авторизованных пользователей с атрибутом *[Authorize]*. Их задача – создавать новые паттерны игры на гитаре (бой или перебор), обновлять существующие или удалять их. Логика работы полностью аналогична управлению аккордами, поэтому здесь будут рассмотрены только ключевые особенности. Код реализации представлен в приложении В.

Метод создания паттерна принимает от клиента объект *DTO*, содержащий название, тип исполнения (бой или перебор – *isFingerStyle*) и описание. После получения идентификатора текущего пользователя формируется команда, которая проверяет отсутствие дубликатов с таким же названием у этого пользователя. При успешном создании возвращается ответ с кодом 201 и объектом созданного паттерна.

Методы обновления и удаления паттерна выполняют проверку прав доступа: пользователь может изменять или удалять только свои собственные паттерны. Если паттерн не найден, возвращается код 404. Если пользователь не является владельцем, возвращается *Forbid()* (код 403). Администратор имеет право управлять любыми паттернами в системе. При успешном обновлении возвращается код 200 с обновлённым объектом, при удалении – код 204 (*NoContent*). Такая унифицированная архитектура обеспечивает единообразие работы со всеми типами контента в системе.

3.5.12 Реализация функций создания, обновления и удаления песен

Функции создания, обновления и удаления песен реализованы по тому же принципу, что и для аккордов и паттернов, но с учётом большей сложности этого типа контента. Методы *CreateSong*, *UpdateSong* и *DeleteSong* обрабатывают *HTTP POST*, *PUT* и *DELETE* запросы, доступны только для авторизованных пользователей с атрибутом *[Authorize]*. Их задача – создавать новые музыкальные произведения, обновлять существующие или удалять их. Код реализации представлен в приложении Г.

Метод создания песни принимает от клиента объект *CreateSongDto*, содержащий значительный объём информации: название, жанр, тематику, аудиоданные в формате *Base64*, тип аудио, исполнителя, описание, флаг публичности, идентификатор родительской песни (если это кавер-версия), а также кастомные *URL* и тип аудио. После получения идентификатора текущего поль-

зователя формируется команда *CreateSongCommand*, которая отправляется через *MediatR*. Обработчик выполняет проверку лимитов на количество созданных песен для бесплатных пользователей, а также валидацию входных данных. При успешном создании возвращается ответ с кодом 201 и объектом созданной песни.

Метод обновления песни принимает идентификатор песни через маршрут и объект *UpdateSongDto* с обновлёнными данными. Команда *UpdateSongCommand* также получает идентификатор и роль текущего пользователя. Обработчик проверяет существование песни и право пользователя на её редактирование: владелец может изменять свою песню, администратор – любую. Если песня не найдена, возвращается код 404, при отсутствии прав – *Forbid()* (код 403). При успешном обновлении возвращается ответ с кодом 200 и обновлённым объектом песни.

Метод удаления песни работает аналогично: проверяет права доступа с учётом роли пользователя. В случае успеха возвращается ответ с кодом 204 (*NoContent*). Особенностью удаления песен является каскадное удаление связанных данных – отзывов, оценок, избранного и подписок. Такая архитектура обеспечивает полный контроль пользователей над своим музыкальным контентом при сохранении возможности административной модерации.

3.5.13 Реализация функций создания, обновления и удаления аудио для песен

Функция *UploadAudio* предназначена для загрузки аудиофайлов к существующим песням. Это асинхронный метод, который обрабатывает *HTTP POST*-запросы, поступающие по маршруту «*{id}/audio*». Его задача – принять аудиофайл от клиента, выполнить его валидацию и сохранить на *WebDAV*-сервере, после чего обновить информацию о песне. Данная функция доступна только для авторизованных пользователей и имеет ограничения по размеру загружаемого файла. Код реализации представлен в приложении Д.

Метод *UploadAudio* принимает идентификатор песни через маршрут и аудиофайл через параметр *IFormFile*. Первым шагом метод получает идентификатор текущего пользователя и загружает информацию о песне из базы данных. Если песня не найдена, возвращается ответ с кодом 404. Затем выполняется проверка прав доступа: пользователь может загрузить аудио только для своей собственной песни. Если пользователь не является владельцем, возвращается *Forbid()* (код 403).

Далее следует комплексная валидация аудиофайла. Проверяется, что файл не пустой, его размер не превышает 100 МБ (лимит задаётся атрибутом [*RequestSizeLimit(100_000_000)*]), а тип содержимого соответствует одному из поддерживаемых форматов: *MP3*, *WAV*, *OGG*, *M4A*, *AAC*, *FLAC*, *Opus* или *WebM*. Если какая-либо проверка не пройдена, возвращается ответ с кодом 400 и понятным сообщением об ошибке.

После успешной валидации формируется имя файла на основе идентификатора песни с соответствующим расширением. Расширение определяется либо из исходного имени файла, либо сопоставляется из *MIME*-типа. Затем метод открывает поток для чтения файла и вызывает сервис *webDavService.UploadAudioAsync*, который сохраняет файл на *WebDAV*-сервере. В ответ сервис возвращает имя сохранённого файла.

Затем обновляется запись песни в базе данных: вызывается метод *Update*, который устанавливает *customAudioUrl* (путь к загруженному файлу) и *customAudioType* (*MIME*-тип аудио). После сохранения изменений в базе данных метод возвращает ответ с кодом 200, содержащий информацию о загруженном файле: имя, тип и размер. В случае возникновения ошибок метод логирует их и возвращает ответ с кодом 500. Для операций удаления аудио существует аналогичный метод *DeleteAudio*, который удаляет файл с *WebDAV*-сервера и обнуляет соответствующие поля в записи песни. Такая архитектура обеспечивает надёжное хранение аудиофайлов и их привязку к музыкальным произведениям.

3.5.14 Реализация функций покупки премиума

Функция *UpgradeToPremium* предназначена для оформления премиум-подписки пользователем. Это асинхронный метод, который обрабатывает HTTP POST-запросы, поступающие по маршруту «upgrade-to-premium». Его задача – принять данные о способе оплаты от клиента, обработать транзакцию через платёжную систему и обновить статус пользователя на премиум. Данная функция доступна только для авторизованных пользователей. Код реализации представлен в листинге 3.21.

```
[HttpPost("upgrade-to-premium")]
public async Task<ActionResult<PremiumUpgradeResultDto>>
UpgradeToPremium([FromBody] PremiumUpgradeDto dto) {
    try {
        var userId = GetCurrentUserId();
        var command = new UpgradeToPremiumCommand(userId,
            dto.PaymentMethod, dto.PaymentToken);
        var result = await _mediator.Send(command);
        return Ok(result);
    }
    catch (KeyNotFoundException ex) {
        return NotFound(new { error = ex.Message });
    }
    catch (InvalidOperationException ex) {
        return BadRequest(new { error = ex.Message });
    }
    catch (Exception ex) {
        return BadRequest(new { error = ex.Message });
    }
}
```

Листинг 3.21 – Функция оформления премиум-подписки

Метод *UpgradeToPremium* принимает от клиента объект *PremiumUpgradeDto*, содержащий информацию о способе оплаты

(*PaymentMethod*) и платёжном токене (*PaymentToken*), который получен от внешней платёжной системы (например, Stripe, PayPal или другой). Первым шагом метод получает идентификатор текущего пользователя с помощью *GetCurrentUserId()*, после чего формирует команду *UpgradeToPremiumCommand* с переданными данными.

Команда отправляется через *MediatR* в соответствующий обработчик, который выполняет основную бизнес-логику. Обработчик сначала проверяет, не является ли пользователь уже премиум-подписчиком. Если пользователь уже имеет активную премиум-подписку, выбрасывается *InvalidOperationException*, и клиент получает соответствующий ответ с кодом 400. Далее обработчик взаимодействует с внешней платёжной системой, используя переданный токен для подтверждения оплаты. Если платёж не прошёл валидацию или был отклонён, также выбрасывается исключение.

После успешного подтверждения оплаты обработчик обновляет статус пользователя в базе данных: устанавливается роль *Premium*, фиксируется дата начала и окончания подписки (обычно на 30 дней). Также может быть записана информация о транзакции для истории платежей. В случае успеха метод возвращает ответ с кодом 200 и объектом *PremiumUpgradeResultDto*, содержащим информацию о новой роли пользователя, дате окончания подписки и других релевантных данных.

При возникновении ошибок метод обрабатывает *KeyNotFoundException* (если пользователь не найден), *InvalidOperationException* (при попытке повторного апгрейда или проблемах с платежом), а также общее исключение, возвращая соответствующие коды 404 или 400 с понятными сообщениями для клиента. Такая архитектура позволяет безопасно и надёжно обрабатывать платёжные транзакции, интегрируясь с внешними платёжными системами.

3.5.15 Реализация функций аутентификация без привилегий

Функция *Login* предназначена для аутентификации уже зарегистрированных пользователей в системе. Это асинхронный метод, который обрабатывает *HTTP POST*-запросы, поступающие по маршруту «*login*». Его задача – принять учётные данные пользователя (*email* и пароль), проверить их корректность и вернуть набор токенов для дальнейшей работы с системой. Данная функция доступна для всех пользователей, включая неавторизованных, что обеспечивается атрибутом *[AllowAnonymous]*. Код реализации представлен в листинге 3.22.

```
[HttpPost("login")] [AllowAnonymous]
public async Task<ActionResult<AuthResponseDto>>
Login([FromBody] LoginUserDto dto) {
    var command = _mapper.Map<LoginUserCommand>(dto);
    var result = await _mediator.Send(command);
    return Ok(result); }
```

Листинг 3.22 – Функция входа в систему

Метод *Login* принимает от клиента объект *LoginUserDto*, который содержит адрес электронной почты и пароль пользователя. Поскольку метод асинхронный, он не блокирует поток выполнения сервера во время обработки запроса, что особенно важно при работе с базой данных и проверке пароля.

После получения входных данных происходит их преобразование с помощью *AutoMapper*: объект *LoginUserDto* сопоставляется в команду *LoginUserCommand*. Использование паттерна *Mediator* позволяет разгрузить контроллер от бизнес-логики: контроллер отвечает только за приём данных и возврат результата, а вся логика аутентификации инкапсулирована в обработчике команды.

Обработчик выполняет необходимые проверки: ищет пользователя в базе данных по указанному *email*, проверяет корректность пароля (с учётом хеширования и соли), а также убеждается, что учётная запись активна (например, не заблокирована и не удалена). Если пользователь не найден или пароль неверен, обработчик выбрасывает соответствующее исключение, которое глобальный обработчик преобразует в ответ с кодом 401 (*Unauthorized*) или 400.

В случае успешной аутентификации генерируется пара токенов: *Access Token* для авторизации при последующих запросах и *Refresh Token* для обновления доступа. Обработчик возвращает результат, который контроллер упаковывает в *HTTP*-ответ с кодом 200 (*OK*). Тело ответа содержит объект *AuthResponseDto*, включающий сгенерированные токены. Важно отметить, что в отличие от регистрации, метод входа не создаёт новых пользователей и требует дополнительных проверок лимитов или прав доступа, поэтому он относится к функциям «без привилегий».

3.6 Реализация функций для премиума

3.6.1 Реализация функций создания, обновления и удаления контента без лимита

Как было описано ранее, система ограничивает обычных (бесплатных) пользователей в количестве создаваемого контента – например, не более 5 песен, 10 аккордов или 3 альбомов. Однако для пользователей с премиум-подпиской или ролью администратора эти ограничения отсутствуют. По сути, создание, обновление и удаление контента для премиум-пользователей и администраторов реализовано через те же самые методы, но с одним важным отличием: проверка лимитов в обработчиках команд просто пропускается.

Таким образом, когда премиум-пользователь или администратор вызывает методы *CreateSong*, *CreateChord*, *CreatePattern* или *CreateAlbum*, в обработчике команды выполняется проверка статуса пользователя. Если система определяет, что пользователь имеет активную премиум-подписку или роль администратора, то подсчёт уже созданных сущностей и сравнение с лимитами не производится – операция создания разрешается всегда, независимо от того,

сколько контента пользователь уже создал ранее. Для обычных же пользователей эта проверка выполняется, и при превышении лимита операция отклоняется с соответствующим сообщением.

Аналогичным образом для операций обновления и удаления контента премиум-пользователи и администраторы также имеют расширенные права. Администратор может обновлять или удалять любой контент в системе независимо от владельца, а премиум-пользователь – только свой собственный (как и обычный пользователь), но без ограничений на количество создаваемого контента. Такая система позволяет предоставлять дополнительные возможности платным подписчикам и администраторам, стимулируя бесплатных пользователей к приобретению премиум-подписки.

3.6.2 Реализация функций создания, обновления и удаления альбома

Функции *CreateAlbum*, *UpdateAlbum* и *DeleteAlbum* предназначены для управления альбомами в системе. Это асинхронные методы, которые обрабатывают *HTTP POST*, *PUT* и *DELETE* запросы. Их задача – создавать новые альбомы, обновлять существующие или удалять их. Ключевое отличие от других типов контента заключается в том, что создание альбомов доступно только пользователям с премиум-подпиской. Код представлен в приложении Е.

Метод *CreateAlbum* принимает от клиента объект *CreateAlbumDto*, содержащий название альбома, флаг публичности, жанр, тематику, *URL* обложки и описание. Первым шагом метод получает идентификатор текущего пользователя и загружает его информацию через сервис *_userService*. Затем вызывается вспомогательный метод *user.CanCreateAlbum()*, который проверяет, имеет ли пользователь право создавать альбомы. Эта проверка возвращает *true* только для премиум-пользователей и администраторов. Если пользователь является обычным (бесплатным), метод возвращает *Forbid()* с сообщением о необходимости приобретения премиум-подписки. Такое ограничение связано с тем, что альбомы требуют больше ресурсов и являются премиум-функцией системы.

После успешного прохождения проверки формируется команда *CreateAlbumCommand* с переданными данными. Обработчик выполняет проверку на дублирование: у пользователя не может быть двух альбомов с одинаковым названием. При успешном создании возвращается ответ с кодом 201 (*Created*) и объектом созданного альбома.

Метод *UpdateAlbum* принимает идентификатор альбома через маршрут и объект *UpdateAlbumDto* с обновлёнными данными. Команда *UpdateAlbumCommand* отправляется в обработчик, который проверяет существование альбома и является ли текущий пользователь его владельцем. Если альбом не найден, возвращается код 404. Если пользователь не является владельцем, выбрасывается *UnauthorizedAccessException*, и метод возвращает *Forbid()* (код 403). При успешном обновлении возвращается ответ с кодом 200 и обновлённым объектом альбома.

Метод *DeleteAlbum* принимает идентификатор альбома и формирует команду *DeleteAlbumCommand*. Обработчик выполняет аналогичные проверки прав доступа: пользователь может удалить только свой собственный альбом, а администратор – любой. Кроме того, при удалении альбома система автоматически обрабатывает связанные данные – например, подписки пользователей на этот альбом. При успешном удалении возвращается ответ с кодом 204 (*NoContent*). Такая архитектура обеспечивает контроль доступа к премиум-функциям и безопасное управление альбомами.

3.6.3 Реализация функций аутентификация с привилегиями

В отличие от обычной аутентификации, которая только проверяет учётные данные пользователя и выдаёт токены, аутентификация с привилегиями учитывает статус пользователя для предоставления расширенных возможностей в системе. Сам процесс входа в систему для премиум-пользователей и администраторов ничем не отличается от входа обычных пользователей – используется тот же метод *Login*, который проверяет *email* и пароль и возвращает токены. Однако после успешной аутентификации вступают в силу привилегии, связанные с ролью пользователя.

Главное отличие заключается в том, что при проверке прав доступа к определённым функциям система анализирует роль пользователя, полученную из токена. Премиум-пользователи и администраторы получают следующие привилегии: снятие лимитов на создание контента (песен, аккордов, паттернов), возможность создавать альбомы (доступно только премиум-пользователям и администраторам), а также расширенные права при обновлении и удалении контента.

Таким образом, можно сказать, что аутентификация с привилегиями – это тот же процесс входа в систему, но в результате которого пользователь получает токен, содержащий информацию о его роли (*Premium* или *Admin*). При последующих запросах система извлекает эту роль и предоставляет пользователю доступ к функциям, недоступным обычным пользователям. Например, при попытке создать альбом проверяется роль пользователя, и если она не соответствует премиум-статусу, операция отклоняется. При создании песен для премиум-пользователей не выполняется проверка лимитов, а администраторы могут удалять или изменять любой контент независимо от владельца. Такая архитектура позволяет гибко управлять доступом к функциям системы в зависимости от статуса пользователя.

3.7 Реализация функций для администратора

3.7.1 Реализация функции выдачи или изъятия прав администратора

Функция *ToggleUserRole* предназначена для управления ролями пользователей в системе – а именно для выдачи или изъятия прав администратора.

Это асинхронный метод, который обрабатывает *HTTP PUT*-запросы, поступающие по маршруту «*toggle-user-role*». Его задача – изменить роль указанного пользователя на противоположную: если пользователь был обычным (или премиум), он становится администратором; если был администратором – теряет эти права. Данная функция доступна только для действующих администраторов, что должно проверяться на уровне обработчика команды. Код реализации представлен в листинге 3.23.

```
[HttpPut("toggle-user-role")]
public async Task<ActionResult> ToggleUserRole([FromBody]
WorkWithUserByEmailDto dto){
    try{
        var currentAdminId = GetCurrentUserId();
        var command = new ToggleUserRoleCommand(
            Email: dto.Email,
            AdminId: currentAdminId);
        var result = await _mediator.Send(command);
        return Ok(new{
message = $"The user role with the email \"{dto.Email}\" has
been changed.\r\n",
            userId = result});}
    catch (KeyNotFoundException ex) {
        return NotFound(new { error = ex.Message });}
    catch (InvalidOperationException ex) {
        return BadRequest(new { error = ex.Message });}
    catch (Exception ex) {
        return BadRequest(new { error = ex.Message });}}
```

Листинг 3.23 – Функция выдачи или изъятия прав администратора

Метод *ToggleUserRole* принимает от клиента объект *WorkWithUserByEmailDto*, содержащий адрес электронной почты пользователя, чью роль необходимо изменить. Первым шагом метод получает идентификатор текущего пользователя (который должен быть администратором) с помощью *GetCurrentUserId()*, после чего формирует команду *ToggleUserRoleCommand* с *email* целевого пользователя и идентификатором администратора.

Команда отправляется через *MediatR* в соответствующий обработчик, который выполняет основную бизнес-логику. Обработчик сначала проверяет, существует ли пользователь с указанным *email* – если нет, выбрасывается *KeyNotFoundException*. Затем проверяется, имеет ли текущий пользователь (администратор, инициировавший операцию) права на выполнение этого действия. Важным ограничением является то, что администратор не может изменить роль самому себе – это предотвращает случайное или преднамеренное лишение себя прав. В такой ситуации выбрасывается *InvalidOperationException*.

Если все проверки пройдены успешно, обработчик изменяет роль пользователя. Обычно система имеет две основные роли с административными

правами: *Admin* и, возможно, *SuperAdmin*. При переключении роли обычный пользователь становится администратором, а администратор – обычным пользователем (или ему присваивается роль *User*). В случае успеха метод возвращает ответ с кодом 200 и сообщением об успешном изменении роли пользователя, включая идентификатор изменённого пользователя.

При возникновении ошибок метод обрабатывает *KeyNotFoundException* (пользователь не найден), *InvalidOperationException* (например, попытка изменить свою собственную роль), а также общее исключение, возвращая соответствующие коды 404 или 400. Такая архитектура обеспечивает безопасное управление привилегиями и позволяет администраторам гибко настраивать права доступа в системе.

3.7.2 Реализация функций блокировки и разблокировки пользователей

Функции *BlockUser* и *UnblockUser* предназначены для управления доступом пользователей к системе. Это асинхронные методы, которые обрабатывают *HTTP PUT*-запросы, поступающие по маршрутам «*block-user*» и «*unblock-user*». Их задача – временно или на постоянной основе заблокировать пользователя либо снять блокировку. Данные функции доступны только для администраторов системы. Код реализации представлен в листинге 3.24.

Метод *BlockUser* принимает от клиента объект *BlockUserDto*, содержащий адрес электронной почты пользователя, причину блокировки и дату окончания блокировки. Первым шагом метод получает идентификатор текущего администратора с помощью *GetCurrentUserId()*, после чего формирует команду *ToggleBlockStatusCommand* с *email* целевого пользователя, причиной блокировки, датой окончания и идентификатором администратора. Дата блокировки преобразуется в *UTC* для единообразного хранения в базе данных. Перед формированием команды метод также выполняет базовую валидацию входящих данных: проверяется, что *email* имеет корректный формат, а причина блокировки не пуста и не превышает максимально допустимой длины. Если какое-либо из этих условий нарушено, клиенту немедленно возвращается ошибка с кодом 400 и указанием конкретного поля, вызвавшего проблему. Дополнительно метод проверяет, что дата окончания блокировки, если она указана, не находится в глубоком прошлом, что позволило бы избежать бессмысленных блокировок с уже истёкшим сроком. При успешном прохождении всех проверок команда отправляется через *MediatR* в соответствующий обработчик, что обеспечивает слабую связанность между слоем контроллера и бизнес-логикой, а также упрощает последующее тестирование и расширение функциональности.

Команда отправляется через *MediatR* в обработчик, который выполняет основную логику. Обработчик проверяет существование пользователя с указанным *email*. Если пользователь не найден, выбрасывается

KeyNotFoundException. Также проверяется, что администратор не пытается заблокировать самого себя – это недопустимо, и в таком случае выбрасывается *InvalidOperationException*. Затем обработчик устанавливает пользователю статус блокировки, сохраняет причину и дату окончания блокировки. Если *BlockedUntil* указана в будущем, блокировка будет автоматически снята после этой даты; если дата не указана или указана в прошлом, блокировка считается бессрочной (до ручного снятия администратором).

```
[HttpPut("block-user")]
public async Task<ActionResult<BlockUserResponseDto>>
BlockUser([FromBody] BlockUserDto dto) {
    try{
        var currentAdminId = GetCurrentUserId();
        var command = new ToggleBlockStatusCommand(
            email: dto.Email,
            reason: dto.Reason,
            blockedUntil: dto.BlockedUntil.ToUniversalTime(),
            adminId: currentAdminId);
        var result = await _mediator.Send(command);
        return Ok(result); }
    catch (ArgumentException ex) {
        return BadRequest(new { error = ex.Message });}
    catch (KeyNotFoundException ex) {
        return NotFound(new { error = ex.Message });}
    catch (InvalidOperationException ex) {
        return BadRequest(new { error = ex.Message });}
    catch (Exception ex) {
        return BadRequest(new { error = ex.Message });}}

[HttpPut("unlock-user")]
public async Task<ActionResult<BlockUserResponseDto>>
UnblockUser([FromBody] WorkWithUserByEmailDto dto) {
    try{
        var currentAdminId = GetCurrentUserId();
        var command = new ToggleBlockStatusCommand(
            email: dto.Email,
            adminId: currentAdminId);
        var result = await _mediator.Send(command);
        return Ok(result); }
    catch (KeyNotFoundException ex) {
        return NotFound(new { error = ex.Message });}
    catch (InvalidOperationException ex) {
        return BadRequest(new { error = ex.Message });}
    catch (Exception ex) {
        return BadRequest(new { error = ex.Message });}}}
```

Листинг 3.24 – Функции блокировки и разблокировки пользователей

Метод *UnblockUser* работает аналогично, но принимает только *email* пользователя (объект *WorkWithUserByEmailDto*) и идентификатор администратора. Соответствующая команда *ToggleBlockStatusCommand* (без парамет-

ров причины и даты) снимает блокировку: очищает статус блокировки, причину и дату окончания. При успешном выполнении оба метода возвращают ответ с кодом 200 и объектом *BlockUserResponseDto*, содержащим информацию о статусе пользователя после операции.

При возникновении ошибок методы обрабатывают *ArgumentException* (некорректные входные данные, например, неверный формат *email*), *KeyNotFoundException* (пользователь не найден), *InvalidOperationException* (попытка заблокировать себя или уже заблокированного/разблокированного пользователя), возвращая соответствующие коды 400 или 404. Такая архитектура позволяет администраторам эффективно управлять доступом пользователей и реагировать на нарушения правил системы.

3.7.3 Реализация функции просмотра любого контента

Администраторы системы обладают особыми привилегиями при просмотре контента, в отличие от обычных и даже премиум-пользователей. Если обычные пользователи видят только публичный контент других пользователей и свой собственный (включая приватный), а неавторизованные пользователи – только публичный контент, то администратор имеет возможность просматривать абсолютно любой контент в системе без каких-либо ограничений.

Это означает, что администратор может видеть приватные песни, аккорды, паттерны и альбомы любого пользователя, независимо от того, кто является их владельцем и какой флаг приватности у них установлен. Администратор также может просматривать контент заблокированных пользователей, находить его через поиск и фильтрацию, а также получать детальную информацию о любой сущности по её идентификатору.

Такая возможность реализована на уровне обработчиков запросов (например, *SearchSongsQuery*, *GetAllChordsQuery*, *GetAllPatternsQuery*, *SearchAlbumsQuery*). В этих обработчиках выполняется проверка роли текущего пользователя. Если система определяет, что пользователь имеет роль администратора, все фильтры, ограничивающие выборку только публичным контентом или контентом текущего пользователя, просто игнорируются. Формируется запрос к базе данных, возвращающий все сущности без каких-либо условий на *IsPublic* или *OwnerId*. Это позволяет администратору эффективно модерировать контент, выявлять нарушения и управлять системой в целом.

3.7.4 Реализация функции удаления любого контента

Администраторы системы наделены расширенными правами не только при просмотре, но и при удалении любого контента, созданного любым пользователем. Если обычные пользователи могут удалять только свои собственные песни, аккорды, паттерны и альбомы, а премиум-пользователи также ограничены своим контентом, то администратор имеет возможность удалять абсолютно любой контент в системе независимо от владельца.

Это означает, что администратор может удалить любую песню, аккорд, ритмический паттерн или альбом, созданный любым пользователем системы, включая приватный контент и контент заблокированных пользователей. Такая возможность реализована на уровне обработчиков команд удаления – *DeleteSongCommand*, *DeleteChordCommand*, *DeletePatternCommand* и *DeleteAlbumCommand*. В этих обработчиках выполняется проверка роли текущего пользователя.

Важным ограничением является то, что даже администратор не может удалить самого себя или изменить свою роль, что защищает систему от случайной или преднамеренной потери единственного администратора. При удалении контента администратором система также обрабатывает связанные данные: например, при удалении песни удаляются или перепривязываются её отзывы, оценки, записи в избранном, а также аудиофайл, хранящийся на *WebDAV*-сервере. Такая архитектура позволяет администраторам эффективно модерировать контент, удаляя нарушения правил системы.

3.8 Маршруты контроллеров

В архитектуре *REST API* используется для работы с различными сущностями, такими как аутентификация, блокировки, чаты и другие, используются различные маршруты и методы *HTTP*. Каждый маршрут связан с определенным контроллером и «*action*», который выполняет необходимую операцию.

В таблице 3.1 описаны основные маршруты *HTTP*-сервера.

Таблица 3.1 – Описание маршрутов

URL	Метод	Контроллер	Описание
1	2	3	4
<i>/api/auth/register</i>	<i>POST</i>	<i>AuthController</i>	Регистрация с созданием аккаунта нового пользователя
<i>/api/songs</i>	<i>GET</i>	<i>SongsController</i>	Возможность просматривать песни, которые доступны для определенной роли
<i>/api/chords</i>	<i>GET</i>	<i>ChordsController</i>	Возможность просматривать аккорды, которые доступны для определенной роли
<i>/api/patterns</i>	<i>GET</i>	<i>PatternsController</i>	Возможность просматривать паттерны, которые доступны для определенной роли
<i>/api/chords/search</i>	<i>GET</i>	<i>ChordsController</i>	Возможность искать контент (например аккорды), по определенным полям и в определенном порядке

Продолжение таблицы 3.1

1	2	3	4
/api/albums	GET	AlbumsController	Возможность просматривать альбомы, которые доступны для определенной роли
/api/albums/favorite/{songId}	POST	AlbumsController	Возможность добавлять песню в избранное
/api/reviews/songs/{songId}	POST	SongsController	Возможность оставлять отзывы к песням
/api/subscriptions/{albumId}	POST	AlbumsController	Возможность подписываться на альбомы
/api/chords	POST	ChordsController	Возможность создания аккорда
/api/patterns	POST	PatternsController	Возможность создания паттерна
/api/songs	POST	SongsController	Возможность создания песни
/api/songs/{songId}/audio	POST	SongsController	Возможность добавлять для песни аудио
/api/payments/upgrade-to-premium	POST	PaymentsController	Возможность купить премиум роль
/api/auth/login	POST	AuthController	Возможность аутентифицироваться
/api/albums	POST	AlbumsController	Возможность создания альбомов
/api/usermanagement/toggle-user-role	PUT	UserManagementController	Возможность выдачи прав администратору обычному пользователю
/api/usermanagement/block-user	PUT	UserManagementController	Возможность заблокировать пользователя

В представленной таблице маршрутов *REST API* охвачен широкий спектр операций: от базовой аутентификации и управления пользователями (регистрация, вход, блокировка, выдача прав) до полноценного управления музыкальным контентом (создание и просмотр песен, аккордов, паттернов, альбомов), а также взаимодействия с пользовательскими предпочтениями (добавление в избранное, подписка на альбомы, отзывы к песням) и обработки платежей за премиум-доступ. Такая структура чётко разделяет ответственность между контроллерами, а соответствие *HTTP*-методов выполняемым действиям обеспечивает интуитивную понятность *API*. Завершая описание, стоит отметить, что для повышения безопасности и надёжности все маршруты, кроме /api/auth/register и /api/auth/login, должны быть защищены *middleware*-проверкой аутентификации, а для методов изменения данных (особенно

`/api/usermanagement/toggle-user-role` и `/api/usermanagement/block-user`) необходима дополнительная авторизация с проверкой роли администратора.

3.9 Реализация клиентской части при помощи библиотеки React

3.9.1 Реализация структуры проекта

Клиентская часть выполнена как современное веб-приложение на *React* и *TypeScript* с маршрутизацией *Next.js App Router*. Структура каталогов ориентирована на модульность и предсказуемое размещение кода. Основные папки приведены в таблице 3.2.

Таблица 3.2 – Описание структуры проекта

Папка	Описание
<i>app</i>	Маршруты и страницы (<i>App Router</i>), корневой <i>layout</i> , глобальные стили (<i>globals.css</i>), группы маршрутов (например, (<i>auth</i>)), вложенные разделы <i>home/*</i> .
<i>app/contexts</i>	<i>React</i> -контексты и связанное с ними состояние фич (редактор таблицы песни, сценарии создания и редактирования композиций).
<i>components</i>	Переиспользуемые компоненты интерфейса: предметная область (песни, аккорды, паттерны, альбомы), администрирование, провайдеры, а также набора базовых <i>ui</i> -элементов.
<i>components/layout</i>	Элементы каркаса интерфейса: шапка, переключение темы и языка.
<i>components/provider</i>	Обёртки приложения: авторизация, тема, тосты, локализация.
<i>hooks</i>	Пользовательские хуки: работа с аудио, квотами, уведомлениями, редактором песни, <i>debounce</i> и др.
<i>lib</i>	Инфраструктурный код: <i>HTTP</i> -клиент и сервисы <i>API</i> (<i>lib/api</i>), локализация (<i>lib/i18n</i>), конвертация таблицы/песни, работа со звуком, валидации.
<i>lib/utils</i>	Вспомогательные утилиты (разбор ошибок <i>API</i> , пароль, общие функции).
<i>types</i>	Описание типов сущностей и <i>DTO</i> для типобезопасного взаимодействия с <i>API</i> и <i>UI</i> .
<i>public</i>	Статические файлы, доступные по <i>URL</i> (изображения, <i>SVG</i> и др.).

В отличие от шаблонов «страница = файл в *pages*», здесь страницы соответствуют файлам *page.tsx* внутри *app*, что является принятой моделью *Next.js 13+*. Такое разнесение упрощает сопровождение: маршруты, контексты фич, визуальные блоки и обращение к серверу лежат в разных, чётко обозначенных зонах.

3.9.2 Реализация компонент

Компоненты формируют пользовательский интерфейс: от форм входа до редактора песни, библиотек аккордов и паттернов, управления пользователями. Ниже (таблица 3.3) приведены основные компоненты по функциональным группам; каталог *components/ui* содержит низкоуровневые примитивы (кнопки, поля ввода, диалоги и т.д.), на которых строится остальной интерфейс.

Таблица 3.3 – Описание компонент

Компонента	Описание
<i>LoginForm, RegisterForm</i>	Формы входа и регистрации.
<i>AuthProvider</i>	Контекст авторизации и текущего пользователя.
<i>Header</i>	Шапка и навигация по приложению.
<i>SongTextEditor, Song-TextViewer</i>	Редактирование и просмотр песни.
<i>SegmentTable</i>	Таблица сегментов строки (ядро редактора).
<i>ToolPanel, EditToolPanel</i>	Панели инструментов редактора.
<i>AddChordModal, AddPattern-Modal</i>	Добавление аккорда и паттерна в песню.
<i>AudioInputPanel, AudioPlayer-Panel</i>	Привязка аудиофайла и воспроизведение.
<i>ChordDisplay, ChordGrid, Fret-boardEditor</i>	Просмотр, список и редактирование аккордов.
<i>PatternGrid, StrummingEditor, FingerStyleEditor</i>	Просмотр и создание паттернов боя/щипка.
<i>AlbumCover, ConfirmDialog</i>	Альбомы: обложка и подтверждение действий.
<i>ProfilePanel</i>	Профиль пользователя.
<i>UserManagementTable, ManageBlockDialog</i>	Администрирование пользователей и блокировки.
<i>LimitWarningAlert</i>	Предупреждение о лимитах создания контента.

Остальной UI (*components/ui*, тосты, тема, локализацию) при желании упомяните одной фразой в тексте после таблицы: «Базовые элементы интерфейса реализованы набором компонентов в каталоге *components/ui* и провайдерами (*ThemeProvider, ToastProvider, I18nProvider*)».

3.9.3 Реализация хранилища при помощи библиотеки Redux Toolkit

Глобальное и привязанное к отдельным сценариям состояние в клиентском приложении не реализовано через *Redux Toolkit*. Используются встроенные возможности *React* – контекст, локальное состояние (*useState*), а для сложной логики в границах одной фичи – централизованные обновления через *useReducer*. Обмен с сервером идёт через сервисный слой в каталоге *lib/api* на базе *HTTP*-клиента с единой настройкой адреса *API*, заголовков авторизации и разбора ошибок.

Интерфейсные сценарии покрываются контекстами и локальным состоянием: текущий пользователь и сценарии входа и выхода, состояние табличного редактора песни (сегменты, связь с аккордами и паттернами, замена и переупорядочивание), а также контексты сценариев создания и редактирования композиций. У табличного редактора редьюсер сгруппирован с контекстом и задаёт правила смены состояния без отдельного стора *Redux*. Запросы к *REST API* инкапсулированы в сервисах: аутентификация и профиль, песни, аккорды, паттерны, альбомы, подписки и платежи, уведомления, отзывы. Поведение, сходное с автоматическим кэшем *RTK Query*, не вынесено в отдельную библиотеку – актуальность данных подтверждается повторными запросами и обновлением состояния в компонентах и контекстах после успешных операций.

Сводную картину модулей состояния и доступа к данным даёт таблица 3.4.

Таблица 3.4 – Логические модули состояния и доступа к данным

Модуль	Описание
<i>AuthProvider / AuthService</i>	Состояние аутентификации: токен, текущий пользователь, вход, регистрация, выход, обновление профиля.
<i>table-editor-context</i>	Состояние табличного редактора песни: сегменты, аккорды и паттерны в рамках редактирования, цветовое сопоставление, операции над сегментами и замена ресурсов.
Контексты создания и редактирования песни	Данные и этапы сценариев создания и изменения композиции.
<i>SongService</i>	Запросы к <i>API</i> по песням: списки, детали, создание, изменение, удаление.
<i>ChordsService</i>	Работа с аккордами через <i>API</i> .
<i>PatternsService</i>	Работа с паттернами исполнения через <i>API</i> .
<i>AlbumService</i>	Альбомы и их состав на стороне <i>API</i> .
<i>ProfileService</i>	Профиль пользователя.
<i>SubscriptionsService / PaymentsService</i>	Подписки и платёжные операции.
<i>NotificationsService</i>	Уведомления.
<i>ReviewService</i>	Отзывы и оценки.

Итог в том, что приложение обходится без единого глобального хранилища *Redux* и без *RTK Query*: ясность архитектуры обеспечивается контекстами по фи-чам, обработчиком состояний в контексте редактора и явным слоем *HTTP*-серви-сов, согласованным с типами. Такой состав подходит для объёма клиентской части курсового проекта и сохраняет строгую типизацию на уровне *TypeScript*.

3.10 Используемые форматы передачи данных

В клиент-серверном взаимодействии проекта основной формат обмена дан-ными – *JSON*. Через *JSON* передаются *DTO* для авторизации, профиля, песен, ак-кордов, паттернов, альбомов, уведомлений, отзывов и подписок. Такой подход обеспечивает типобезопасную обработку на фронтенде (*TypeScript*), удобную се-риализацию на бэкенде и единый формат для большинства *GET*, *POST*, *PUT* и *DELETE* запросов.

Для операций, где требуется отправка файла, применяется формат *multipart/form-data*. Это актуально для сценариев, связанных с медиа-контентом проекта, в первую очередь при загрузке аудио к песне и в других формах, где вме-сте с файлом передаются дополнительные поля (например, идентификаторы сущ-ностей, названия или служебные параметры). В таком запросе тело состоит из не-скольких частей: текстовые поля и бинарные данные передаются совместно, но разделяются специальными границами *multipart*-формата.

Ответы *API* в стандартных бизнес-операциях также возвращаются в *JSON*. При работе с медиа сервер может отдавать не *JSON*-объект, а непосредственно файл или бинарный поток с соответствующим *MIME*-типом. Для аудио это, как правило, поток аудиоданных; для изображений – графические форматы, пригод-ные для прямого отображения в интерфейсе. Таким образом, в проекте использу-ется комбинированная модель: *JSON* как базовый формат для прикладных данных и *multipart/form-data*/бинарные ответы для медиа-операций.

3.11 Выводы по разделу

В ходе разработки веб-приложения была спроектирована и реализована полнофункциональная музыкальная платформа с ролевой моделью доступа, включающей четыре уровня привилегий: гость, гитарист, премиум и администра-тор. Архитектура бэкенда построена на *ASP.NET Core* с использованием совре-менных паттернов проектирования: *Mediator* для снижения связанности компо-нентов, *CQRS* для разделения операций чтения и записи, а также *Repository* и *Unit of Work* через *Entity Framework Core* для абстрагирования доступа к данным. В ка-честве базы данных выбрана *PostgreSQL*, обеспечивающая надежность и под-держку сложных запросов, а для контейнеризации используется *Docker* с *Docker Compose*, что гарантирует согласованность среды исполнения.

4 Тестирование веб-приложения

Тестирование веб-приложения представляет собой неотъемлемую часть процесса обеспечения качества программного продукта и направлено на выявление ошибок, несоответствий требованиям и потенциальных уязвимостей на ранних этапах жизненного цикла разработки. Оно позволяет не только убедиться в работоспособности отдельных компонентов, но и в надёжности и устойчивости системы в целом при различных условиях эксплуатации.

Перед проведением тестирования была выполнена подготовка тестовой среды, включающей в себя развертывание серверной части приложения в изолированном контейнере, запуск клиентского интерфейса в современном браузере и подключение к тестовой базе данных. Все элементы среды были синхронизированы и имитировали реальные условия функционирования, что позволило воспроизвести типичные пользовательские сценарии.

Особое внимание в ходе тестирования уделялось воспроизводимости ошибок, корректности отображения интерфейсных компонентов, а также логике обработки пользовательского ввода. Для этого были подготовлены входные данные, охватывающие как корректные, так и преднамеренно ошибочные или граничные значения. Такой подход обеспечивает проверку устойчивости системы к непредсказуемым действиям пользователя и демонстрирует её способность справляться с ошибками, не нарушая общей работоспособности.

Важно отметить, что тестированию подвергались как публичные, так и защищённые функциональные области приложения. Последние включают механизмы авторизации и аутентификации, проверку прав доступа к закрытым разделам, управление данными в рамках пользовательских сессий и корректную обработку исключений при попытке несанкционированного доступа. При этом особое внимание уделялось тому, чтобы сообщения об ошибках были информативными, но при этом не раскрывали внутреннюю структуру веб-приложения, что является требованием информационной безопасности.

Кроме того, в рамках тестирования оценивалась стабильность интерфейса и соответствие его элементов требованиям к юзабилити. Это включало проверку на наличие визуальных дефектов, несоответствие элементам дизайна, некорректную навигацию и прочие аспекты, влияющие на пользовательский опыт. Все найденные несоответствия были зафиксированы в списке задач и устранены.

Таким образом, тестирование выполнялось поэтапно, с применением как ручных, так и автоматизированных методов, и включало в себя анализ работы приложения в типичных, граничных и нестандартных сценариях использования.

					ДП 04.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.		Селицкий Н.Е.			4 Тестирование веб-приложения	Лит.	Лист	Листов
Пров.		Харланович А.В.				У	1	9
						БГТУ 1-40 01 01, 2025		
Н. контр.		Авдеева В.Д.						
Утв.		Смелов В.В.						

4.1 Функциональное тестирование

Функциональное тестирование направлено на проверку корректной работы функций приложения в различных ситуациях, включая стандартные и крайние случаи. Ниже приведена таблица 4.1, содержащая описание тестов, их ожидаемые результаты и статус выполнения.

Таблица 4.1 – Описание функциональных тестов

Но- мер	Функция	Совершаемые действия	Ожидаемый результат	Итог те- стирова- ния функ- ции
1	2	3	4	5
1	Регистрация	Открыть <i>/register</i> . Заполнить <i>Email, Nickname, Password</i> (не менее 8 символов, заглавные/строчные, цифра и спецсимвол) и <i>Confirm Password</i> тем же значением. Нажать « <i>Create Account</i> ».	Пересылка на <i>/home</i> , уведомление об успешной регистрации, пользователь авторизован.	Успешно
2	Просмотр публичных песен	Войти в систему. На <i>/home</i> нажать « <i>Go to Songs</i> ».	Открывается <i>/home/songs</i> со списком публичных песен.	Успешно
3	Просмотр публичных аккордов	Войти в систему. На <i>/home</i> нажать « <i>Go to Chords</i> ».	Открывается <i>/home/chords</i> с каталогом доступных аккордов.	Успешно
4	Просмотр публичных паттернов	Войти в систему. На <i>/home</i> нажать « <i>Go to Patterns</i> ».	Открывается <i>/home/patterns</i> со списком паттернов боя/щипка.	Успешно
5	Фильтрация и сортировка контента	На страницах песен, аккордов, паттернов или альбомов задать поисковую строку, фильтры (жанр, тема, видимость) и параметры сортировки (поле и порядок).	Список обновляется и содержит только записи, удовлетворяющие выбранным критериям.	Успешно
6	Просмотр своего контента	Войти под пользователем. Открыть раздел с любым контентом и/или включить фильтр « <i>Only my ...</i> » в соответствующем каталоге.	Отображаются только публичные объекты и/или созданные текущим пользователем.	Успешно

Продолжение таблицы 4.1

1	2	3	4	5
7	Просмотр своих и публичных паттернов	На <i>/home/patterns</i> просмотреть общий список, затем включить и выключить чекбокс « <i>Only my patterns</i> ».	Без фильтра видны публичные и свои паттерны; с фильтром – только собственные.	Успешно
8	Просмотр своих и публичных аккордов	На <i>/home/chords</i> просмотреть общий список, затем включить и выключить « <i>Only my chords</i> ».	Без фильтра видны публичные и свои аккорды; с фильтром – только собственные.	Успешно
9	Просмотр своих и публичных песен	На <i>/home/songs</i> просмотреть каталог, включить « <i>Only my songs</i> », открыть любую публичную и любую собственную песню.	Фильтр оставляет только свои песни; детальные страницы <i>/home/songs/[songId]</i> открываются корректно.	Успешно
10	Просмотр своих и публичных альбомов	На <i>/home/albums</i> просмотреть каталог, включить « <i>Only my Albums</i> », открыть альбом.	Фильтр показывает только свои альбомы; страница <i>/home/albums/[albumId]</i> отображает метаданные и состав.	Успешно
11	Добавление/удаление песен в/из избранного	Открыть публичную песню не владельца. Нажать « <i>Add to Favorites</i> », затем повторно – для удаления из избранного.	Кнопка меняет состояние (« <i>In Favorites</i> » / « <i>Add to Favorites</i> »); песня появляется и исчезает в альбоме Favorite.	Успешно
12	Написать отзыв и поставить оценки песен	Открыть чужую публичную песню, заполнить текст отзыва, при необходимости указать оценки красоты и сложности, нажать « <i>Submit Review</i> ».	Отзыв отображается в списке отзывов песни с указанными оценками.	Успешно

Продолжение таблицы 4.1

1	2	3	4	5
13	Создание, обновление и удаление контента с лимитами	Войти под бесплатным пользователем без <i>Premium</i> . Создать песни/аккорды/паттерны/альбомы до исчерпания квоты, затем повторить создание.	До лимита операции выполняются; при превышении отображается предупреждение <i>LimitWarning</i> и создание блокируется.	Успешно
14	Создание или удаление подписки	На странице альбома или пользователя нажать « <i>Subscribe</i> », проверить <i>/home/subscriptions</i> , затем отписаться.	Подписка появляется в списке; после отмены запись удаляется, уведомления по ней не приходят.	Успешно
15	Создание, обновление и удаление аккордов	На <i>/home/chords/create</i> создать аккорд (имя, аппликатура на грифе). Отредактировать через <i>Edit</i> . Удалить через <i>Delete</i> с подтверждением.	Аккорд создаётся, изменения сохраняются, после удаления отсутствует в каталоге.	Успешно
16	Создание, обновление и удаление паттерна	На <i>/home/patterns/create</i> задать имя и схему боя или щипка. Изменить на <i>/home/patterns/edit/[id]</i> . Удалить через диалог подтверждения.	Паттерн создаётся, обновляется и удаляется без ошибок интерфейса.	Успешно
17	Создание, обновление и удаление песен	На <i>/home/songs/create</i> заполнить метаданные, таблицу сегментов, аккорды и паттерны, сохранить. Изменить на <i>/home/songs/edit/[songId]</i> . Удалить с подтверждением.	Песня создаётся и открывается на <i>/home/songs/[songId]</i> ; правки сохраняются; после удаления отсутствует в списке.	Успешно
18	Создание, обновление и удаление аудио для песен	В редакторе песни выбрать режим <i>Record</i> или <i>Upload</i> , записать/загрузить аудио, сохранить. Прослушать на странице песни. Удалить или заменить файл.	Аудиоплеер отображается под информацией о песне; воспроизведение работает; удаление убивает блок аудио.	Успешно

Продолжение таблицы 4.1

1	2	3	4	5
19	Купить премиум	Перейти на <i>/home/premium</i> , заполнить платёжную форму (номер карты, держатель, срок, CVC), нажать кнопку оплаты.	<i>hasPremium</i> становится <i>true</i> , лимиты снимаются, отображается статус <i>Premium</i> .	Успешно
20	Аутентификация без привилегий	Войти на <i>/login</i> под обычным пользователем или работать как <i>Guest</i> . Попробовать открыть <i>/home/user-management</i> .	Вход выполняется; административные разделы недоступны; при ошибочных данных входа показывается сообщение об ошибке.	Успешно
21	Создание, обновление и удаление контента без лимита	Войти под <i>Premium</i> или <i>Admin</i> . Создать несколько песен, аккордов, паттернов и альбомов сверх бесплатной квоты.	Ограничения не применяются; все <i>CRUD</i> -операции выполняются успешно.	Успешно
22	Создание, обновление и удаление альбомов	На <i>/home/albums/create</i> задать название, видимость и описание, создать альбом. Изменить на <i>/home/albums/edit/[albumId]</i> . Удалить альбом.	Альбом открывается на <i>/home/albums/[albumId]</i> ; изменения сохраняются; после удаления не отображается в каталоге.	Успешно
23	Аутентификация с привилегиями	Войти на <i>/login</i> под учётной записью <i>Admin</i> .	Пересылка на <i>/home</i> ; в меню доступен пункт <i>User Management</i> ; видны все песни и приватный контент.	Успешно
24	Выдача/изъятие прав администратора	<i>Admin</i> : <i>/home/user-management</i> затем « <i>Make Admin</i> » у выбранного пользователя, подтвердить; затем « <i>Remove Admin</i> ».	Роль пользователя меняется; кнопка переключается между <i>Make Admin</i> и <i>Remove Admin</i> .	Успешно

Продолжение таблицы 4.1

1	2	3	4	5
25	Блокировка/разблокировка пользователя	<i>Admin</i> : « <i>Block</i> » затем указать причину и срок « <i>Block User</i> ». Затем « <i>Manage Block</i> » / <i>Unblock</i> .	Заблокированный пользователь не может войти и видит причину и срок; после разблокировки вход восстанавливается.	Успешно
26	Просмотр любого контента	<i>Admin</i> : открыть приватную песню, альбом, аккорд или паттерн другого пользователя.	Контент отображается независимо от флага публичности и владельца.	Успешно
27	Удаление любого контента	<i>Admin</i> : открыть чужую песню или альбом, нажать <i>Delete</i> , подтвердить.	Объект удаляется из системы и исчезает из общих списков.	Успешно

Было проведено 27 функциональных тестов, которые показали, что веб-приложение работает корректно.

4.2 Модульное тестирование

Модульное тестирование направлено на проверку отдельных компонентов серверной логики (*handlers, validators, services, policies*) и клиентских утилит (валидация пароля, разбор ошибок *API*) в изоляции от *UI* и базы данных с использованием *mock*-объектов. Таблица 4.2 содержит те же функциональные области, что и таблица 4.1, но с фокусом на модульные проверки.

Таблица 4.2 – Описание модульных тестов

Номер	Функция	Совершаемые действия	Ожидаемый результат	Итог тестирования функции
1	2	3	4	5
1	Регистрация	<i>Unit</i> -тест <i>RegisterUserHandler</i> / валидации <i>RegisterRequest</i> : корректный <i>DTO</i> , несовпадение паролей, слабый пароль, дубликат <i>email</i> .	<i>Handler</i> возвращает успех или <i>ValidationException</i> с ожидаемым кодом ошибки.	Успешно

Продолжение таблицы 4.2

1	2	3	4	5
2	Просмотр публичных песен	Unit-тест <i>SongQueryService.GetPublicSongs</i> : фильтрация <i>IsPublic=true</i> , пагинация.	Возвращается только публичный набор <i>DTO</i> .	Успешно
3	Просмотр публичных аккордов	Unit-тест <i>ChordRepository/GetAll</i> с <i>mock</i> БД: публичные записи.	Список не содержит частных чужих объектов.	Успешно
4	Просмотр публичных паттернов	Unit-тест <i>PatternService.GetAllPatterns</i> .	Корректный маппинг <i>PatternDto</i> .	Успешно
5	Фильтрация и сортировка контента	Unit-тесты методов <i>ApplyFilters</i> и <i>ApplySort</i> для <i>SongSearchQuery</i> .	Порядок и состав коллекции соответствуют параметрам.	Успешно
6	Просмотр своего контента	Unit-тест фильтра <i>CreatedById == currentUserId</i> .	В выборке только записи владельца.	Успешно
7	Просмотр своих и публичных паттернов	Unit-тест комбинированного предиката (<i>public OR owner</i>).	Объединение наборов без дубликатов.	Успешно
8	Просмотр своих и публичных аккордов	Unit-тест комбинированного предиката для аккордов.	Корректное разделение <i>own/public</i> .	Успешно
9	Просмотр своих и публичных песен	Unit-тест <i>SongRepository</i> для <i>my-songs</i> и <i>public list</i> .	Два сценария возвращают ожидаемые множества.	Успешно
10	Просмотр своих и публичных альбомов	Unit-тест <i>AlbumService.GetMyAlbums / SearchAlbums</i> .	<i>DTO</i> содержат метаданные и флаг видимости.	Успешно
11	Добавление/удаление песен в/из избранного	Unit-тест <i>FavoriteAlbumService.AddSong / RemoveSong</i> .	Песня добавляется в системный альбом <i>Favorite</i> один раз.	Успешно
12	Написать отзыв и поставить оценки песне	Unit-тест <i>CreateReviewValidator</i> : текст, диапазон оценок 1–5.	Невалидные оценки отклоняются.	Успешно
13	Создание контента с лимитами	Unit-тест <i>CreationQuotaService</i> для <i>User</i> без <i>Premium</i> .	При <i>count >= limit</i> выбрасывается <i>QuotaExceededException</i> .	Успешно

Продолжение таблицы 4.2

1	2	3	4	5
14	Создание или удаление подписки	Unit-тест <i>SubscriptionService.Subscribe/Unsubscribe</i> .	Запись создаётся и удаляется.	Успешно
15	CRUD аккордов	Unit-тесты <i>CreateChord, UpdateChord, DeleteChord</i> с <i>mock</i> репозитория.	Проверка прав владельца и маппинга полей.	Успешно
16	CRUD паттернов	Unit-тесты <i>PatternCommandHandler create/update/delete</i> .	Изменения сохраняются только для автора.	Успешно
17	CRUD песен	Unit-тест <i>BuildSongStructure</i> : сегменты, <i>chordId, patternId</i> .	Структура сериализуется без потери порядка.	Успешно
18	CRUD аудио для песен	Unit-тест <i>AudioAttachmentService</i> : <i>MIME</i> , размер файла.	Неподдерживаемый формат отклоняется.	Успешно
19	Купить премиум	Unit-тест <i>PaymentsService.UpgradeToPremium</i> : <i>mock</i> платёжного шлюза.	<i>hasPremium</i> устанавливается при <i>success=true</i> .	Успешно
20	Аутентификация без привилегий	Unit-тест <i>JwtTokenGenerator</i> для роли <i>User</i> ; <i>Authorize(Roles=Admin)</i> policy.	Токен содержит <i>role=User</i> ; <i>policy fails</i> для <i>Admin</i> .	Успешно
21	Создание без лимита	Unit-тест <i>CreationQuotaService</i> для <i>Premium/Admin</i> .	<i>CanCreate</i> всегда <i>true</i> .	Успешно
22	CRUD альбомов	Unit-тесты <i>AlbumCommandHandler create/update/delete</i> .	Соблюдение прав и связей.	Успешно
23	Аутентификация с привилегиями	Unit-тест генерации <i>JWT</i> для <i>Admin</i> .	<i>Claim role=Admin</i> присутствует.	Успешно
24	Выдача/изъятие прав администратора	Unit-тест <i>ToggleUserRoleHandler</i> .	Роль переключается <i>User ⇔ Admin</i> .	Успешно
25	Блокировка/разблокировка пользователя	Unit-тест <i>BlockUserHandler / UnblockUserHandler</i> .	<i>IsBlocked</i> и <i>BlockedUntil</i> обновляются.	Успешно
26	Просмотр любого контента	Unit-тест <i>Admin bypass</i> в <i>SongAccessPolicy</i> .	<i>Admin</i> получает доступ ко всем объектам.	Успешно
27	Удаление любого контента	Unit-тест <i>AdminDeleteSongHandler</i> .	Удаление чужого объекта разрешено только <i>Admin</i> .	Успешно

Выполнено 27 модульных проверок ключевых компонентов; критичная бизнес-логика обрабатывается предсказуемо.

4.3 Выводы по разделу

В процессе тестирования были проверены ключевые процессы веб-приложения, включая регистрацию и авторизацию пользователей, работу с файлами, отправку сообщений, а также управление чатами и стикерами. Особое внимание уделялось корректности обработки ошибочных ситуаций, поскольку от этого зависит пользовательский опыт и устойчивость системы к некорректным действиям клиентов. Все предусмотренные тестовыми сценариями функции продемонстрировали корректное поведение: при возникновении ошибок система возвращает понятные текстовые и лаконичные уведомления, информируя пользователя о причинах, по которым запрашиваемое действие не может быть выполнено.

В ходе функционального тестирования было проведено 27 тестов, охватывающих основные сценарии использования системы. Проверка авторизации показала, что при вводе некорректного *email* или пароля приложение возвращает соответствующую ошибку и не допускает пользователя в систему. Тестирование загрузки файлов подтвердило, что система корректно отклоняет слишком большие файлы, не превышая установленные лимиты. Также были проверены операции создания папок и поиска пользователей с неверными параметрами: во всех случаях приложение выдаёт уведомления об ошибках, не выполняя некорректных действий. При отправке сообщений в удалённые чаты и при попытке редактирования несуществующих сообщений ошибки также были обработаны верно – пользователь получает понятное сообщение, а система сохраняет стабильность. Работа со стикерами была протестирована на предмет загрузки файлов с неверным форматом: приложение отклоняет такие вложения и уведомляет пользователя о неподдерживаемом типе.

Помимо функционального тестирования, было проведено нагрузочное тестирование на двух наиболее критичных функциях веб-приложения: поиск пользователей по фотографии и отправка сообщений. Результаты нагрузочного тестирования показали, что для стабильной работы приложения под высокой нагрузкой необходимы мощные серверные ресурсы, способные обрабатывать большое количество одновременных клиентских запросов. При низкой и средней интенсивности запросов веб-приложение отработало корректно, демонстрируя приемлемое время отклика и отсутствие ошибок. Однако при возрастании нагрузки до высоких значений наблюдаются задержки, что указывает на необходимость оптимизации соответствующих эндпоинтов либо масштабирования серверной инфраструктуры. В целом, система прошла функциональные проверки успешно, а выявленные в ходе нагрузочного тестирования узкие места могут быть устранены путём кэширования, оптимизации запросов к базе данных или горизонтального масштабирования.

5 Руководство пользователя

5.1 Руководство клиента

5.1.1 Регистрация

При открытии сайта гость попадает на страницу для аутентификации. Страницу для аутентификации представлена на рисунке 5.1.

Рисунок 5.1 – Страницу для аутентификации

Если гость уже имеет аккаунт, то он может ввести свой электронный адрес и пароль и получить права роли Гитарист.

Со страницы для аутентификации гость может перейти на страницу регистрации нажав на кнопку «Sign up» сверху или снизу страницы. Форма регистрации представлена на рисунке 5.2.

Рисунок 5.2 – Форма регистрации

					ДП 05.00.ПЗ				
		ФИО	Подпись	Дата					
Разраб.	Селицкий Н.Е.				5 Руководство пользователя	Лит.	Лист	Листов	
Пров.	Харланович А.В.					У	1	12	
						БГТУ 1-40 01 01, 2025			
Н. контр.	Авдеева В.Д.								
Утв.	Смелов В.В.								

Здесь необходимо заполнить форму, указав свою, электронную почту, никнейм и пароль. При нажатии на кнопку «*Create Account*», будет произведена попытка регистрации нового пользователя.

После успешной регистрации или аутентификации, пользователь получает права роли Гитарист.

5.2 Руководство Гитариста

5.2.1 Просмотреть песен

На главной странице Гитариста есть навигационные кнопки. Страница /home представлена на рисунке 5.3.

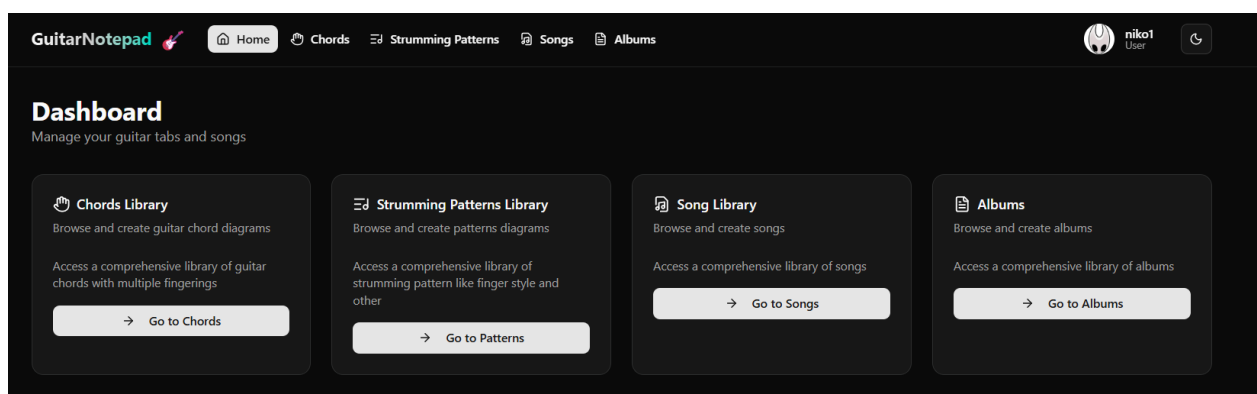


Рисунок 5.3 – Главная страница

Если перейти по *Go to Songs*, то откроется страница для просмотра всех песен, которые написал пользователь и публичные песни других пользователей. Страница /home/songs представлена на рисунке 5.4.

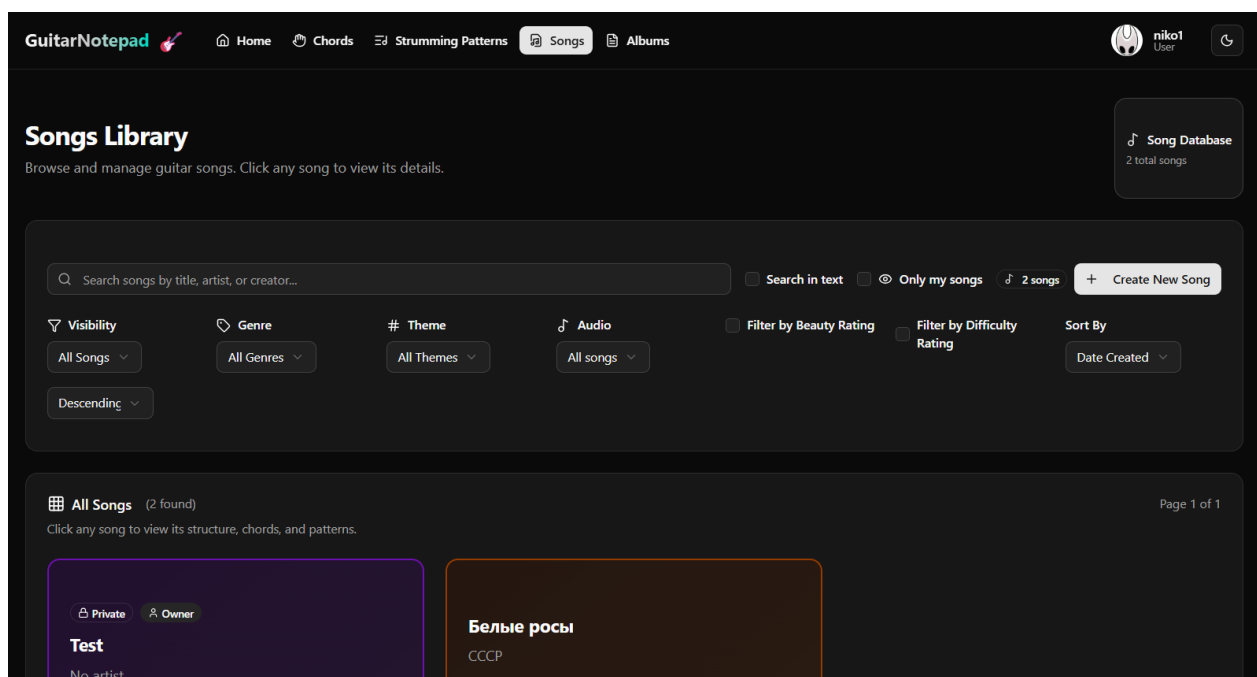


Рисунок 5.4 – страница /home/songs

На этой странице есть фильтры, которые позволяют быстрее найти нужные песни. Есть фильтры по названию, жанру, теме, красоте, сложности, а так же выводить только мои песни.

5.2.2 Создание песни

При нажатии на кнопку Create new Song откроется страница `/home/songs/create`. Страница `/home/songs/create` представлена на рисунке 5.5.

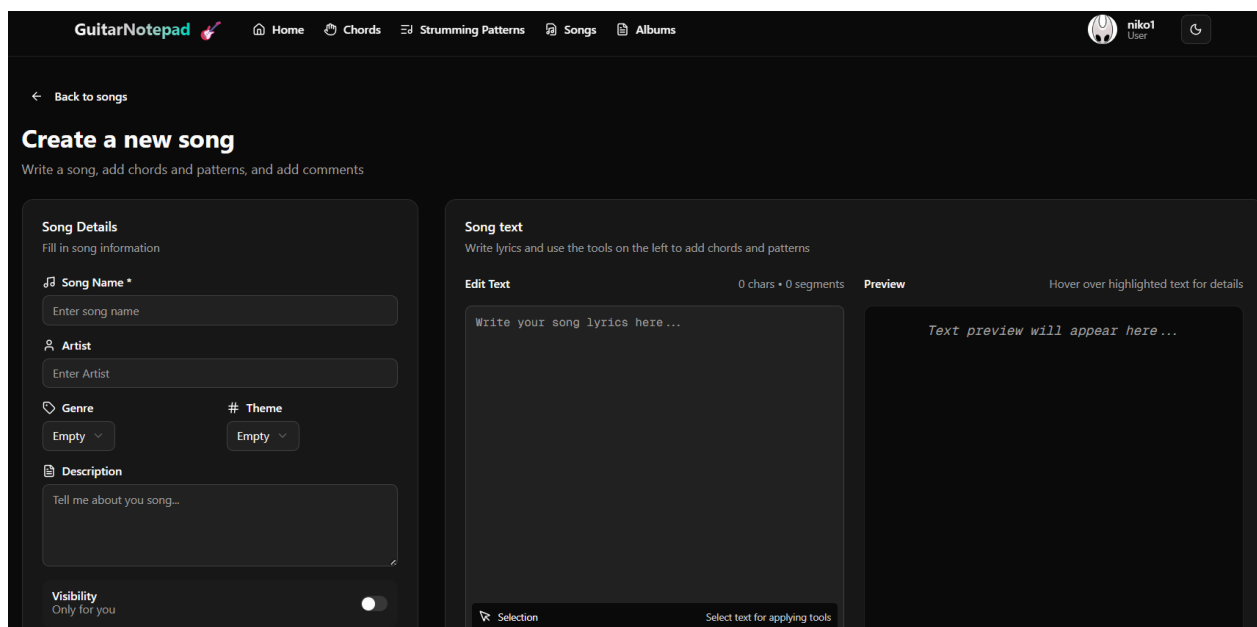


Рисунок 5.5 – страница `/home/songs/create`

Эта страница нужна для создания песен с использованием области отображения и области инструментов.

5.2.3 Запись аудио

На странице создания песен есть возможность записать аудиофайл. Окно для записи аудиофайла представлено на рисунке 5.6.

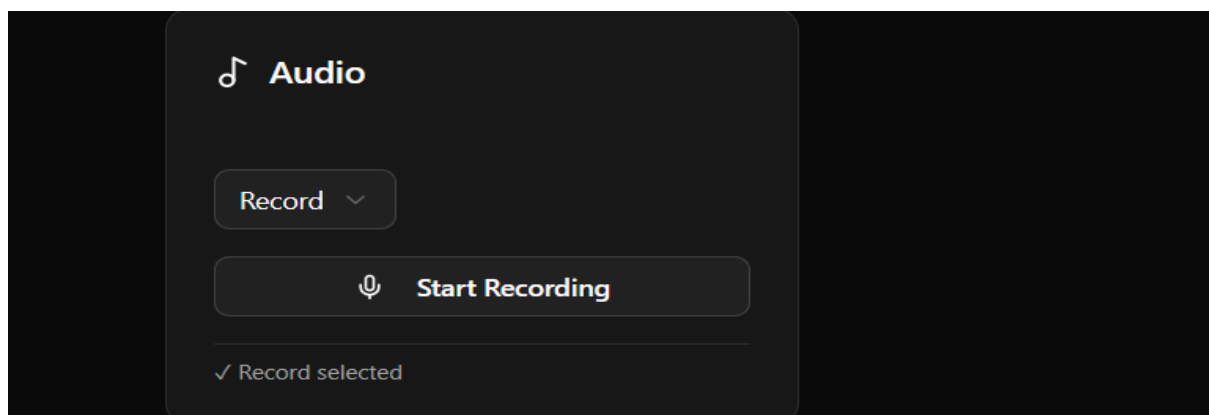


Рисунок 5.6 – Блок для записи аудиофайла

В этом блоке можно не только записывать аудио, но и загружать его и кидать ссылки на иные ресурсы.

5.2.4 Просмотр информации о песне

На странице `/home/song` при нажатии на любую песню открывается страница для рассмотрения `/home/songs/[songId]`.

Страница `/home/songs/[songId]` представлена на рисунке 5.7.

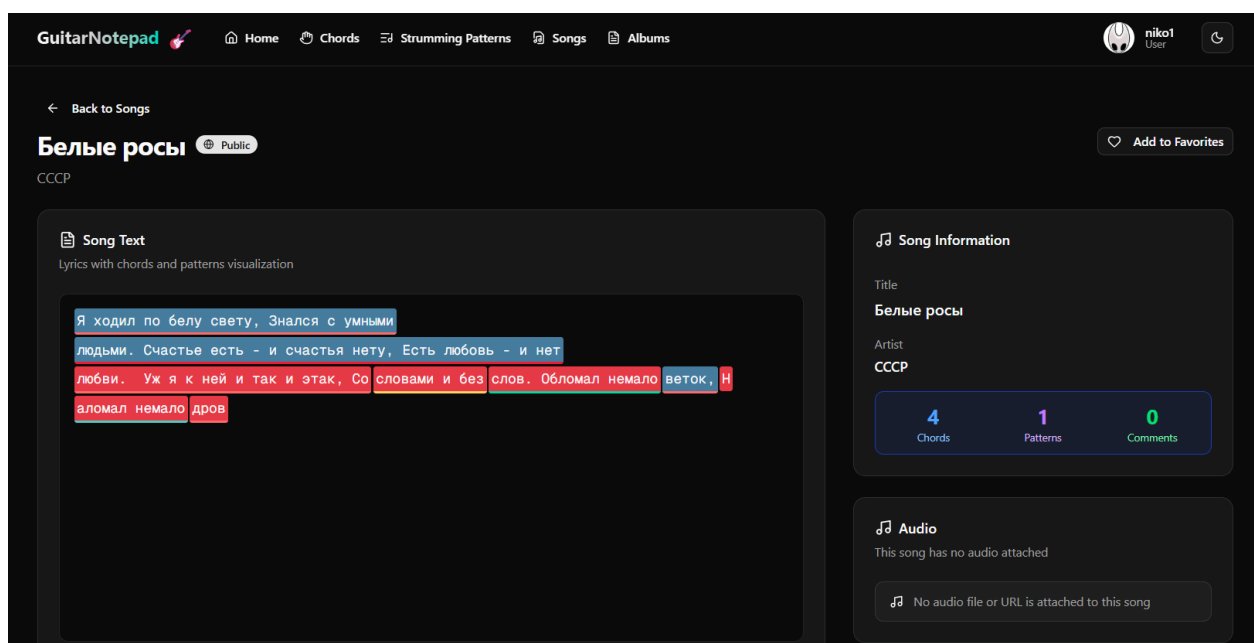


Рисунок 5.7 – Страница для просмотра песни

Чуть ниже этой страницы находятся комментарии, так же если эта песня была написана нами, то у нас есть возможность удалить или изменить ее.

5.2.5 Оставить комментарий

Оставлять комментарии можно только опубликованным песням и только чужим. Часть страницы `/home/songs/[songId]` с комментариями представлена на рисунке 5.8.

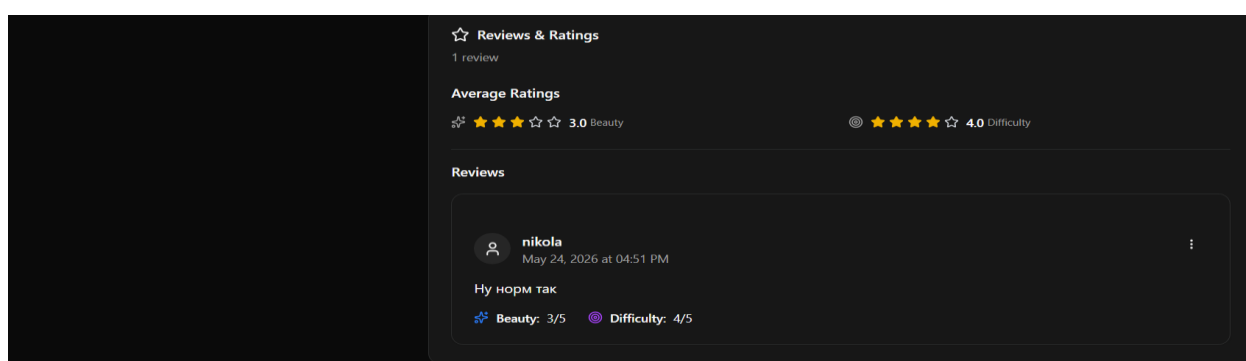


Рисунок 5.8 – Часть страницы с комментарием

В комментариях можно оставлять текстовые послания так и оценки красоты и сложности песни.

5.2.6 Просмотр альбомов

Помимо песен, так же есть и альбомы, где эти песни могут храниться, попасть на страницу просмотра альбомов можно через верхнее меню или через главную страницу. Страница с альбомами представлен на рисунке 5.9.

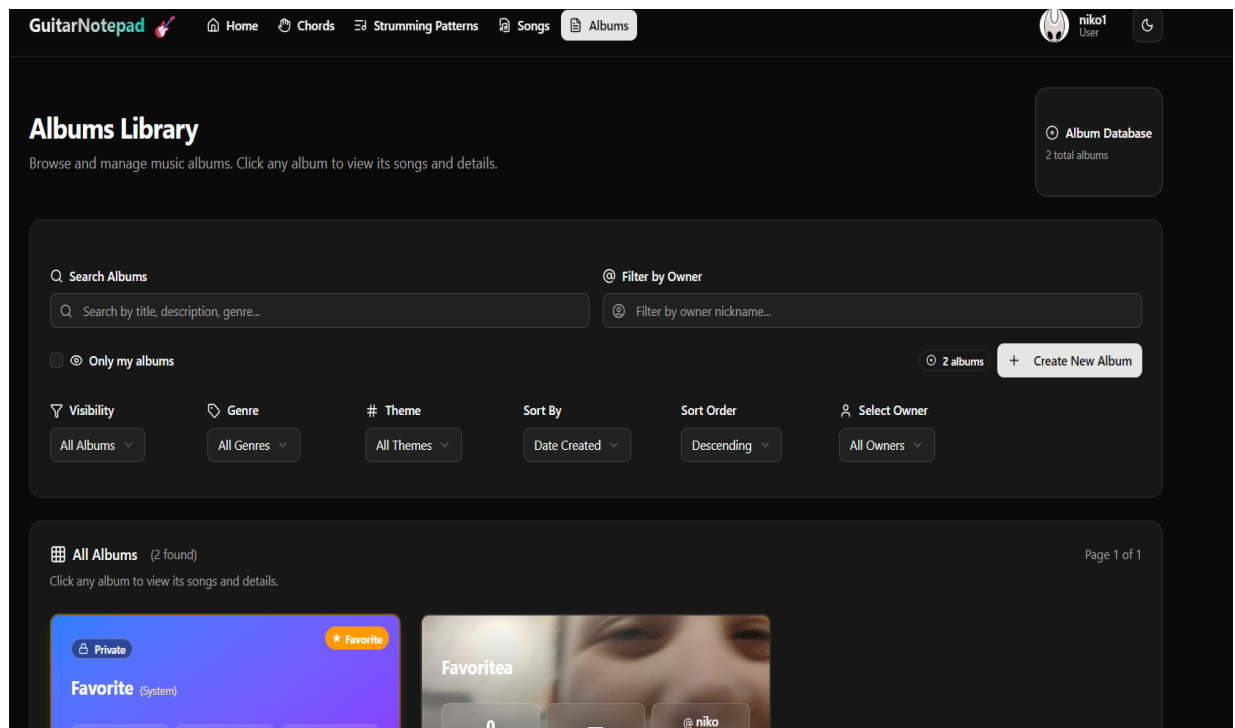


Рисунок 5.9 – Страница с альбомами

У каждого пользователя есть альбом Favorite который исключительно приватный и его нельзя удалить или редактировать.

5.2.7 Создание альбома

На ранее рассмотренной странице есть кнопка *Add new Album* которая ведет к странице для создания альбома. Страница для создания альбомов представлен на рисунке 5.10.

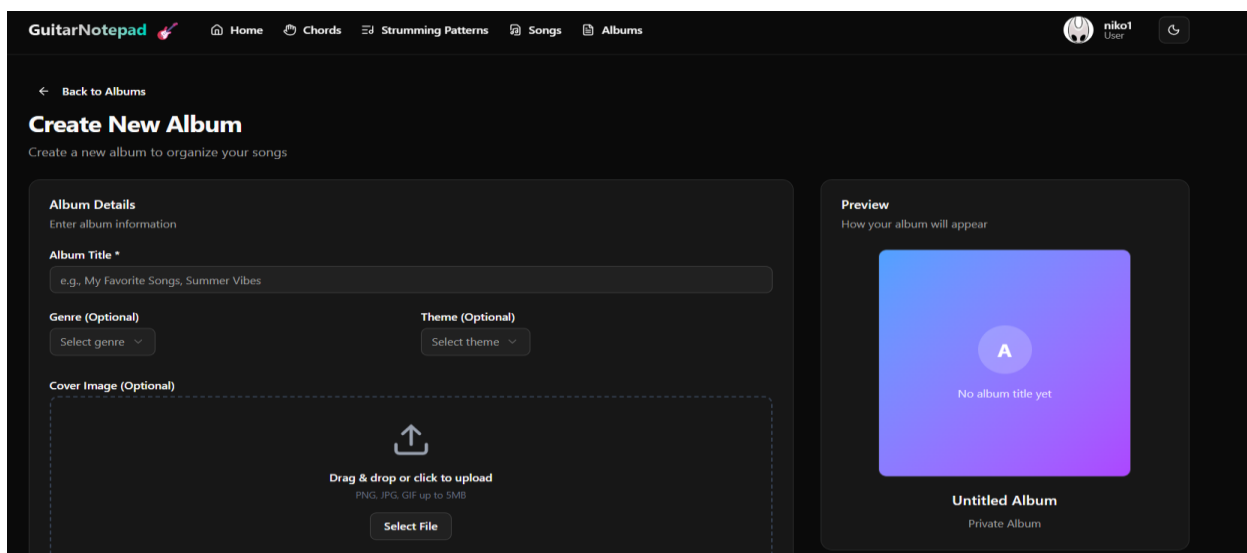


Рисунок 5.10 – Страница для создания альбома

На этой странице можно создавать свои собственные альбомы, в которые можно будет записывать песни.

5.3 Руководство администратора

5.3.1 Просмотр всей информации

Если от администратора перейти на ранее рассмотренную страницу `/home/songs`, то можно увидеть, что администратор видит все, в том числе и приватные песни и альбомы. Страница `/home/songs` от администратора представлена на рисунке 5.11.

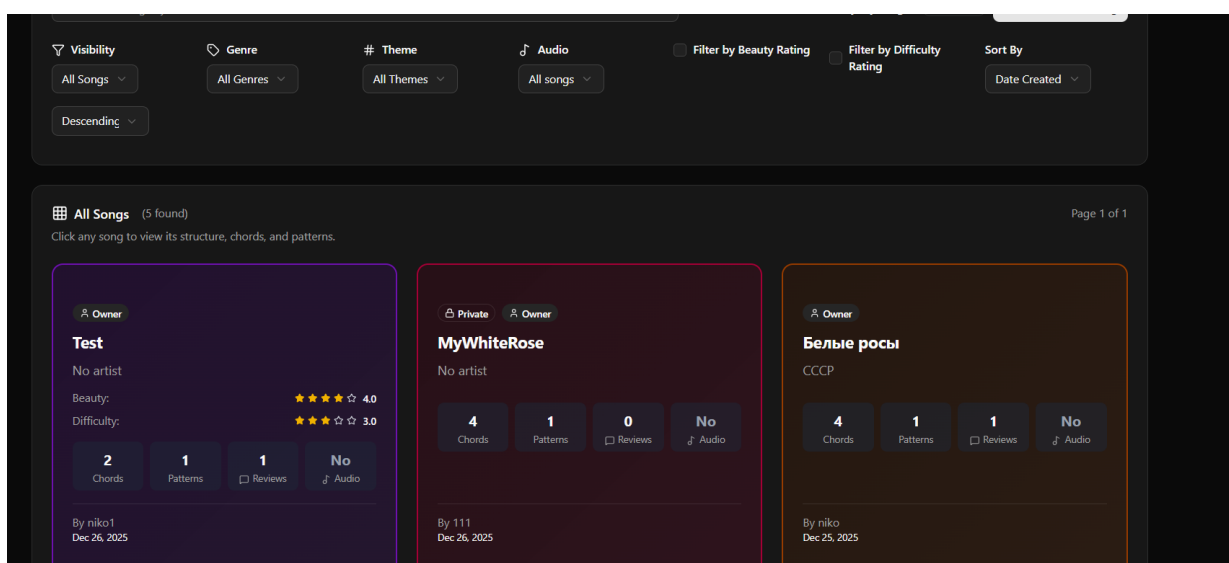


Рисунок 5.11 – Страница `/home/songs` от администратора

Так же администратор может просматривать даже приватные песни и альбомы и пусть он не может их никак менять, зато он может их удалять, в

случае если они что-либо нарушают. Страница просмотра чужой страницы от администратора представлена на рисунке 5.12.

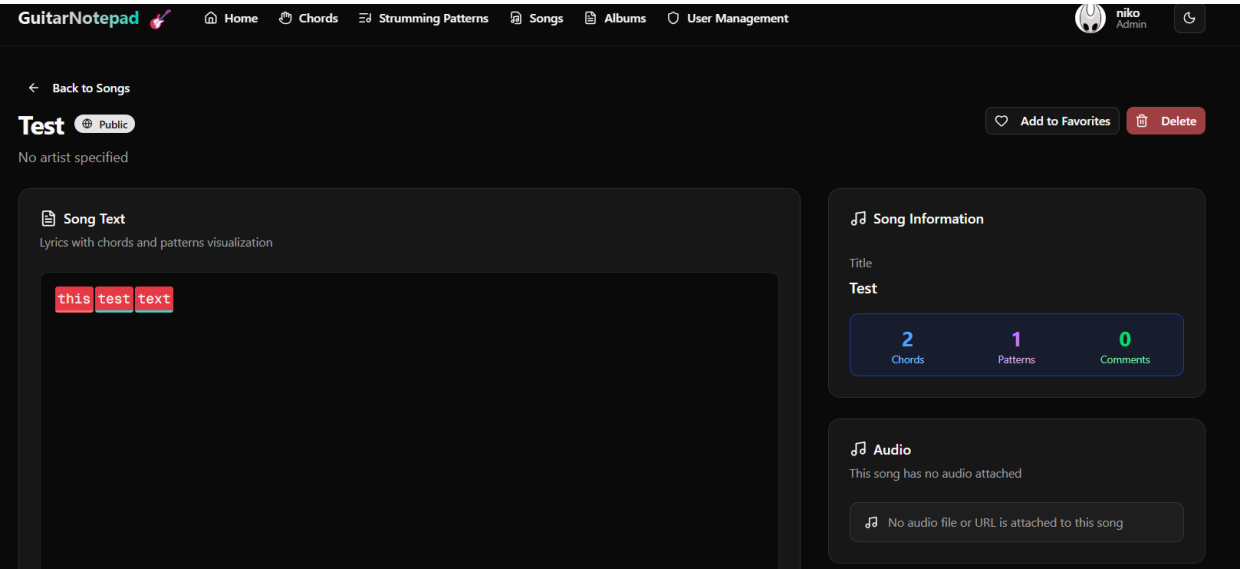


Рисунок 5.12 – Страница информации о песне от администратора

Так же мы можем увидеть, что администратор может удалять любой контент, но не может его изменять.

5.3.2 Управление пользователями

Так же администратор имеет власть над Гитаристами, в верхнем меню есть кнопка *UserManagment*, которая ведет на страницу управления Гитаристами. Страница управления Гитаристами представлена на рисунке 5.13.

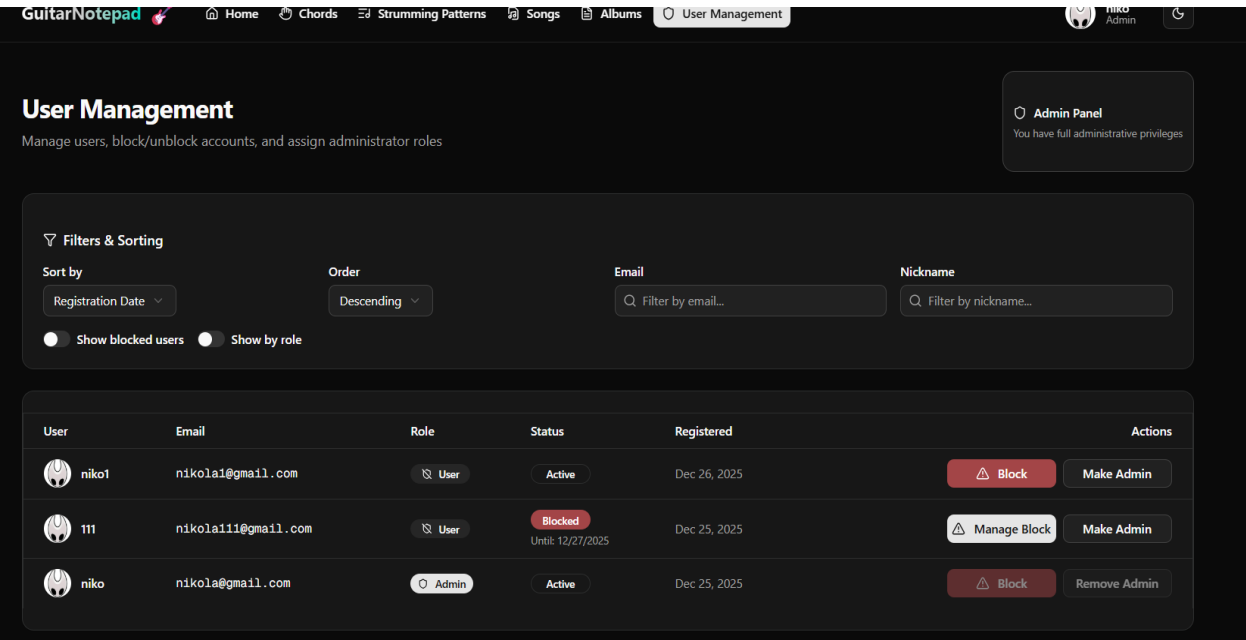


Рисунок 5.13 – Статистика управления Гитаристами

На этой странице можно искать определенных пользователей, выдавать или изымать права администратора, а также блокировать пользователей, диалоговое окно для выдачи прав администратора представлено на рисунке 5.14, диалоговое окно для блокировки пользователя представлено на рисунке 5.15.

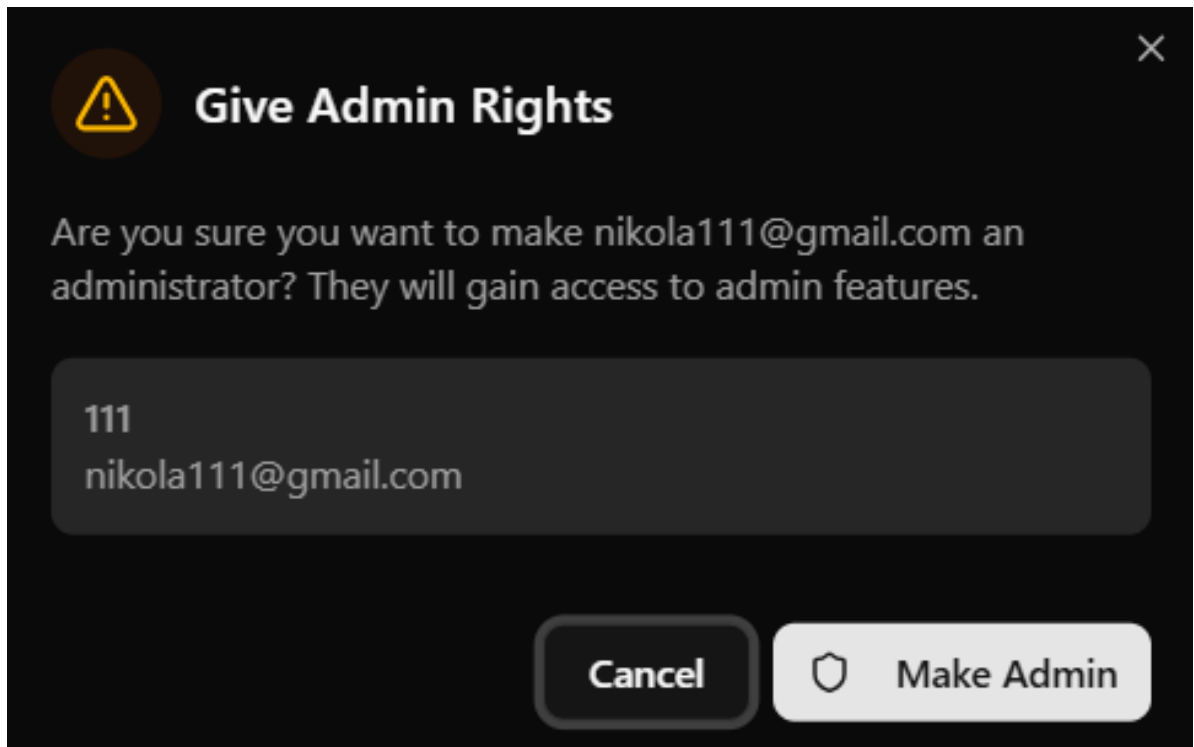


Рисунок 5.14 – Диалоговое окно для выдачи прав администратора

Можно как подтвердить блокировку, так и отменить ее.

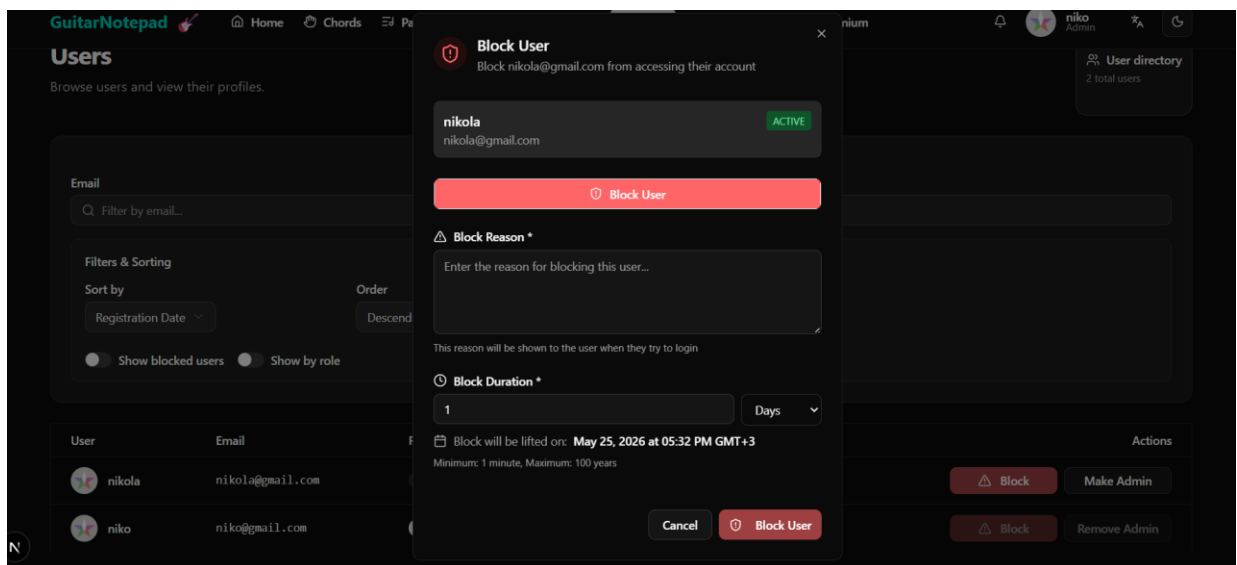


Рисунок 5.15 – Диалоговое окно для блокировки пользователя

Стоит обратить внимание, что нельзя заблокировать человека без причины и на вечно, поля причины и времени блокировки обязательны. Так же человека в любой момент можно разбанить.

5.4 Выводы по разделу

В данном руководстве подробно описаны действия, которые могут выполнять пользователи различных ролей в системе: гостя, гитариста и администратора. Каждая роль имеет свой уникальный набор функций и уровень доступа к взаимодействию с веб-приложением, что обеспечивает гибкость, безопасность и удобство использования платформы. Гость, будучи неавторизованным пользователем, имеет доступ только к просмотру публичного контента – песен, аккордов и паттернов, опубликованных другими пользователями. Однако его ключевой функцией является возможность зарегистрироваться в системе, что позволяет перейти к роли гитариста или администратора и получить доступ к расширенным возможностям платформы.

Гитарист, являясь основной рабочей ролью, может аутентифицироваться, создавать, просматривать, редактировать и удалять собственные песни, аккорды и паттерны, а также просматривать чужие опубликованные произведения. Ему доступно формирование персональных коллекций: добавление песен в избранное, объединение их в альбомы, написание отзывов с оценками и подписка на альбомы других пользователей для отслеживания обновлений. Обычные гитаристы имеют ограничения на количество создаваемого контента, в то время как премиум-пользователи получают неограниченные возможности и доступ к созданию альбомов.

Таким образом, представленная ролевая модель охватывает весь жизненный цикл взаимодействия пользователя с системой: от пассивного ознакомления с контентом до активного управления собственным творчеством и коллекциями. Подобная градация прав не только разграничивает ответственность между гостями, гитаристами и администратором, но и создает прозрачную экосистему, где каждый участник понимает границы своих возможностей. В конечном счете такой подход способствует здоровой коллаборации внутри сообщества, мотивирует обычных гитаристов на переход в премиум-статус и упрощает администратору контроль за наполнением платформы, сохраняя баланс между открытостью и безопасностью.

6 Технико-экономическое обоснование проекта

6.1 Общая характеристика разрабатываемого программного средства

Основная цель экономического раздела – обосновать экономическую целесообразность разработки и возможной последующей коммерциализации веб-приложения, созданного в рамках выпускной квалификационной работы. В разделе приводятся расчёты затрат по этапам жизненного цикла программного продукта, а также оцениваются ориентировочная рыночная стоимость решения и ожидаемый эффект от его реализации.

Разрабатываемое веб-приложение *Guitar Notepad* предназначено для работы с гитарным материалом в единой среде: составление и просмотр песен с табличным представлением, привязка аккордов и паттернов исполнения, работа с пользовательскими коллекциями аккордов и паттернов, формирование альбомов, прикрепление и воспроизведение аудио, отзывы и оценки, уведомления, профиль пользователя и роли с административными функциями.

При разработке использовались технологии, ориентированные на промышленное применение и сопровождение. Серверная часть реализована на *ASP.NET Core* с разделением на слои домена, приложения, инфраструктуры и веб-API, *Entity Framework Core* и *PostgreSQL* для хранения данных, *JWT* для аутентификации, *Swagger* для описания и проверки API. Клиентская часть построена на *Next.js 16* и *React 19* с *TypeScript*, стилизация – *Tailwind CSS*; обмен с API организован через *HTTP*-клиент и сервисный слой, состояние – с помощью контекста *React* и локальных механизмов управления состоянием в границах отдельных сценариев. Для воспроизведения и обработки звука в браузере задействованы клиентские медиа-библиотеки. В корне репозитория предусмотрен *Docker Compose* для упрощения развёртывания и согласованной работы компонентов инфраструктуры. Архитектура рассчитана на расширяемость предметной области, разграничение прав и интеграцию с внешними системами через *REST API*.

6.2 Исходные данные для проведения расчетов для продажи

В качестве исходных данных для выполнения дальнейших расчётов используются действующие нормативно-правовые акты и положения законодательства, регулирующие сферу разработки и реализации программного обеспечения. Все необходимые численные значения и нормативы, применяемые при расчёте стоимости программного продукта, приведены в таблице 6.1.

Указанные данные являются основой для определения полной себестоимости и последующей оценки экономической эффективности проекта.

					ДП 06.00.ПЗ				
		ФИО	Подпись	Дата					
Разраб.		Селицкий Н.Е.			6 Технико-экономическое обоснование проекта		Лит.	Лист	Листов
Пров.		Харланович А.В.					У	1	10
Консульт.		Познякова Л.С.					БГТУ 1-40 01 01, 2026		
Н. контр.		Авдеева В.Д.							
Утв.		Смелов В.В.							

Таблица 6.1 – Исходный данные для расчетов

Наименование показателя	Условные обозначения	Норматив
Норматив дополнительной заработной платы, %	$H_{дз}$	12
Ставка отчислений в Фонд социальной защиты населения, %	$H_{фсзн}$	34
Ставка отчислений в БРУСП «Белгосстрах», %	$H_{бгс}$	0,6
Норматив прочих прямых затрат, %	$H_{пз}$	25
Норматив расходов на сопровождение и адаптацию, %	$H_{рса}$	10
Норматив накладных расходов, %	$H_{опб,опх}$	44
Норматив дополнительной заработной платы, %	$H_{дз}$	12

Данные таблицы 6.1 понадобятся для расчета затрат и эффективности разрабатываемого веб-приложения.

Для определения цены разрабатываемого приложения был проведен маркетинговый анализ стоимости разработки аналогичного приложения различными компаниями на основании данных сайтов-агрегаторов.

Первым источником послужил сайт <https://thebestapp.ru/calculator>. Результаты анализа стоимости представлены на рисунке 6.1.

The screenshot shows the 'TheBestApp' website's calculator interface. At the top, there are navigation links: 'О нас', 'Проекты', 'Калькулятор' (highlighted), and 'Контакты'. Below the navigation bar are three large buttons for platform selection: 'Android' (grey), 'iOS' (grey), and 'WEB' (blue and highlighted). Underneath these is the heading 'Что Вы хотите получить?' followed by a paragraph explaining the service and a link 'Показать полностью...'. Below this are four service options: 'Приложение с нуля' (blue and highlighted), 'Обновление для приложения' (grey), 'Дизайн и техническое задание' (grey), and 'Доработка проекта' (grey). At the bottom, a large blue box displays the calculated cost: 'Стоимость разработки вашего приложения 1 835 000 ₽'. To the right of this box is a blue button labeled 'ПОЛУЧИТЬ РАСЧЕТ СТОИМОСТИ'.

Рисунок 6.1 – Расчет стоимости сайта *thebestapp.ru*

Как видно из рисунка, стоимость разработки на основании данного сайта составит 1 835 000,00 российских рублей, что составляет 69 401.54 белорусских рублей.

Вторым ресурсом является <https://wecode.ua/ru/kalkulyator-stoimosti-razrabotki-mobilnogo-prilozheniya/>, результаты анализа которого представлены на рисунке 6.2. Как видно из рисунка, стоимость разработки приложения-аналога составит 10 116.00 дол. США, что эквивалентно 28 512.95 белорусских рублей (по курсу Национального банка Республики Беларусь по состоянию на 01.05.26).



Рисунок 6.2 – Расчет стоимости сайта *wecode.ua*

Третьим проанализирован был сайт <https://elapp.ru/stoimost/onlayn-raschet-stoimosti>, результаты поиска которого представлены на рисунке 6.3.

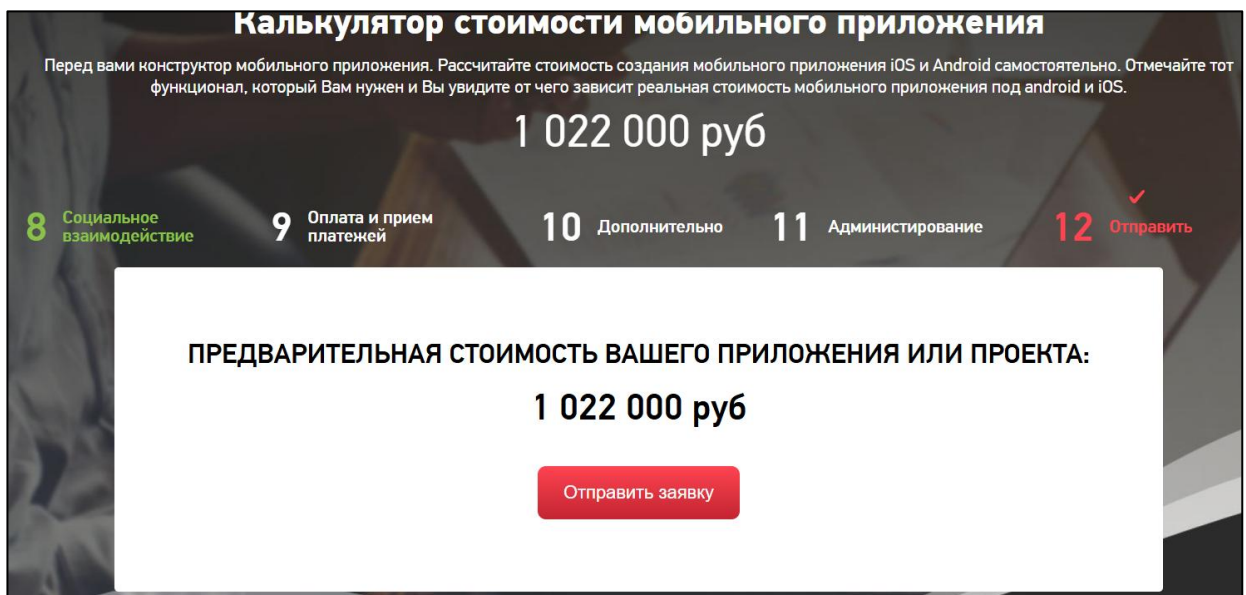


Рисунок 6.3 – Расчет стоимости сайта *elapp.ru*

Как видно из рисунка, стоимость разработки составит 1 022 000,00 российских рублей, что соответствует 37 896.64 (по курсу Национального банка Республики Беларусь на 01.05.26).

Сводная информация о проведенном анализе в таблице 6.2.

Таблица 6.2 – Результаты маркетингового анализа

Источник	Стоимость с НДС, руб.
https://thebestapp.ru/calculator	69 401.54
https://wecode.ua/ru/kalkulyator-stoimosti-razrabotki-mobilnogo-prilozheniya/	28 512.95
https://elapp.ru/stoimost/onlayn-raschet-stoimosti	37 896.64

Средняя цена разработки подобного приложения составляет 45 270.37 руб.

6.3 Обоснование цены программного средства

6.3.1 Расчет затрат рабочего времени на разработку программного средства

В таблице 6.3 указаны в укрупненном виде все работы, выполненные для создания, указанного в дипломной работе программного средства и трудозатраты в человеко-часах, затрачиваемые на выполнение этих работ в организации.

Таблица 6.3 – Затраты рабочего времени на разработку ПС

Содержание работ	Исполнитель	Трудозатраты, чел.-час.
Составление календарного плана, постановка целей и задач, контроль их выполнения, согласование требований	проджект-менеджер	40
Создание и проработка технического задания	бизнес-аналитик	18
Создание дизайна веб-приложения	ux/ui дизайнер	100
Разработка клиентской части приложения	frontend-разработчик	140
Разработка серверной части приложения	backend-разработчик	160
Тестирование приложения	тестировщик	80
Всего		538

Как видно из таблицы, при разработке программного средства проджект-менеджер привлекался на 40 чел.-часов, бизнес-аналитик – на 18 чел.-часа, ux/ui

дизайнер – на 100 чел.-часов, *backend*-разработчик – на 160 чел.-часов, *frontend*-разработчик – на 140 чел.-часов, тестировщик – на 80 чел.-часов. Всего программное средство разрабатывалось на протяжении 538 чел.-часов.

6.3.2 Расчет основной заработной платы

Для определения величины основной заработной платы, было проведено исследование величин заработных плат для специалистов уровня Junior на позиции бизнес-аналитика, проджект-менеджера, *ux/ui* дизайнера, *backend*-разработчика, *frontend*-разработчика, тестировщика.

Данные исследования, расчет основной заработной платы и времени разработки приведены в таблице 6.4.

При расчете основной заработной платы важным показателем является часовая ставка специалиста, выполняющего конкретный вид работ.

Основная заработная плата будет рассчитываться по формуле 6.1

$$C_{оzi} = T_{рази} \cdot C_{зпi} \quad (6.1)$$

где $C_{оzi}$ – основная заработная плата, руб.;

$T_{рази}$ – время разработки, чел.-часов;

$C_{зпi}$ – средняя часовая заработная плата, руб.

Таблица 6.4 – Расчет основной заработной платы

Вид работ	Месячная заработная плата	Часовая тарифная ставка, руб.	Трудоза-траты, чел-часов	Основная за-работная плата, руб.
Backend-разработчик	3 200.00	19.05	160	3 048.00
Frontend-разработчик	2 950.00	17.56	140	2 458.40
ux/ui дизайнер	2 650.00	15.77	100	1 577.00
Бизнес-аналитик	2 850.00	16.96	18	305.28
Тестировщик	2 400.00	14.29	80	1 143.20
Проджект-менеджер	3 100.00	18.45	40	738.00
Всего			538	9 269.88

Основная заработная плата составит 9 269.88 руб.

В дальнейшем для других расчетов используется основная заработная плата, рассчитанная по указанной выше методике.

6.3.3 Расчет дополнительной заработной платы

На основании законодательства о труде и положениями об оплате труда в организации предусматривается наличие выплат (оплата отпусков, льготных часов, времени выполнения государственных обязанностей и других выплат, не связанных с основной деятельностью исполнителей), которые определяются на основании статистических данных по нормативу в процентах к основной заработной плате по формуле 6.2.

$$C_{\text{д.з.}} = \frac{C_{\text{о.з.}} \cdot N_{\text{д.з.}}}{100} \quad (6.2)$$

где $C_{\text{оз}}$ – основная заработная плата, руб.;

$N_{\text{дз}}$ – норматив дополнительной заработной платы, %.

$$C_{\text{дз}} = 9\,269.88 \cdot 12 / 100 = 1\,112.39 \text{ руб.}$$

Таким образом дополнительная заработная плата составила 1 112.39 руб.

6.3.4 Расчет отчислений в Фонд социальной защиты населения и по обязательному страхованию

Отчисления в Фонд социальной защиты населения (ФСЗН) и по обязательному страхованию от несчастных случаев на производстве и профессиональных заболеваний в БРУСП «Белгосстрах» определяются в соответствии с действующими законодательными актами по нормативу в процентном отношении к фонду основной и дополнительной зарплаты исполнителей.

Отчисления в Фонд социальной защиты населения (ФСЗН) вычисляются по формуле 6.3

$$C_{\text{фсзн}} = \frac{(C_{\text{оз}} + C_{\text{дз}}) \cdot N_{\text{фсзн}}}{100}, \quad (6.3)$$

где $C_{\text{оз}}$ – основная заработная плата, руб.;

$C_{\text{дз}}$ – дополнительная заработная плата на конкретное ПС, руб.;

$N_{\text{фсзн}}$ – норматив отчислений в Фонд социальной защиты, %.

Отчисления в БРУСП «Белгосстрах» вычисляются по формуле 6.4

$$C_{\text{бгс}} = \frac{(C_{\text{оз}} + C_{\text{дз}}) \cdot H_{\text{бгс}}}{100}, \quad (6.4)$$

где $H_{\text{бгс}}$ – норматив отчислений в БРУСП «Белгосстрах», %.

$$C_{\text{фсзн}} = (9\,269.88 + 1\,112.39) \cdot 34 / 100 = 3\,529.97 \text{ руб.}$$

$$C_{\text{бгс}} = (9\,269.88 + 1\,112.39) \cdot 0.6 / 100 = 62.3 \text{ руб.}$$

Таким образом, общие отчисления в БРУСП «Белгосстрах» составили 62.3 руб., а в фонд социальной защиты населения – 3 529.97 руб.

6.3.5 Расчет суммы прочих прямых затрат

Сумма прочих затрат $C_{\text{пз}}$ определяется как произведение основной заработной платы исполнителей на конкретное программное средство $C_{\text{оз}}$ на норматив прочих прямых затрат в целом по организации $H_{\text{пз}}$ (формула 6.5).

К прочим прямым затратам можно будет отнести расходы на аренду облачного сервера для развертывания серверной части приложения, пересылку оригиналов договоров и актов заказчику курьерской службой.

$$C_{\text{пз}} = \frac{C_{\text{оз}} \cdot H_{\text{пз}}}{100} \quad (6.5)$$

$$C_{\text{пз}} = 9\,269.88 \cdot 25 / 100 = 2\,317.47 \text{ руб.}$$

Сумма прочих прямых затрат составит 2 317.47 руб.

6.3.6 Расчет суммы накладных расходов

Сумма накладных расходов $C_{\text{обп,опх}}$ определяется как произведение основной заработной платы исполнителей на конкретное программное средство $C_{\text{оз}}$ на норматив накладных расходов в целом по организации $H_{\text{обп,опх}}$ и рассчитывается по формуле 6.6

$$C_{\text{обп,опх}} = \frac{C_{\text{оз}} \cdot H_{\text{обп,опх}}}{100}, \quad (6.6)$$

$$C_{\text{обп,опх}} = 9\,269.88 \cdot 44 / 100 = 4\,078.75 \text{ руб.}$$

Размер накладных расходов составит 4 078.75 руб.

6.3.7 Сумма расходов на разработку программного средства

Сумма расходов на разработку программного средства C_p определяется как сумма основной и дополнительной заработной платы исполнителей на конкретное программное средство, отчислений на социальные нужды, суммы прочих затрат и суммы накладных расходов (формула 6.7)

$$C_p = C_{оз} + C_{дз} + C_{фсзн} + C_{бгс} + C_{пз} + C_{обп,обх} \quad (6.7)$$

Все данные необходимые для вычисления есть, поэтому можно определить сумму расходов на разработку программного средства:

$$C_p = 9\,269.88 + 1\,112.39 + 3\,529.97 + 62.3 + 2\,317.47 + 4\,078.75 = 20\,370.75 \text{ руб.}$$

Общая сумма расходов на разработку средства составит 20 370.75 руб.

6.3.8 Расходы на сопровождение и адаптацию

Сумма расходов на сопровождение и адаптацию программного средства $C_{рса}$ определяется как произведение суммы расходов на разработку на расходы на сопровождение и адаптацию $H_{рса}$.

Таким образом сумма расходов на сопровождение и адаптацию программного средства вычисляется по формуле 6.8

$$C_{рса} = \frac{C_p \cdot H_{рса}}{100} \quad (6.8)$$

Все данные необходимые для вычисления есть, поэтому можно определить сумму расходов на сопровождение и адаптацию программного средства.

$$C_{рса} = 20\,370.747 \cdot 10 / 100 = 2\,037.08 \text{ руб.}$$

Сумма расходов на сопровождение и адаптацию была вычислена на основе данных, рассчитанных ранее в данном разделе, и составила 2 037.08 руб.

Все проведенные выше расчеты необходимы для вычисления полной себестоимости проекта.

6.3.9 Расчет полной себестоимости

Общая сумма расходов на разработку с затратами на сопровождение и адаптацию программного средства $C_{п}$ определяется как сумма двух элементов: суммы расходов на разработку C_p и суммы расходов на сопровождение и адаптацию программного средства $C_{рса}$ (формула 6.9).

$$C_{\pi} = C_p + C_{\text{рса}}. \quad (6.9)$$

$$C_{\pi} = 20\,370.747 + 2\,037.075 = 22\,407.82 \text{ руб.}$$

Размер полной себестоимости, исходя из вышеприведенных расчетов, составит 22 407.822 руб.

6.3.10 Определение цены, оценка эффективности

Для определения эффективности реализации разработанного программного средства выше был произведен маркетинговый анализ, на основе которого была определена стоимость продукта.

Так как полная себестоимость разработки и поддержки C_{π} составляет 22 407.82 руб., есть возможность установить цену, соответствующую средней – 37 896.64 руб.

Цена программного средства без НДС рассчитывается по формуле 6.10

$$C_{\text{безНДС}} = \frac{C_p}{(H_{\text{НДС}} + 100)/100} \quad (6.10)$$

$$C_{\text{безНДС}} = 37\,896.64 \cdot (100 / 120) = 31\,580.53 \text{ руб.}$$

Прибыль от реализации рассчитывается по формуле 6.11

$$\Pi_{\text{пс}} = \Pi_p - C_{\pi} \quad (6.11)$$

$$\Pi_{\text{пс}} = 31\,580.53 - 22\,407.82 = 9\,172.71 \text{ руб.}$$

Рентабельность разрабатываемого продукта:

$$P_{\text{пс}} = (9\,172.71 / 22\,407.82) \cdot 100 = 40.94 \, \%.$$

Чистая прибыль рассчитывается по формуле 6.12

$$\Pi_q = \Pi_{\text{пс}} \cdot \left(1 - \frac{H_{\pi}}{100}\right) \quad (6.12)$$

$$\Pi_q = 9\,172.71 \cdot (1 - 20/100) = 7\,338.17 \text{ руб.}$$

На основании расчетов видно, что проект является экономически целесообразным, так как основные показатели эффективности выше нуля.

6.4 Выводы по разделу

В таблице 6.5 приведены результаты расчетов экономических показателей, проведенных в данном разделе.

Таблица 6.5 – Результаты расчетов

Наименование показателя	Значение показателя
Время разработки, чел.-часов	538
Основная заработная плата, руб.	9 269.88
Дополнительная заработная плата, руб.	1 112.39
Отчисления в Фонд социальной защиты населения, руб.	3 529.97
Отчисления в Белгосстрах	62.29
Прочие прямые затраты, руб.	2 317.47
Накладные расходы, руб.	4 078.75
Себестоимость разработки программного средства, руб.	20 370.75
Расходы на сопровождение и адаптацию, руб.	2 037.08
Полная себестоимость, руб.	22 407.82
Рентабельность разрабатываемого продукта, %	40.94
Чистая прибыль, руб.	7 338.17
Цена программного средства без НДС, руб	31 580.53
Прибыль от реализации, руб	9 172.71

К программному средству предъявлены требования по удобству пользования и полноте функционала, что должно обеспечить качественную работу. Разработанное программное средство предоставляет необходимый для качественного оказания услуг специалистов функционал.

Таким образом, проведенные в разделе расчеты подтверждают экономическую целесообразность разработки и внедрения программного средства. Полученные значения рентабельности (40,94%) и чистой прибыли (7 338,17 руб.) свидетельствуют о достаточной эффективности проекта с коммерческой точки зрения. Установленная цена без НДС в размере 31 580,53 руб. является обоснованной с учетом всех произведенных затрат и планируемого уровня доходности. В совокупности с заложенными требованиями к удобству и функциональности, представленные экономические показатели позволяют сделать вывод о том, что разрабатываемое программное средство является не только технически полноценным, но и финансово состоятельным продуктом, готовым к внедрению в практическую деятельность.

Заключение

В ходе выполнения дипломного проекта было разработано веб-приложение «*Guitar Notepad*» – платформа для создания, хранения и публикации гитарных партий, аккордов, паттернов боя и альбомов. Проект реализовал поставленные задачи: анализ аналогов, проектирование архитектуры, разработку клиентской и серверной частей, тестирование, документирование и технико-экономическое обоснование монетизации через подписки и *Premium*, а также подготовку руководства пользователя.

Анализ существующих сервисов для гитаристов выявил ключевые ожидания пользователей: поиск и фильтрация контента, работа с аккордами и ритмом, личные коллекции и платные тарифы. Эти требования были учтены при разработке.

Архитектура построена на многослойной серверной части и клиенте на Next.js, React и TypeScript, что обеспечивает производительность, сопровождаемость и масштабируемость.

Серверная логика реализована по CQRS с JWT-аутентификацией; клиент использует централизованный API-слой, контекст локализацию RU/EN.

Функционал включает регистрацию и вход, каталоги песен, аккордов и паттернов, редактор песен с сегментами, альбомы, избранное, отзывы, подписки на альбомы, покупку *Premium* и администрирование (роли, блокировки, доступ к любому контенту). Для бесплатных пользователей действуют квоты на создание объектов.

Тестирование подтвердило стабильность основных сценариев: функциональные проверки UI, модульные и API-тесты, что свидетельствует о готовности системы к промышленной эксплуатации. Руководство пользователя обеспечивает удобную работу обычным пользователям и администраторам.

Технико-экономическое обоснование показало целесообразность дохода от продажи и определило направления развития: социальные функции, улучшение поиска и оптимизацию отклика при больших каталогах.

Таким образом, «*Guitar Notepad*» достиг поставленных целей и готов к использованию, предлагая гитаристам удобную среду для работы с партитурами и обмена музыкальными материалами.

					ДП 00.00.ПЗ			
		ФИО	Подпись	Дата	Заключение			
Разраб.		Селицкий Н.Е.						
Пров.		Харланович А.В.						
Н. контр.		Авдеева В.Д.						
Утв.		Смелов В.В.			БГТУ 1-40 01 01, 2025			
					Лит.	Лист	Листов	
					У	1	1	

Список используемых источников

1. ASP.NET Core Official Documentation. [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/aspnet/core> – Дата обращения: 25.05.2026.
2. Next.js Official Documentation. [Электронный ресурс] – Режим доступа: <https://nextjs.org/docs> – Дата обращения: 25.05.2026.
3. Ultimate Guitar Website. [Электронный ресурс] – Режим доступа: <https://www.ultimate-guitar.com/> – Дата обращения: 25.05.2026.
4. Songsterr Website. [Электронный ресурс] – Режим доступа: <https://www.songsterr.com/> – Дата обращения: 25.05.2026.
5. Node.js Official Documentation. [Электронный ресурс] – Режим доступа: <https://nodejs.org/en/docs/> – Дата обращения: 25.05.2026.
6. TypeScript Official Documentation. [Электронный ресурс] – Режим доступа: <https://www.typescriptlang.org/docs/> – Дата обращения: 25.05.2026.
7. PostgreSQL Official Documentation. [Электронный ресурс] – Режим доступа: <https://www.postgresql.org/docs/> – Дата обращения: 25.05.2026.
8. Docker Official Documentation. [Электронный ресурс] – Режим доступа: <https://docs.docker.com/> – Дата обращения: 25.05.2026.
9. Docker Compose Official Documentation. [Электронный ресурс] – Режим доступа: <https://docs.docker.com/compose/> – Дата обращения: 25.05.2026.
10. Swagger OpenAPI Documentation. [Электронный ресурс] – Режим доступа: <https://swagger.io/docs/> – Дата обращения: 25.05.2026.
11. REST API Tutorial by RESTfulAPI.net. [Электронный ресурс] – Режим доступа: <https://restfulapi.net/> – Дата обращения: 25.05.2026.
12. Entity Framework Core Documentation. [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/ef/core/> – Дата обращения: 25.05.2026.
13. ORM (Object-Relational Mapping) – Microsoft Learn. [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/dotnet/architecture/cloud-native/relational-vs-nosql-data> – Дата обращения: 25.05.2026.
14. Axios GitHub Repository & Documentation. [Электронный ресурс] – Режим доступа: <https://github.com/axios/axios> – Дата обращения: 25.05.2026.
15. React Official Documentation. [Электронный ресурс] – Режим доступа: <https://react.dev/> – Дата обращения: 25.05.2026.
16. Express.js Official Documentation. [Электронный ресурс] – Режим доступа: <https://expressjs.com/> – Дата обращения: 25.05.2026.

					ДП 00.00.ПЗ			
		ФИО	Подпись	Дата	Список используемых источников			
Разраб.		Селицкий Н.Е.						
Пров.		Харланович А.В.						
Н. контр.		Авдеева В.Д.						
Утв.		Смелов В.В.			БГТУ 1-40 01 01, 2025			
					Лит. Лист Листов			
					У 1 2			

17. Onion Architecture – Jeffrey Palermo (Architecture Guide). [Электронный ресурс] – Режим доступа: <https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/> – Дата обращения: 25.05.2026.

18. Dependency Injection in .NET – Microsoft Learn. [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/dotnet/core/extensions/dependency-injection> – Дата обращения: 25.05.2026.

19. Repository Pattern – Microsoft Learn. [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design> – Дата обращения: 25.05.2026.

20. Unit of Work Pattern – Martin Fowler (Catalog of Patterns). [Электронный ресурс] – Режим доступа: <https://martinfowler.com/eaCatalog/unitOfWork.html> – Дата обращения: 25.05.2026.

21. Middleware in ASP.NET Core – Microsoft Learn. [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/aspnet/core/fundamentals/middleware> – Дата обращения: 25.05.2026.

22. Singleton Pattern – Refactoring Guru. [Электронный ресурс] – Режим доступа: <https://refactoring.guru/design-patterns/singleton> – Дата обращения: 25.05.2026.

23. Builder Pattern – Refactoring Guru. [Электронный ресурс] – Режим доступа: <https://refactoring.guru/design-patterns/builder> – Дата обращения: 25.05.2026.

24. Mediator Pattern – Refactoring Guru. [Электронный ресурс] – Режим доступа: <https://refactoring.guru/design-patterns/mediator> – Дата обращения: 25.05.2026.

25. CQRS Pattern – Microsoft Learn. [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/apply-simplified-microservice-cqrs-ddd-patterns> – Дата обращения: 25.05.2026.

Диаграмма вариантов использования ДП 01.00 ГЧ

Логическая схема базы данных ДП 02.00 ГЧ

Диаграмма развертывания ДП 03.00 ГЧ

Диаграмма классов ДП 04.00 ГЧ

Блок схема функции отправки сообщения ДП 05.00 ГЧ

Скриншот-работа ДП 06.00 ГЧ

ПРИЛОЖЕНИЕ А

```

using Microsoft.EntityFrameworkCore;
using Domain.Entities;
using System.Reflection;
namespace Infrastructure.Data;
public class AppDbContext : DbContext{
    public AppDbContext() { }
    public AppDbContext(DbContextOptions<AppDbContext> options)
: base(options){}
    public DbSet<User> Users { get; set; }
    public DbSet<Song> Songs { get; set; }
    public DbSet<Chord> Chords { get; set; }
    public DbSet<StrummingPattern> StrummingPatterns {get;set;}
    public DbSet<SongReview> SongReviews { get; set; }
    public DbSet<SongSegment> SongSegments { get; set; }
    public DbSet<SongSegmentPosition>
SongSegmentPositions{get;set;}
    public DbSet<SongStructure> SongStructures { get; set; }
    public DbSet<SongPattern> SongPatterns { get; set; }
    public DbSet<SongComment> SongComments { get; set; }
    public DbSet<SongChord> SongChords { get; set; }
    public DbSet<Album> Albums { get; set; }
    public DbSet<SongAlbum> SongAlbums { get; set; }
    public DbSet<Subscription> Subscriptions { get; set; }
    public DbSet<Notification> Notifications { get; set; }
    protected override void OnModelCreating(ModelBuilder
modelBuilder){
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<User>(entity =>{
            entity.HasKey(e => e.Id);
            entity.Property(e =>
e.Email).IsRequired().HasMaxLength(256);
            entity.Property(e =>
e.NikName).IsRequired().HasMaxLength(100);
            entity.Property(e =>
e.PasswordHash).IsRequired().HasMaxLength(500);
            entity.Property(e =>
e.Role).IsRequired().HasMaxLength(50);
            entity.Property(e =>
e.AvatarUrl).HasMaxLength(10000);
            entity.Property(e => e.Bio).HasMaxLength(1000);
            entity.Property(e =>
e.BlockReason).HasMaxLength(500);
            entity.Property(e => e.HasPremium).IsRequired();
            entity.Property(e => e.CreateAt).IsRequired();
            entity.HasIndex(e => e.Email).IsUnique();
            entity.HasIndex(e => e.NikName).IsUnique();
            entity.HasMany(e => e.Songs).WithOne(e =>
e.Owner).HasForeignKey(e =>
e.OwnerId).OnDelete(DeleteBehavior.Restrict);

```

```

        entity.HasMany(e => e.Chords).WithOne(e =>
e.CreatedBy).HasForeignKey(e =>
e.CreatedByUserId).OnDelete(DeleteBehavior.Restrict);
        entity.HasMany(e => e.StrummingPatterns).WithOne(e
=> e.CreatedBy).HasForeignKey(e =>
e.CreatedByUserId).OnDelete(DeleteBehavior.Restrict);
        entity.HasMany(e => e.Reviews).WithOne(e =>
e.User).HasForeignKey(e =>
e.UserId).OnDelete(DeleteBehavior.Restrict);
        entity.HasMany(e => e.Comments).WithOne(e =>
e.User).HasForeignKey(e =>
e.UserId).OnDelete(DeleteBehavior.Restrict);
        entity.HasMany(e => e.Subscriptions).WithOne(e =>
e.Subscriber).HasForeignKey(e =>
e.UserId).OnDelete(DeleteBehavior.Restrict));});
        modelBuilder.Entity<Subscription>(entity =>{
            entity.HasKey(e => e.Id);
            entity.Property(e => e.UserId).IsRequired();
            entity.Property(e =>
e.TargetAlbumId).IsRequired(false);
            entity.Property(e => e.CreatedAt).IsRequired();
            entity.HasOne(e => e.Subscriber).WithMany(e =>
e.Subscriptions).HasForeignKey(e =>
e.UserId).OnDelete(DeleteBehavior.Restrict);
            entity.HasOne(e => e.TargetAlbum).WithMany(e =>
e.Subscribers).HasForeignKey(e =>
e.TargetAlbumId).OnDelete(DeleteBehavior.Restrict);
            entity.HasIndex(e => e.UserId);
            entity.HasIndex(e => e.TargetAlbumId);
            entity.HasIndex(e => new { e.UserId,
e.TargetAlbumId }).IsUnique().HasFilter("\"TargetAlbumId\" IS
NOT NULL");
            entity.HasIndex(e => e.CreatedAt);});
        modelBuilder.Entity<Song>(entity =>{
            entity.HasKey(e => e.Id);
            entity.Property(e =>
e.Title).IsRequired().HasMaxLength(200);
            entity.Property(e => e.Artist).HasMaxLength(200);
            entity.Property(e =>
e.Description).HasMaxLength(2000);
            entity.Property(e =>
e.Genre).IsRequired().HasMaxLength(100);
            entity.Property(e =>
e.Theme).IsRequired().HasMaxLength(100);
            entity.Property(e =>
e.CustomAudioUrl).HasMaxLength(10000);
            entity.Property(e =>
e.CustomAudioType).HasMaxLength(50);
            entity.Property(e => e.FullText).IsRequired();
            entity.Property(e => e.CreatedAt).IsRequired();
            entity.Property(e => e.UpdatedAt);

```

```

        entity.Property(e =>
e.ReviewCount).IsRequired().HasDefaultValue(0);
        entity.Property(e =>
e.AverageBeautifulRating).HasColumnType("decimal(3,2)");
        entity.Property(e =>
e.AverageDifficultyRating).HasColumnType("decimal(3,2)");
        entity.HasOne(e => e.ParentSong).WithMany(e =>
e.ChildSongs).HasForeignKey(e =>
e.ParentSongId).OnDelete(DeleteBehavior.Restrict);
        entity.HasOne(e => e.Structure).WithOne(e =>
e.Song).HasForeignKey<SongStructure>(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
        entity.HasMany(e => e.Comments).WithOne(e =>
e.Song).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
        entity.HasMany(e => e.Reviews).WithOne(e =>
e.Song).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
        entity.HasMany(e => e.SongChords).WithOne(e =>
e.Song).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
        entity.HasMany(e => e.SongPatterns).WithOne(e =>
e.Song).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
        entity.HasMany(e => e.SongAlbums).WithOne(e =>
e.Song).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
        entity.HasMany(e => e.ChildSongs).WithOne(e =>
e.ParentSong).HasForeignKey(e =>
e.ParentSongId).OnDelete(DeleteBehavior.Restrict);
        entity.HasIndex(e => e.OwnerId);
        entity.HasIndex(e => e.ParentSongId);
        entity.HasIndex(e => e.IsPublic);
        entity.HasIndex(e => e.Title);
        entity.HasIndex(e => e.Genre);
        entity.HasIndex(e => e.Theme);
        entity.HasIndex(e => e.CreatedAt); });
    modelBuilder.Entity<Chord>(entity =>{
        entity.HasKey(e => e.Id);
        entity.Property(e =>
e.Name).IsRequired().HasMaxLength(50);
        entity.Property(e =>
e.Fingering).IsRequired().HasMaxLength(100);
        entity.Property(e =>
e.Description).HasMaxLength(500);
        entity.Property(e => e.CreatedAt)
.IsRequired();
        entity.Property(e => e.UpdatedAt);
        entity.HasOne(e => e.CreatedBy).WithMany(e =>
e.Chords).HasForeignKey(e =>
e.CreatedByUserId).OnDelete(DeleteBehavior.Restrict);

```



```

entity.HasMany(e => e.SongChords).WithOne(e =>
e.Chord).HasForeignKey(e =>
e.ChordId).OnDelete(DeleteBehavior.Cascade);
        entity.HasIndex(e => e.CreatedByUserId);
        entity.HasIndex(e => e.Name); });
        modelBuilder.Entity<StrummingPattern>(entity =>{
            entity.HasKey(e => e.Id);
            entity.Property(e =>
e.Name).IsRequired().HasMaxLength(100);
            entity.Property(e =>
e.Pattern).IsRequired().HasMaxLength(500);
            entity.Property(e =>
e.Description).HasMaxLength(1000);
            entity.Property(e =>
e.IsFingerStyle).IsRequired().HasDefaultValue(false);
            entity.Property(e => e.CreatedAt).IsRequired();
            entity.Property(e => e.UpdatedAt);
            entity.HasOne(e => e.CreatedBy).WithMany(e =>
e.StrummingPatterns).HasForeignKey(e =>
e.CreatedByUserId).OnDelete(DeleteBehavior.Restrict);
            entity.HasMany(e => e.SongPatterns).WithOne(e =>
e.StrummingPattern).HasForeignKey(e =>
e.StrummingPatternId).OnDelete(DeleteBehavior.Cascade);
            entity.HasIndex(e => e.CreatedByUserId);
            entity.HasIndex(e => e.Name);
            entity.HasIndex(e => e.IsFingerStyle);});
        modelBuilder.Entity<SongStructure>(entity =>{
            entity.HasKey(e => e.Id);
            entity.Property(e => e.SongId).IsRequired();
            entity.HasOne(e => e.Song).WithOne(e =>
e.Structure).HasForeignKey<SongStructure>(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
            entity.HasIndex(e => e.SongId)
                .IsUnique();});
        modelBuilder.Entity<SongSegment>(entity =>{
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Lyric).HasMaxLength(1000);
            entity.Property(e =>
e.Description).HasMaxLength(500);
            entity.Property(e => e.Color).HasMaxLength(50);
            entity.Property(e =>
e.BackgroundColor).HasMaxLength(50);
            entity.Property(e =>
e.ContentHash).IsRequired().HasMaxLength(100);
            entity.Property(e =>
e.Type).IsRequired().HasConversion<string>().HasMaxLength(20);
            entity.Property(e => e.Duration);
            entity.HasOne(e =>
e.Chord).WithMany().HasForeignKey(e =>
e.ChordId).OnDelete(DeleteBehavior.SetNull);

```

```

        entity.HasOne(e =>
e.Pattern).WithMany().HasForeignKey(e =>
e.PatternId).OnDelete(DeleteBehavior.SetNull);
        entity.HasMany(e => e.Positions)
            .WithOne(e => e.Segment)
            .HasForeignKey(e => e.SegmentId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasMany(e => e.Comments)
            .WithOne(e => e.Segment)
            .HasForeignKey(e => e.SegmentId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasIndex(e => e.ContentHash);
        entity.HasIndex(e => e.Type);
        entity.HasIndex(e => e.ChordId);
        entity.HasIndex(e => e.PatternId);
        entity.HasIndex(e => new { e.ChordId,
e.PatternId});});});
        modelBuilder.Entity<SongSegmentPosition>(entity =>
        {
            entity.HasKey(e => e.Id);
            entity.Property(e => e.SongId).IsRequired();

            entity.Property(e => e.SegmentId).IsRequired();
            entity.Property(e => e.PositionIndex).IsRequired();
            entity.Property(e =>
e.RepeatGroup).HasMaxLength(100);
            entity.HasOne(e => e.Segment).WithMany(e =>
e.Positions).HasForeignKey(e =>
e.SegmentId).OnDelete(DeleteBehavior.Cascade);
            entity.HasIndex(e => new { e.SongId,
e.PositionIndex }).IsUnique();
            entity.HasIndex(e => e.PositionIndex);
            entity.HasIndex(e => e.SegmentId);
            entity.HasIndex(e => e.RepeatGroup);});});
        modelBuilder.Entity<SongReview>(entity =>{
            entity.HasKey(e => e.Id);
            entity.Property(e => e.SongId).IsRequired();
            entity.Property(e => e.UserId).IsRequired();
            entity.Property(e =>
e.ReviewText).IsRequired().HasMaxLength(2000);
            entity.Property(e =>
e.BeautifulLevel).HasColumnType("int");
            entity.Property(e =>
e.DifficultyLevel).HasColumnType("int");
            entity.Property(e => e.CreatedAt).IsRequired();
            entity.Property(e => e.UpdatedAt);
            entity.HasOne(e => e.User).WithMany(e =>
e.Reviews).HasForeignKey(e =>
e.UserId).OnDelete(DeleteBehavior.Restrict);
            entity.HasOne(e => e.Song).WithMany(e =>
e.Reviews).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);

```

```

        entity.HasIndex(e => new { e.UserId, e.SongId
    }).IsUnique();
        entity.HasIndex(e => e.SongId);
        entity.HasIndex(e => e.UserId);
        entity.HasIndex(e => e.CreatedAt);});
    modelBuilder.Entity<SongPattern>(entity =>{
        entity.HasKey(e => e.Id);
        entity.Property(e => e.SongId).IsRequired();
        entity.Property(e =>
e.StrummingPatternId).IsRequired();
        entity.HasOne(e => e.Song).WithMany(e =>
e.SongPatterns).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(e => e.StrummingPattern).WithMany(e
=> e.SongPatterns).HasForeignKey(e =>
e.StrummingPatternId).OnDelete(DeleteBehavior.Cascade);
        entity.HasIndex(e => new { e.SongId,
e.StrummingPatternId }).IsUnique();});
    modelBuilder.Entity<SongChord>(entity =>{
        entity.HasKey(e => e.Id);
        entity.Property(e => e.SongId).IsRequired();
        entity.Property(e => e.ChordId).IsRequired();
        entity.HasOne(e => e.Song).WithMany(e =>
e.SongChords).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(e => e.Chord).WithMany(e =>
e.SongChords).HasForeignKey(e =>
e.ChordId).OnDelete(DeleteBehavior.Cascade);
        entity.HasIndex(e => new { e.SongId, e.ChordId
    }).IsUnique();});
    modelBuilder.Entity<SongComment>(entity =>{
        entity.HasKey(e => e.Id);
        entity.Property(e => e.UserId).IsRequired();
        entity.Property(e => e.SongId).IsRequired();
        entity.Property(e =>
e.Text).IsRequired().HasMaxLength(1000);
        entity.Property(e => e.CreatedAt).IsRequired();
        entity.HasOne(e => e.User).WithMany(e =>
e.Comments).HasForeignKey(e =>
e.UserId).OnDelete(DeleteBehavior.Restrict);
        entity.HasOne(e => e.Song).WithMany(e =>
e.Comments).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(e => e.Segment).WithMany(e =>
e.Comments).HasForeignKey(e =>
e.SegmentId).OnDelete(DeleteBehavior.Restrict);
        entity.HasIndex(e => e.SongId);
        entity.HasIndex(e => e.UserId);
        entity.HasIndex(e => e.SegmentId);
        entity.HasIndex(e => e.CreatedAt);});
    modelBuilder.Entity<Album>(entity =>{
        entity.HasKey(e => e.Id);

```

```

        entity.Property(e =>
e.Title).IsRequired().HasMaxLength(200);
        entity.Property(e =>
e.CoverUrl).HasMaxLength(10000);
        entity.Property(e =>
e.Description).HasMaxLength(2000);
        entity.Property(e => e.Genre).HasMaxLength(100);
        entity.Property(e => e.Theme).HasMaxLength(100);
        entity.Property(e =>
e.IsPublic).IsRequired().HasDefaultValue(false);
        entity.Property(e => e.CreatedAt).IsRequired();
        entity.Property(e => e.UpdatedAt);
        entity.HasOne(e =>
e.Owner).WithMany().HasForeignKey(e =>
e.OwnerId).OnDelete(DeleteBehavior.Restrict);
        entity.HasMany(e => e.SongAlbums).WithOne(e =>
e.Album).HasForeignKey(e =>
e.AlbumId).OnDelete(DeleteBehavior.Cascade);
        entity.HasMany(e => e.Subscribers).WithOne(e =>
e.TargetAlbum).HasForeignKey(e =>
e.TargetAlbumId).OnDelete(DeleteBehavior.Restrict);
        modelBuilder.Entity<SongAlbum>(entity =>{
            entity.HasKey(e => e.Id);
            entity.Property(e => e.AlbumId).IsRequired();
            entity.Property(e => e.SongId).IsRequired();
            entity.HasOne(e => e.Album).WithMany(e =>
e.SongAlbums).HasForeignKey(e =>
e.AlbumId).OnDelete(DeleteBehavior.Cascade);
            entity.HasOne(e => e.Song).WithMany(e =>
e.SongAlbums).HasForeignKey(e =>
e.SongId).OnDelete(DeleteBehavior.Cascade);
            entity.HasIndex(e => e.AlbumId);
            entity.HasIndex(e => e.SongId);});
        modelBuilder.Entity<Notification>(entity =>{
            entity.HasKey(e => e.Id);
            entity.Property(e => e.UserId).IsRequired();
            entity.Property(e => e.Type)
.IsRequired().HasConversion<string>().HasMaxLength(50);
            entity.Property(e =>
e.Message).IsRequired().HasMaxLength(500);
            entity.Property(e => e.CreatedAt).IsRequired();
            entity.Property(e =>
e.IsRead).IsRequired().HasDefaultValue(false);
            entity.HasOne(e => e.User).WithMany(u =>
u.Notifications).HasForeignKey(e =>
e.UserId).OnDelete(DeleteBehavior.Restrict);
            entity.HasIndex(e => e.UserId);
            entity.HasIndex(e => e.CreatedAt);
            entity.HasIndex(e => e.IsRead);
            entity.HasIndex(e => e.Type); });});

```

ПРИЛОЖЕНИЕ Б

```

[HttpPost]
[Authorize]
public async Task<ActionResult<ChordDto>>
CreateChord([FromBody] CreateChordDto dto) {
    try{
        var userId = GetCurrentUser();
        var command = new CreateChordCommand(
            dto.Name,
            dto.Fingering,
            userId,
            dto.Description);
        var result = await _mediator.Send(command);
        return CreatedAtAction(nameof(GetChordById), new { id =
result.Id }, result);
    } catch (InvalidOperationException ex) {
        return Conflict(new { error = ex.Message });
    } catch (Exception ex) {
        return BadRequest(new { error = ex.Message });
    }
}

[HttpPut("{id}")]
[Authorize]
public async Task<ActionResult<ChordDto>> UpdateChord(Guid id,
[FromBody] UpdateChordDto dto) {
    try{
        var userId = GetCurrentUser();
        var command = new UpdateChordCommand(id, userId,
dto.Name, dto.Fingering, dto.Description);
        var result = await _mediator.Send(command);
        return Ok(result);
    } catch (KeyNotFoundException ex) {
        return NotFound(new { error = ex.Message });
    } catch (Exception ex) {
        return BadRequest(new { error = ex.Message });
    }
}

[HttpDelete("{id}")]
[Authorize]
public async Task<ActionResult> DeleteChord(Guid id) {
    try{
        var userId = GetCurrentUser();
        var userRole = GetCurrentUserRole();
        var command = new DeleteChordCommand(id, userId,
userRole);
        await _mediator.Send(command);
        return NoContent();
    } catch (KeyNotFoundException ex) {
        return NotFound(new { error = ex.Message });
    } catch (UnauthorizedAccessException) {
        return Forbid();
    } catch (Exception ex){
        return BadRequest(new { error = ex.Message });
    }
}

```

ПРИЛОЖЕНИЕ В

```

[HttpPost] [Authorize]
public async Task<ActionResult<StrummingPatternsDto>>
CreatePattern([FromBody] CreatePatternDto dto) {
    try{
        var userId = GetCurrentUser();
        var command = new CreatePatternCommand(dto.Name,
        dto.Pattern, dto.IsFingerStyle, userId, dto.Description);
        var result = await _mediator.Send(command);
        return CreatedAtAction(nameof(GetStrummingPatternById),
new { id = result.Id }, result);
    } catch (InvalidOperationException ex) {
        return Conflict(new { error = ex.Message });
    } catch (Exception ex) {
        return BadRequest(new { error = ex.Message });
    }
}

[HttpPut("{id}")] [Authorize]
public async Task<ActionResult<StrummingPatternsDto>>
UpdatePattern(Guid id, [FromBody] UpdatePatternDto dto) {
    try{
        var userId = GetCurrentUser();
        var command = new UpdatePatternCommand(id, userId,
        dto.Name, dto.Pattern, dto.IsFingerStyle, dto.Description);
        var result = await _mediator.Send(command);
        return Ok(result);
    } catch (KeyNotFoundException ex) {
        return NotFound(new { error = ex.Message });
    } catch (UnauthorizedAccessException) {
        return Forbid();
    } catch (InvalidOperationException ex) {
        return Conflict(new { error = ex.Message });
    } catch (Exception ex) {
        return BadRequest(new { error = ex.Message });
    }
}

[HttpDelete("{id}")] [Authorize]
public async Task<ActionResult> DeletePattern(Guid id) {
    try{
        var userId = GetCurrentUser();
        var userRole = GetCurrentUserRole();
        var command = new DeletePatternCommand(id, userId,
        userRole);
        await _mediator.Send(command);
        return NoContent();
    } catch (KeyNotFoundException ex) {
        return NotFound(new { error = ex.Message });
    } catch (UnauthorizedAccessException) {
        return Forbid();
    } catch (Exception ex) {
        return BadRequest(new { error = ex.Message });
    }
}

```

ПРИЛОЖЕНИЕ Г

```

[HttpPost] [Authorize]
public async Task<ActionResult<SongDto>> CreateSong([FromBody]
CreateSongDto dto) {
    try{
        var userId = GetCurrentUserId();
        var command = new CreateSongCommand(userId, dto.Title,
dto.Genre, dto.Theme, dto.AudioBase64, dto.AudioType,
dto.Artist, dto.Description, dto.IsPublic, dto.ParentSongId,
dto.CustomAudioUrl, dto.CustomAudioType);
        var result = await _mediator.Send(command);
        return CreatedAtAction(nameof(GetSongById), new { id =
result.Id }, result);    }
    catch (ArgumentException ex) {
        return BadRequest(new { error = ex.Message });}}
[HttpPut("{id}")] [Authorize]
public async Task<ActionResult<SongDto>> UpdateSong(
Guid id,
[FromBody] UpdateSongDto dto) {
    try{
        var userId = GetCurrentUserId();
        var userRole = GetCurrentUserRole();
        var command = new UpdateSongCommand(id, userId,
userRole, dto);
        var result = await _mediator.Send(command);
        return Ok(result); }
    catch (KeyNotFoundException) {
        return NotFound(new { error = "Song not found" });}
    catch (UnauthorizedAccessException) {
        return Forbid();}}
[HttpDelete("{id}")] [Authorize]
public async Task<ActionResult> DeleteSong(Guid id) {
    try    {
        var userId = GetCurrentUserId();
        var userRole = GetCurrentUserRole();
        var command = new DeleteSongCommand(id, userId,
userRole);
        await _mediator.Send(command);
        return NoContent();
    }
    catch (KeyNotFoundException)
    {
        return NotFound(new { error = "Song not found" });}
    }
    catch (UnauthorizedAccessException)
    {
        return Forbid();
    }
}

```

ПРИЛОЖЕНИЕ Д

```

[HttpPost("{id}/audio")] [Authorize]
[RequestSizeLimit(100_000_000)] [DisableRequestSizeLimit]
public async Task<ActionResult> UploadAudio(Guid id, IFormFile
audioFile, CancellationToken cancellationToken) {try{
    var userId = GetCurrentUserId();
    var song = await _unitOfWork.Songs.GetByIdAsync(id,
cancellationToken);
if (song == null) {
logger.LogWarning("Song not found: {SongId}", id);
    return NotFound(new { error = "Song not found" });}
    if (song.OwnerId != userId) {
        _logger.LogWarning("User {UserId} not authorized
to upload audio for song {SongId}", userId, id);
        return Forbid();}
    if (audioFile == null || audioFile.Length == 0) {
        return BadRequest(new { error = "No audio file provided" });}
    if (audioFile.Length > 100_000_000) {
return BadRequest(new { error = "File too large. Maximum size
is 100MB" });}
    var allowedTypes = new[]{};
    if (!allowedTypes.Contains(audioFile.ContentType)) {
logger.LogWarning("Unsupported audio type: {ContentType}",
audioFile.ContentType);
        return BadRequest(new { error = $"Unsupported
audio type: {audioFile.ContentType}. Supported: mp3, wav, ogg,
m4a, aac, flac, opus, webm" });}string fileExtension;
    if (!string.IsNullOrEmpty
(Path.GetExtension(audioFile.FileName))) {
        fileExtension =
Path.GetExtension(audioFile.FileName); }
    else{fileExtension = audioFile.ContentType switch{}};
    var fileName = $"{song.Id}{fileExtension}";
    _logger.LogInformation("Uploading audio for song
{SongId}: {FileName}, Size: {Size} bytes, Type: {ContentType}",
song.Id, fileName, audioFile.Length,
audioFile.ContentType);
    using var stream = audioFile.OpenReadStream();
    var uploadedFileName = await
_webDavService.UploadAudioAsync(stream, fileName, song.Id);
song.Update(customAudioUrl: uploadedFileName, customAudioType:
audioFile.ContentType); await
_unitOfWork.Songs.UpdateAsync(song, cancellationToken);
await _unitOfWork.SaveChangesAsync(cancellationToken);
    _logger.LogInformation("Audio uploaded successfully
for song {SongId}: {FileName}",
song.Id, uploadedFileName);
    return Ok(new{fileName = uploadedFileName,
audioType = audioFile.ContentType,size = audioFile.Length});}
    return StatusCode(500, new { error = "Failed to
upload audio", details = ex.Message });}}

```


ПРИЛОЖЕНИЕ Е

```
[HttpPost]
public async Task<ActionResult<AlbumDto>>
CreateAlbum([FromBody] CreateAlbumDto dto) {
    try{
        var userId = GetCurrentUser();
        var user = await _userService.GetByIdAsync(userId);
        if (!user.CanCreateAlbum()) {
            return Forbid("Only Premium users can create albums. Upgrade to Premium!");
        }
        var command = new CreateAlbumCommand(userId,
            dto.Title, dto.IsPublic, dto.Genre, dto.Theme, dto.CoverUrl,
            dto.Description);
        var result = await _mediator.Send(command);
        return CreatedAtAction(nameof(GetAlbumById), new { id
            = result.Id }, result);
    }
    catch (InvalidOperationException ex) {
        return Conflict(new { error = ex.Message });
    }
}

[HttpPut("{id}")]
public async Task<ActionResult<AlbumDto>> UpdateAlbum(Guid id,
[FromBody] UpdateAlbumDto dto) {
    try{
        var userId = GetCurrentUser();
        var command = new UpdateAlbumCommand(userId: userId,
            albumId: id, title: dto.Title, coverBase64: dto.CoverUrl,
            description: dto.Description, isPublic: dto.IsPublic, genre:
            dto.Genre, theme: dto.Theme);
        var result = await _mediator.Send(command);
        return Ok(result);
    }
    catch (KeyNotFoundException) {
        return NotFound(new { error = "Album not found" });
    }
    catch (UnauthorizedAccessException) {
        return Forbid();
    }
    catch (InvalidOperationException ex) {
        return Conflict(new { error = ex.Message });
    }
}

[HttpDelete("{id}")]
public async Task<ActionResult> DeleteAlbum(Guid id) {
    try{
        var userId = GetCurrentUser();
        var command = new DeleteAlbumCommand(userId, id);
        await _mediator.Send(command);
        return NoContent();
    }
    catch (KeyNotFoundException) {
        return NotFound(new { error = "Album not found" });
    }
    catch (UnauthorizedAccessException) {
        return Forbid();
    }
    catch (InvalidOperationException ex) {
        return BadRequest(new { error = ex.Message });
    }
}
```